SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Cycle de formation des ingénieurs en Télécommunications

*Option :*

Management de l'Innovation Technologique (MIT)

# Rapport de projet de fin d'études

*Thème :*

# *Système de surveillance de l'intérieur du véhicule*


*Réalisé par :*

**Abir AMARA**

*Encadrants :*

Mme. Ferdaous CHAABENE
Mme. Sawcen GHEDIFI


*Travail proposé et réalisé en collaboration avec*

Sagemcom

Année universitaire : 2020 /2021

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Sawcen GHEDIFI**

**Signature**

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Ferdaous CHAABENE**

**Signature**

# Dédicaces

*Le cœur plein de joie, je dédie ce travail qui a exigé de moi tant d'efforts :*

*A mon très cher papa dont les sacrifices et les encouragements ont fait de moi la personne que je suis actuellement et dont tu en es fier. Merci pour tout l'amour que tu me voues et ta tendresse que tu me prodigués.*

*A ma merveilleuse maman, merci pour tout ce que tu as fait pour moi, pour ton amour, pour ta patience et pour ta tendresse dont je reste reconnaissante.*

*A mon frère et mes sœurs, les meilleures personnes sur qui je peux compter en cas de besoin, merci pour votre soutien moral et encouragement multiples.*

*A tous mes proches, mes biens aimés, qui m'ont encouragé pour faire de moi l'être le plus chanceux.*
*A tous ceux qui nous ont chers et à qui je dois ma reconnaissance.*
*Je vous remercie tous et que ce travail soit le témoignage de mon amour et mon immense reconnaissance.*

*AMARA Abir*

# Remerciements

Chercher les mots qu'il faut pour exprimer ma gratitude envers mes maîtres et formateurs n'est pas une entreprise facile car, outre les émotions qui me submergent à l'heure où j'écris, la crainte de ne pas leur rendre justice, eu égard à leurs efforts et dévouements dans l'exercice de leur Savoir, me pousse à m'excuser d'avance.

J'exprime ma parfaite reconnaissance et mes sincères remerciements à mon encadrante académique, Mme **Ferdaous CHAABENE**, pour le temps qu'elle a bien voulu consacrer au suivi de mon travail, le partage de l'expertise qu'elle effectuait au quotidien, son engagement à me mettre sur la bonne voie et ses conseils dans l'élaboration de ce rapport.

Mes remerciements vont de même à Mme **Wiem CHEBBI** et Mme **Sawcen GHEDIFI** pour l'accueil qu'elles m'ont réservé, le temps qu'elles m'ont accordé et et l'amabilité avec laquelle j'ai été reçue, si bien que ses aides au quotidien m'a permis de m'accomplir dans mon projet de fin d'études.

Mes sincères remerciements iront également à tout le cadre pédagogique et administratif l'Ecole Supérieure des Communications de Tunis - **SUP'COM** pour la qualité de l'enseignement qu'ils me prodiguaient afin de me procurer d'une formation de haut niveau.

Je tiens à témoigner toute ma reconnaissance au personnel de l'entreprise **SAGEMCOM Software & Technologies**, pour l'expérience enrichissante et pleine d'intérêt qu'il m'a fait vivre durant ces six mois.

Finalement, je profite de cette occasion pour remercier pareillement les membres du jury qui ont accepté d'évaluer mon projet de fin d'études en espérant qu'ils trouvent dans ce rapport les qualités de clarté et de motivation qu'ils attendent.

# Résumé

Le présent rapport synthétise le travail effectué dans le cadre du ce projet de fin d'études en ingénierie de Télécommunications. Ce dernier a été effectué au sein de l'entreprise SAGEMCOM Software & Technologies. L'objectif de ce travail consiste à développer un système de surveillance de l'intérieur de véhicule basé sur l'apprentissage profond. Le travail intitulé « Système de surveillance de l'intérieur du véhicule » consiste à développer un système de surveillance de l'ensemble de l'environnement cabine en traitant des séquences vidéo captées en temps réel par une caméra frontale. Ce système permet premièrement la détection d'objets et d'occupants ainsi que leur comptage. Deuxièmement, il classifie les états de distraction du conducteur et finalement, il assure la reconnaissance faciale du conducteur. L'ensemble de ce système a été intégré et déployé dans une puissante carte embarquée qui supporte l'intelligence artificielle.

**Mots clés** : Détection d'objets, Distraction du conducteur, Reconnaissance faciale, Apprentissage profond, Réseaux de neurones convolutifs, Apprentissage par transfert, Nvidia Jetson Nano.

# Abstract

This report summarizes the work carried out within the framework of this end of studies project in Telecommunication Engineering. The latter was established within the company SAGEMCOM Software & Technologies. The objective of this work is to develop a vehicle interior monitoring system based on deep learning. The work entitled « Vehicle interior surveillance system » consists on developing a surveillance system of the entire cabin environment by processing video sequences captured in real time by a front camera. At first, this system performs objects and occupants detection as well as their counting. Second, it classifies driver distraction states and finally it ensures driver facial recognition. This whole system has been integrated and deployed in a powerful on-board card that supports artificial intelligence.

**Keywords** : Object detection, Driver distraction, Face Recognition, Deep learning, Convolutional neural networks, Transfer learning, Nvidia Jetson Nano card.

# Table des matières

**Introduction générale** **1**

**1 Positionnement du projet dans son contexte** **3**

Introduction . . . 4

1.1 Cadre général . . . 4

1.2 Présentation de l'organisme d'accueil . . . 4

1.2.1 Présentation du groupe SAGEMCOM . . . 4

1.2.2 SAGEMCOM Software & Technologies (SS&T) . . . 5

1.3 Présentation du projet . . . 7

1.3.1 Problématique . . . 7

1.3.2 Solution proposée . . . 7

1.4 Méthode de gestion de projet . . . 8

1.4.1 Présentation de la méthode CRISP-DM . . . 9

Conclusion . . . 10

**2 Etude Théorique** **11**

Introduction . . . 12

2.1 Théorie de l'apprentissage profond . . . 12

2.2 Vision par ordinateur . . . 18

2.3 Réseaux de neurones convolutifs (CNN) . . . 18

2.4 Apprentissage par transfert . . . 22

2.4.1 Convnet comme extracteur de caractéristique fixe . . . 23

2.4.2 Ajuster les paramètres du Convnet (Fine tuning) . . . 23

2.4.3 Les types de l'apprentissage par transfert . . . 23

2.5 Technologies utilisées pour la détection des occupants et des objets . . . 24

2.5.1 Regional Convolutional Neural Network (R-CNN) . . . 26

2.5.2 Single Shot MultiBox Detector (SSD) . . . 27

2.5.3 You Only Look Once(YOLO) . . . 28

2.5.4 Métrique d'évaluation . . . 31

2.6 Technologies utilisées pour la classification de l'état de distraction du conducteur . . 32
2.6.1 Visual Geometric Group (VGG) . . . . . 33
2.6.2 Residual Networks 50 (ResNet 50) . . . . . 35
2.6.3 MobilNet . . . . . 37
2.6.4 Métrique d'évaluation . . . . . 38
2.7 Technologies utilisées pour la reconnaissance facile du conducteur . . . . . 39
2.7.1 Détection de visage . . . . . 39
2.7.2 Extraction des caractéristiques faciales . . . . . 41
2.7.3 Classification des visages . . . . . 43
Conclusion . . . . . 45

**3 Implémentation des solutions OMS 46**

Introduction . . . . . 47
3.1 Présentation de l'environnement du travail . . . . . 47
3.1.1 Environnements matériels de travail . . . . . 47
3.1.2 Environnement logiciel de travail . . . . . 48
3.2 Détection des occupants et des objets . . . . . 49
3.2.1 Collecte des données . . . . . 49
3.2.2 Annotation des données . . . . . 50
3.2.3 Augmentation des données . . . . . 52
3.2.4 Modélisation des détecteurs des occupants et des objets . . . . . 55
3.2.5 Résultat et évaluation des modèles . . . . . 58
3.2.6 Choix du modèle . . . . . 60
3.3 La classification de l'état de distraction du conducteur . . . . . 63
3.3.1 Collecte des données . . . . . 63
3.3.2 Préparation des données . . . . . 64
3.3.3 Entrainement et évaluation . . . . . 66
3.4 La reconnaissance faciale du conducteur . . . . . 79
3.4.1 Préparation de la base de données . . . . . 79
3.4.2 Détection de visage . . . . . 80
3.4.3 Extraction des caractéristiques . . . . . 81
3.4.4 Classification des visages . . . . . 82

3.5 L'intégration des différentes parties du système de surveillance de l'intérieur de la voiture . . . . . 84
3.6 Déploiement de la solution développée . . . . . 86
3.6.1 Evaluation des performances de la carte . . . . . 88
Conclusion . . . . . 89

**Bibliographie** **91**

# Table des figures


1.1 Logo de SAGEMCOM . . . . . 4
1.2 Centres de Recherche et Développement de SAGEMCOM . . . . . 5
1.3 Structure de SAGEMCOM Software et Technologies . . . . . 6
1.4 Modèle de processus CRISP-DM . . . . . 9

2.1 Comparaison entre un neurone humain (à gauche) et le modèle mathématique d'un neurone artificiel (à droite) . . . . . 12
2.2 Réseaux de neurones avec deux couches cachées. . . . . 13
2.3 Fonction sigmoid (à gauche) et sa dérivée (à droite) . . . . . 15
2.4 Calcul de gradient avec le chain rule . . . . . 16
2.5 Architecture du CNN . . . . . 19
2.6 Produit de convolution . . . . . 19
2.7 Illustration du principe de max pooling . . . . . 20
2.8 Caractéristiques extraits d'un visage . . . . . 20
2.9 Résultat de la fonction Softmax . . . . . 21
2.10 Principe de l'apprentissage par transfert . . . . . 22
2.11 Exemple de détection des occupants et des objets dans une voiture . . . . . 25
2.12 Composants du Faster RCNN . . . . . 26
2.13 Architecture globale de Faster-RCNN. . . . . 27
2.14 Architecture SSD VGG16 . . . . . 28
2.15 Processus de détection d'objets pour YOLO . . . . . 29
2.16 Architecture YOLOv4 . . . . . 30
2.17 Architecture de base finale de YOLOv4 . . . . . 30
2.18 L'intersection over Union . . . . . 31
2.19 Architecture VGG16 . . . . . 34
2.20 Les couches de vgg16 et vgg19 . . . . . 34
2.21 Le modèle Resnet50. . . . . 35
2.22 Les stages du modèle Resnet50. . . . . 35
2.23 Architecture du ResNet50. . . . . 36

2.24 Les convolutions séparables utilisées dans l'architecture MobilNet. . . . . 37
2.25 Forme du matrice de confusion . . . . . 39
2.26 Schéma de l'algorithme de HOG. . . . . 41
2.27 Illustration de l'architecture de MMOD . . . . . 41
2.28 128 masques de visage . . . . . 42
2.29 Classification par la distance euclidienne . . . . . 44
2.30 Séparation entre classe par un hyper plan . . . . . 44

3.1 Une portion la base de données collectée . . . . . 50
3.2 Annotation format YOLO . . . . . 51
3.3 Étiqueter les données à l'aide de LabelImg . . . . . 52
3.4 Annotation format PascalVOC . . . . . 52
3.5 Deux images générées de la même photo . . . . . 53
3.6 Data augmentation par roboflow . . . . . 53
3.7 Correction des classes des données . . . . . 54
3.8 Répartition de jeu des données . . . . . 54
3.9 Fichiers train.txt et test.txt . . . . . 55
3.10 Fichiers obj.data et obj.names . . . . . 56
3.11 Résultat de conversion des données en fichiers CSV. . . . . 57
3.12 Frame de vidéo n°1. . . . . 58
3.13 Frame de vidéo n°2. . . . . 59
3.14 Frame de vidéo n°3. . . . . 60
3.15 Résultat n°1 . . . . . 61
3.16 Résultat n°2 . . . . . 61
3.17 Résultat n°3 . . . . . 62
3.18 Résultat n°4 . . . . . 62
3.19 Échantillon des vidéos collectés . . . . . 64
3.20 Une portion la base de données collectée et nettoyée . . . . . 65
3.21 Architecture d'un CNN . . . . . 66
3.22 Overfiting . . . . . 69
3.23 Portion de nouvelles images de classe « Smoking ». . . . . 70
3.24 Types d'augmentation des données appliqués. . . . . 70

3.25 Allure des résultats obtenus . . . . . 71
3.26 Les matrices de confusion des 4 modèles . . . . . 72
3.27 Images test 1. . . . . 73
3.28 Résultats erronés . . . . . 74
3.29 Architecture de l'approche fondée sur une cascade de deux modèles. . . . . 75
3.30 Matrices de confusion du modèle 1 et modèle 2. . . . . 77
3.31 Résultats 1 du modéle cascade. . . . . 78
3.32 Résultats 2 du modéle cascade. . . . . 78
3.33 Architecture de modèle reconnaissance faciale. . . . . 79
3.34 Base de données . . . . . 80
3.35 Conversion d'une image en un tableau. . . . . 81
3.36 Vecteur résultant d'extraction des features. . . . . 82
3.37 Matrice de confusion des 3 modèles . . . . . 83
3.38 Résultats de la reconnaissance faciale du conducteur . . . . . 84
3.39 Résultat 1 . . . . . 85
3.40 Résultat 2 . . . . . 85
3.41 Résultat 3 . . . . . 86
3.42 Architecture extérieure de la carte Nvidia Jetson Nano. . . . . 87

# Liste des tableaux


3.1 Caractéristiques de l'orinateur . . . . . 47
3.2 Performances des modèles. . . . . 58
3.3 Résultats vidéo n°1 . . . . . 59
3.4 Résultats vidéo n°2 . . . . . 59
3.5 Résultats vidéo n°3 . . . . . 60
3.6 Types et classes de distraction . . . . . 63
3.7 Résultats CNN . . . . . 67
3.8 Résultats d'entrainement . . . . . 71
3.9 Résultats des Modèle 1 et Modèle 2. . . . . 76
3.10 Comparaison des approches de détection faciale . . . . . 81
3.11 Comparaison entre les scores trouvés . . . . . 83
3.12 Les spécifications techniques de la carte Nvidia Jetson Nano. . . . . 87

# Liste des abréviations

— **ADAS** = **A**dvanced **D**river**A**ssistance **S**ystems

— **CNN** = **C**onvolutional **N**neural **N**etwork

— **DL** = **D**eep **L**earning

— **DMS** = **D**river **M**onitoring **S**ystems

— **FPS** = **F**rame **P**er **S**econd

— **FR** = **F**ace **R**ecognition

— **GPU** = **G**raphics **P**rocessing **U**nit

— **HOG** = **H**istograms **O**riented **G**radients

— **IA** = **I**ntelligence **A**rtificielle

— **OMS** = **O**ccupancy **M**onitoring **S**ystems

— **SGD** = **S**tochastic **G**radient **D**escent

— **SVM** = **S**upport **V**ector **M**achines

— **mAP** = **M**ean **A**verage **P**recision

# Introduction générale

Les progrès technologiques s'accompagnent toutefois de nouvelles opportunités améliorant progressivement le quotidien de l'Homme. Ils ne cessent de lui offrir des solutions efficaces qui répondent à ses différents besoins et lui assurent plus de sécurité.

C'est dans ce cadre que l'intelligence artificielle s'immisce de plus en plus dans l'entreprise avec une omniprésence dans tous les domaines. L'industrie automobile ne fait pas exception. En effet les véhicules autonomes sont l'une des innovations les plus impressionnantes de l'intelligence artificielle, car ils s'apprêtent à révolutionner les moyens de transport partout dans le monde. Le développement poussé de l'IA dans ce domaine pourrait métamorphoser nos habitudes de mobilités, améliorer l'expérience de la conduite, assurer la sécurité du conducteur et éviter tout éventuel accident grâce à des résultats de traitement des images en temps réel.

Dans cette perspective que SAGEMCOM Software & Technologies s'est investie dans l'intégration de l'intelligence artificielle dans tous les domaines plus particulièrement le domaine de l'automobile. Plusieurs applications intégrant la vision par ordinateur et l'intelligence artificielle ont vu le jour afin de veiller sur la sécurité du conducteur. Il existe principalement trois familles d'applications :

— ADAS : s'intéresse à la reconnaissance des objets à l'extérieur de la voiture

— DMS : qui vise à la surveillance de la distraction et la fatigue du conducteur.

— OMS : qui se focalise sur les évènements de sécurité à l'intérieur de la voiture.

C'est dans le contexte des applications OMS que ce projet de stage de fin d'études intitulé « Système de surveillance de l'intérieur du véhicule » se situe.

L'objectif de ce projet est le développement d'un système de surveillance de l'ensemble de l'environnement de la cabine en temps réel. Ce système assure en premier lieu, la détection des objets et des occupants d'une voiture ainsi que leur comptage. En deuxième lieu, il fait la classification des différents états de distraction de conducteur et en troisième lieu, il identifie le conducteur par la reconnaissance faciale.

Le présent rapport s'articule autour de trois chapitres. Le premier présente le cadre général du projet, l'organisme et l'entité d'accueil, la problématique et la solution proposée, ainsi que la méthode de travail suivie. Le deuxième chapitre évoque les notions théoriques et technologiques relatives au projet . Quant au troisième chapitre, il détaille les étapes de réalisation aboutissant à la solution souhaitée ainsi que les étapes de son déploiement dans une carte embarquée. Finalement, le présent rapport est clôturé avec une conclusion générale qui capitalise les différents acquis ainsi que les perspectives envisagées.

*Chapitre 1*

# Positionnement du projet dans son contexte

## Plan


**Introduction** . . . . . 4

**1 Cadre général** . . . . . 4

**2 Présentation de l'organisme d'accueil** . . . . . 4

**3 Présentation du projet** . . . . . 7

**4 Méthode de gestion de projet** . . . . . 8

**Conclusion** . . . . . 10

# Introduction

Dans ce chapitre, nous commençons par une présentation du cadre général de stage et de l'organisme d'accueil. Ensuite, nous situons le projet dans son propre contexte, en identifiant sa problématique et détaillant la solution proposée. Finalement, nous présentons la méthode de travail adoptée tout au long de ce projet.

## 1.1 Cadre général

Ce projet s'inscrit dans le cadre de mon stage de projet de fin d'études afin d'obtenir le diplôme national d'ingénieur en Télécommunications de l'École Supérieure des Communications de Tunis - SUP'COM. Il a été effectué au sein de l'entreprise Sagemcom Software & Technologies (SS&T), que nous commencerons par la présenter.

## 1.2 Présentation de l'organisme d'accueil

Le projet abordé lors de ce stage fait partie des activités du pôle Activité Energie & Télécom au sein de l'entreprise SAGEMCOM Software & Technologies, filiale du groupe SAGEMCOM.

**Figure 1.1:** Logo de SAGEMCOM

### 1.2.1 Présentation du groupe SAGEMCOM

SAGEMCOM [1], est un groupe français implanté dans plus de 40 pays dans le monde. Avec ses 40 ans d'expérience dans le monde des télécoms, il est le leader européen sur le marché des

terminaux communicant à haute valeur ajoutée (décodeurs, box Internet, compteurs électriques, objets connectés...). Le groupe réalise 2,1 milliards d'euros comme chiffre d'affaires, compte 4000 salariés actifs dans plus de 50 pays. Il conçoit, fabrique et livre plus de 31,5 millions de terminaux dans le monde entier. Il opère sur plusieurs secteurs, dont nous citons principalement :

— Broadband (décodeurs, box TV/Internet).

— Smart City (Compteurs intelligents).

— Internet of Things (Objets connectés).

SAGEMCOM vise à devenir l'un des leaders mondiaux des terminaux communicants à forte valeur ajoutée en adressant des marchés dynamiques. Ainsi, il compte être le premier sur le marché à proposer à ses clients des produits intégrants les dernières ruptures technologiques.

### 1.2.2 SAGEMCOM Software & Technologies (SS&T)

SAGEMCOM Software et Technologies, abrégé SS&T et filiale du groupe Sagemcom, est un centre de recherche et de développement fondé le 18 juillet 2005. Il fait partie des sept centres similaires implantés dans les majeurs pays du monde comme la France, les ÉtatsUnis et la Chine. La figure ci-dessous présente les différentes filiales de SAGEMCOM dans le monde.

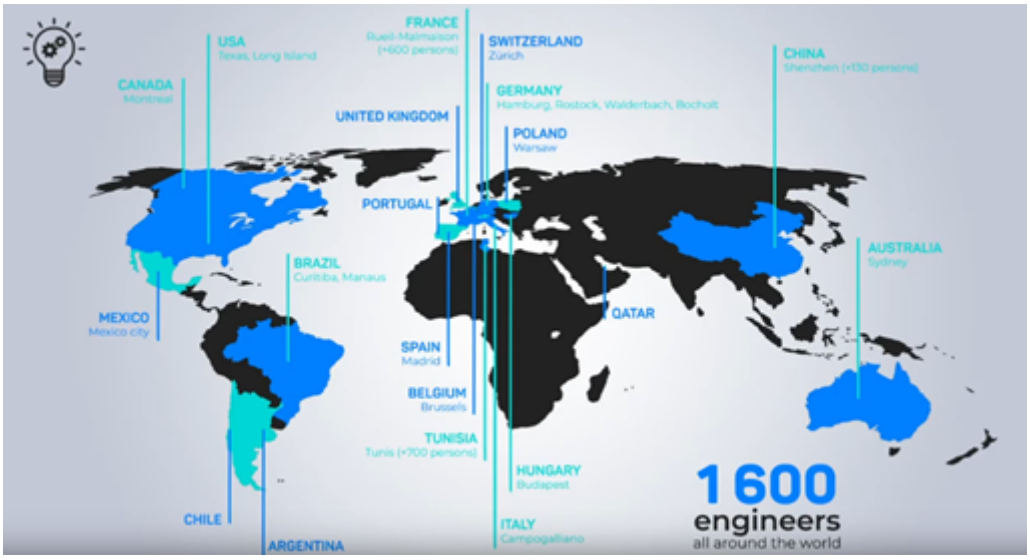


**Figure 1.2:** Centres de Recherche et Développement de SAGEMCOM

Ce centre a pour mission de concevoir, développer et valider des solutions logicielles qui répondent aux besoins du marché de la transmission numérique et de la communication haut débit.

Il comporte trois pôles d'activité et quelques services transverses comme le présente l'organigramme de la figure 1.3.

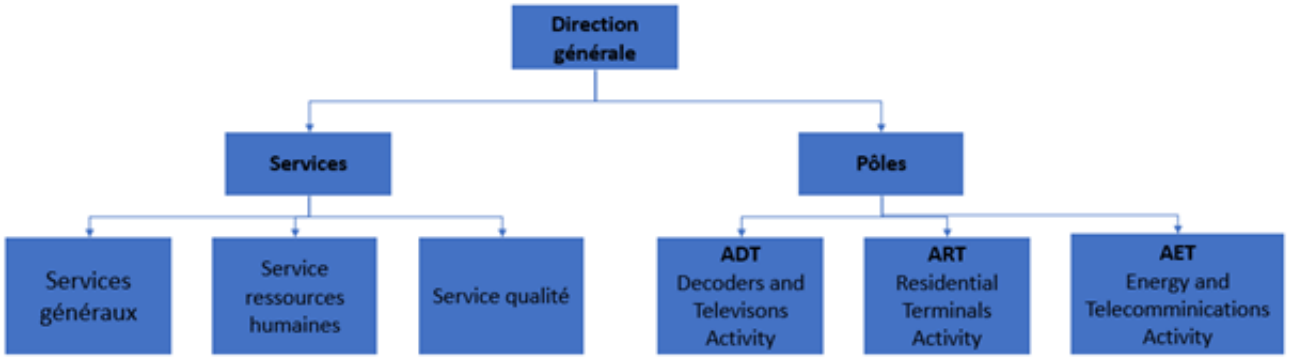


**Figure 1.3:** Structure de SAGEMCOM Software et Technologies

— Le pôle Activités Décodeurs de Télévision numérique (ADT) : Dont les activités sont centralisées sur le développement, l'intégration et le test de solutions logicielles dédiées aux décodeurs TV numérique.

— Le pôle Activités Terminaux Résidentiels (ATR) : Dédié à la conception, le développement et le test de solutions logicielles dédiées aux terminaux résidentiels.

— Le pôle Activités Énergie et Télécoms (AET) : Concentre l'expertise de SAGEMCOM dans les télécommunications et le comptage intelligent, permettant la fourniture de systèmes connectés personnalisés pour les services publics, les opérateurs de télécommunications et des opérateurs de services dans le monde entier. D'ailleurs, ce stage s'est déroulé dans ce pôle au sein de l'équipe innovation compte tenu de la qualité de ses produits et de la force de son expérience et de son capital humain, SAGEMCOM vise à atteindre un grand succès technologique, économique et sociétal afin de maintenir sa position à long terme dans un secteur au sein d'un environnement compétitif. D'ailleurs, afin d'augmenter la compétitivité et de développer sa productivité et surtout pour entrer sur un nouveau marché, SAGEMCOM a commencé à travailler sur l'intelligence artificielle appliquée sur l'automobile, en particulier le système de surveillance intérieure. Elle vise à réaliser des POC (Proof Of Concept) de solutions intelligentes qu'elle fait évoluer vers des produits par la suite.

Grâce aux talents de sa RD et à ses capacités industrielles, SAGEMCOM Energy & Telecom opère dans les marchés du Smart Meter, Smart Grid, Smart Site, Smart Infra et les Smart Services.

La combinaison de ces activités permet de répondre aux besoins croissants des marchés et permet à SAGEMCOM de proposer des solutions efficaces de boutenbout clés en main.

## 1.3 Présentation du projet

### 1.3.1 Problématique

La conduite d'automobile est une tâche complexe, nécessitant l'exécution simultanée de diverses habiletés cognitives, physiques, sensorielles et psychomotrices. Malgré ces complexités, il n'est pas rare d'observer des conducteurs s'engager dans diverses activités non liées à la conduite au volant. Ces activités vont de la conversation avec les passagers, de l'utilisation du téléphone jusqu'à la concentration sur des objets à l'intérieur du véhicule.

Toutes ces activités qui se rivalisent pour attirer l'attention du conducteur pendant la conduite, ont le potentiel de dégrader les performances de conduite et d'avoir de graves conséquences pour la sécurité routière. En effet, des recherches menées par la National Highway Traffic Safety Administration (NHTSA) estiment que l'inattention du conducteur, sous ses diverses formes, contribue à environ 25% des accidents signalés par la police [2].

De ce fait, il est primordial de proposer des solutions pour remédier à cette problématique afin de diminuer les tâches non motrices du conducteur, l'alerter en cas de distraction et l'aider à maintenir une conduite sûre et efficace.

### 1.3.2 Solution proposée

Pour répondre à cette problématique SAGEMCOM a proposé le développement d'un système de surveillance de l'intérieur de toute la voiture où le conducteur n'est plus le seul centre d'intérêt ou attention. En fait, la caméra est positionnée de manière à couvrir tous les sièges dans son champ de vision. Le système peut détecter tous les occupants et peut alors fournir des alertes plus spécifiques à chaque personne dans la voiture.

Ce système basé sur la technologie de l'intelligence artificielle, plus précisément sur

l'apprentissage profond comporte trois parties principales :

— Détection des objets et des occupants : Développement d'un algorithme de détection d'objets afin de fournir des rapports sur le nombre de passagers dans la cabine, présence des bébés, présence de chiens et les objets oubliés dans la voiture à l'aide des techniques de détection d'objets. La technique proposée peut également permettre le comptage des occupants de la voiture et le déclenchement d'alertes de ceinture de sécurité en cas d'absence ou de mauvais port, en ajustant l'environnement de la voiture au nombre des passagers présents.

— Classification de l'états de distraction de conducteur :Développement d'un deuxième algorithme qui permet de classifier l'état de distraction du conducteur (parler au téléphone, boire, fumer...). Cet algorithme assure la détection de la distraction en exploitant caméra de surveillance du conducteur d'angle de vue frontal pour alerter le conducteur à temps réel.

— La reconnaissance faciale du conducteur : Développement d'un troisième algorithme qui permet d'identifier le conducteur par des techniques de reconnaissance faciale. En se basant sur le profil du conducteur enregistré, le système peut ensuite régler automatiquement les paramètres de confort et de commodité personnels tels que la position optimale des sièges et des rétroviseurs, la station de radio préférée ou la température intérieure préférée.

Et pour pouvoir exécuter ces algorithmes en temps réel dans la voiture, nous déployons notre solution dans une carte embarquée qui supporte les algorithmes d'intelligence artificielle "Nvidia Jetson Nano".

## 1.4 Méthode de gestion de projet

L'utilisation d'une méthode de travail est intéressante si nous visons à bien gérer notre projet. Dans la littérature, plusieurs pistes sont intéressantes mais leur adoption varie selon la spécificité des projets. Afin de satisfaire convenablement les besoins de notre projet et fournir des directives pour une exécution organisée et transparente, nous optons pour la méthode CRISP-DM (Cross-Industry Standard Process for Data Mining) qui fournit une approche structurée de la planification d'un projet d'exploration de données.

### 1.4.1 Présentation de la méthode CRISP-DM

CRISP-DM[3], est l'une des méthodes les plus robustes et les plus efficaces pour les projets d'exploration de données, d'analyse et de science des données. Elle a été initialement développée par IBM en 1960. Ce modèle est une séquence idéalisée d'événements. En pratique, de nombreuses tâches peuvent être effectuées dans un ordre différent, et il est souvent nécessaire de revenir à la tâche précédente pour de répéter certaines opérations. Le modèle n'essaie pas de capturer toutes les routes possibles à travers le processus d'exploration de données.Cette méthode comporte six phases illustrées dans la figure suivante.

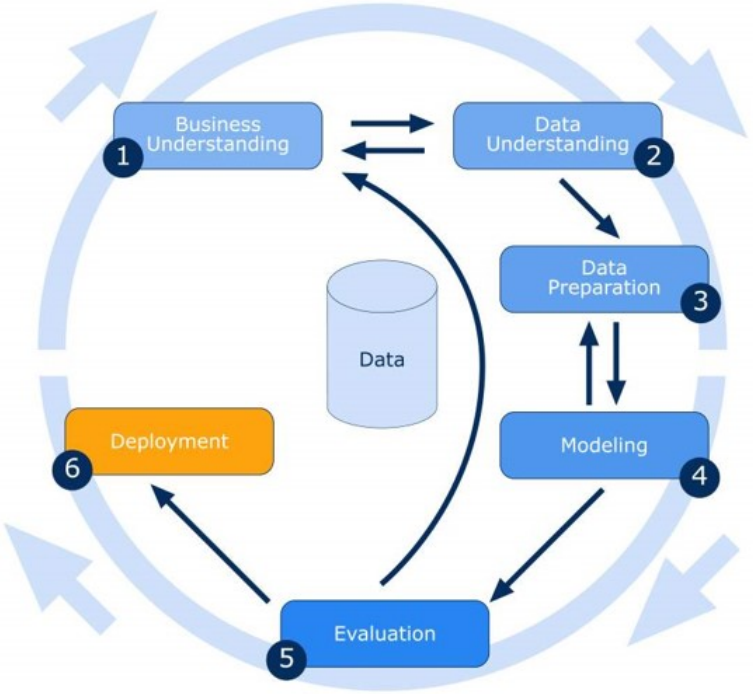


**Figure 1.4:** Modèle de processus CRISP-DM

— La compréhension du problème métier : Il s'agit de la première phase du processus CRISP-DM qui met l'accent sur les facteurs importants, y compris les critères de réussite, la découverte des objectifs et des exigences en matière d'exploration de données, ainsi que les terminologies et les termes techniques.

— La compréhension des données : Ceci est la deuxième phase du processus CRISP-DM qui se concentre sur la collecte de données, la vérification de la qualité et l'exploration des données pour obtenir un aperçu des données et former des hypothèses pour des informations cachées.

— Préparation des données : Il s'agit de la troisième phase du processus CRISP-DM qui se concentre sur la sélection et la préparation des données finales. C'est une phase de nettoyage et de réorganisation des données selon les besoins du projet, elle inclut aussi le recodage des données pour les rendre compatibles avec l'algorithme à utiliser.

— Modélisation : Il s'agit de la quatrième phase du processus de sélection du CRISP-DM et de l'application de diverses techniques de modélisation. Cette phase comprend le choix, le paramétrage et le test de différents algorithmes ainsi que leur enchaînement, qui constitue un modèle.

— Evaluation : C'est la cinquième étape du processus CRISP-DM, le but de cette étape est de valider les modèles ou connaissances acquis pour s'assurer qu'ils répondent aux objectifs fixés au début du processus. À ce stade, des tests seront effectués sur la durabilité et la précision des modèles obtenus.

— Déploiement : C'est la sixième et dernière phase du processus CRISP-DM axée sur la détermination de l'utilisation des connaissances et des résultats obtenus. Son objectif est de placer les connaissances acquises, grâce à la modélisation, sous une forme appropriée et de les intégrer dans le processus décisionnel. Cependant, selon les besoins de l'entreprise, la phase de déploiement peut aller de la simple génération d'un rapport représentant les connaissances acquises jusqu'au développement d'une application permettant l'exploitation du modèle obtenu.

## Conclusion

Tout au long de ce chapitre, le cadre du projet a été posé, le problème et la solution envisagée ainsi que la méthodologie à adopter lors de cette étape ont été définis. A ce stade et afin de mieux appréhender le projet, il est indispensable de réaliser une étude détaillée tout en évoquant les concepts de base relatifs à notre projet.

*Chapitre 2*

# Etude Théorique

## Plan


**Introduction** . . . . . **12**

**1 Théorie de l'apprentissage profond** . . . . . **12**

**2 Vision par ordinateur** . . . . . **18**

**3 Réseaux de neurones convolutifs (CNN)** . . . . . **18**

**4 Apprentissage par transfert** . . . . . **22**

**5 Technologies utilisées pour la détection des occupants et des objets** . . **24**

**6 Technologies utilisées pour la classification de l'état de distraction du conducteur** . . . . . **32**

**7 Technologies utilisées pour la reconnaissance facile du conducteur** . . **39**

**Conclusion** . . . . . **45**

## Introduction

Le but de ce chapitre est de fournir au lecteur une vue d'ensemble des notions fondamentales intervenant lors de la mise en œuvre d'un système de surveillance de l'intérieur de voiture. Dans un premier temps, nous allons présenter l'apprentissage profond. Ensuite, nous définirons les techniques adoptées pour la détection des objets suivie par un aperçu sur les mesures d'évaluation. Dans un deuxième temps, nous expliquerons les techniques utilisées pour la classification de l'état du conducteur ainsi que ces métriques d'évaluation et à la fin nous détaillerons les différentes étapes et techniques de la partie reconnaissance faciale du conducteur.

## 2.1 Théorie de l'apprentissage profond

Le réseau de neurones artificiel est un algorithme d'apprentissage automatique qui s'inspire du cerveau humain. Il s'agit d'un des plus puissants classifieurs maintenant. Il est modélisé mathématiquement par un réseau dans un graphe, plus ou moins complexe, hiérarchique sous forme de couches dont les nœuds élémentaires sont appelés neurones. la figure 2.1 montre la comparaison entre un neurone humain et le modèle mathématique d'un neurone artificiel.

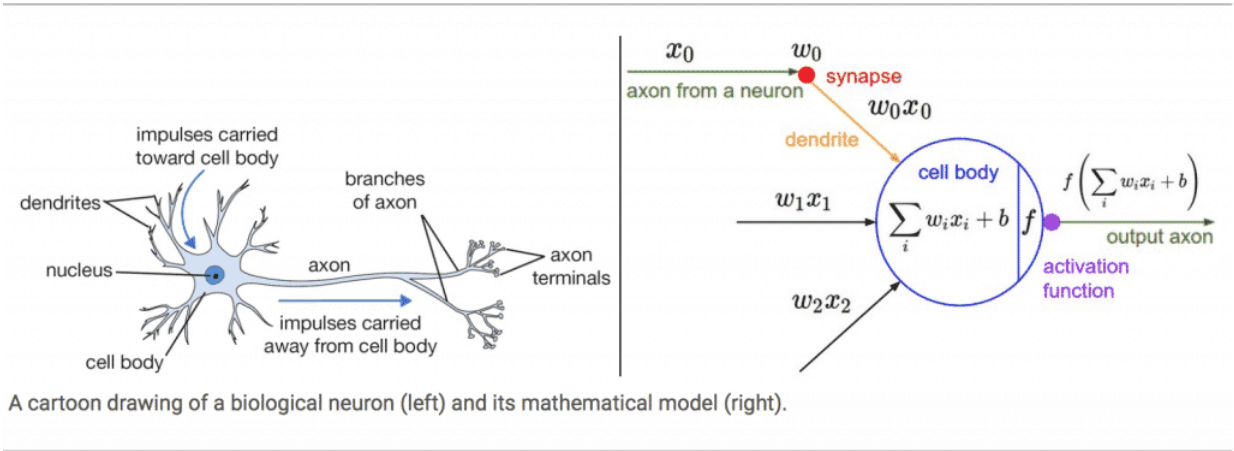


**Figure 2.1:** Comparaison entre un neurone humain (à gauche) et le modèle mathématique d'un neurone artificiel (à droite)

[4]

Le réseau de neurones artificiel est composé généralement de trois types de couches : la couche d'entrée (input layer), couche(s) cachée(s) (hidden layer) et la couche de sortie (output layer)

comme le montre la figure 2.2 La couche d'entrée est composée de neurones qui correspondent aux caractéristiques (features) des données d'entrée (par exemple la matrice de pixels de l'image ou la forme vectorielle de la donnée). La couche de sortie représente les résultats de la tâche assignée au réseau. La taille d'un réseau de neurones est mesurée par le nombre des couches, nombre de nœuds et la façon avec laquelle les neurones sont connectés.

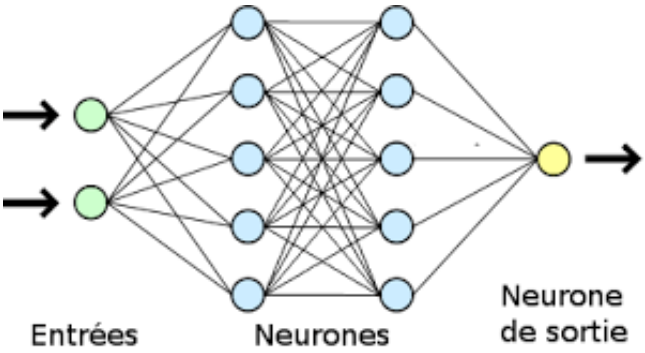


**Figure 2.2:** Réseaux de neurones avec deux couches cachées.

[4]

Théorème d'Hornik : Un réseau de neurones avec 1-couche cachée possède la propriété d'approximation universelle, c'est-à-dire qu'il peut approximer n'importe quelle fonction continue avec une précision arbitraire, à condition de disposer de suffisamment de neurones sur sa couche cachée. Comme on peut faire l'approximation de n'importe quelle fonction continue avec un réseau de neurones à 1-couche cachée (cf théorème d'Hornik), l'idée de l'apprentissage profond vient du fait d'augmenter le nombre de couches cachées du réseau, d'où le terme profond, pour pouvoir apprendre un niveau d'abstraction plus élevé[5][6][7].

Un neurone biologique se caractérise par :

— Synapse : le point de connexion avec les autres neurones.

— Dendrites : les points d'entrées du neurone.

— Axone : le point de sortie.

— Noyau : qui active les sorties en fonction des stimulations en entrée.

Par ressemblance à un neurone biologique, un neurone artificiel se définit par :

— Un vecteur poids w de même taille que les signaux d'entrées.

— des signaux d'entrées $X = x_1...x_n$.

— des signaux d'entrées.

— le neurone calcule le produit scalaire entre les signaux d'entrée et le vecteur poids en ajoutant un biais. Le résultat est appliqué dans la fonction d'activation comme le montre l'équation 2.1.

$$s = g(X.w + b) = g(\sum_{i=1}^{n} x_i.w_i + b) \tag{2.1}$$

avec :

s : la valeur ou état interne, ou score interne du neurone.

b : variable biais du neurone.

La fonction d'activation g permet d'introduire une non-linéarité après l'opération de produit scalaire afin de permettre au réseau d'apprendre des fonctions très complexes pour convertir nos entrées en la (les) sortie(s) voulue(s). Cette non-linéarité permet d'avoir différentes variations de l'état interne sur un objet de la même classe[8]. Les fonctions d'activation les plus utilisées sont :

$$sigmoid(x) = \frac{1}{1 + exp(x)} \tag{2.2}$$

$$tanh(x) = \frac{\exp(\mathrm{x}) + exp(-x)}{\exp(\mathrm{x}) + exp(-x)} \tag{2.3}$$

$$ReLU(x) = max(0.x) \tag{2.4}$$

Les modèles tanh, sigmoid et ReLU (Rectified Linear Unit), sont bien adaptés aux algorithmes d'optimisation comme la descente stochastique de gradient car elles sont différentiables. Mais actuellement c'est le ReLU qui est le plus utilisé dans les réseaux de neurones profonds à cause d'un problème que l'on appelle "vanishing gradient". La figure suivante montre que la fonction d'activation sigmoid produit des valeurs entre 0 et 1, le gradient va se tendre vers 0 au fur et à mesure que l'on a des réseaux profonds (plusieurs couches). Le ReLU permet d'amortir le "vanishing gradient" car sa valeur de sortie est invariante si x est strictement positif.

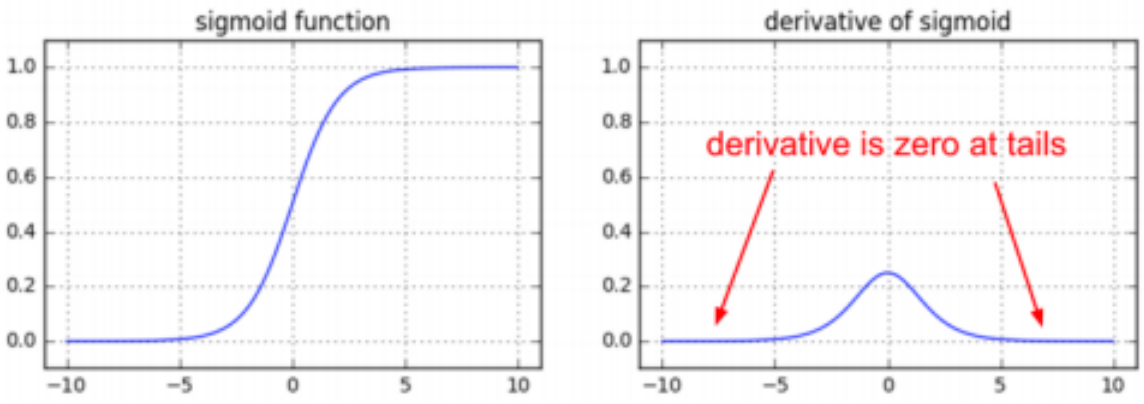


**Figure 2.3:** Fonction sigmoid (à gauche) et sa dérivée (à droite)

[9]

Comme le montre la figure 2.3, la dérivée de la fonction sigmoid est nulle pour des valeurs de x qui s'éloignent de zero. La rétro-propagation est basé sur le "chain rule", la multiplication des dérivées va faire tendre le gradient vers 0. Pour apprendre à partir d'un modèle de réseaux de neurones artificiel, on parcourt le réseau en fixant un nombre d'itérations. Durant chaque itération, on part de la couche d'entrée puis, on calcule successivement les valeurs des états internes de chaque neurone dans les couches cachées jusqu'à la couche de sortie. Ensuite on applique aux valeurs internes de la couche de sortie une fonction nommée fonction coût calculé par l'équation 2.5.

$$L(y, \widehat{y}) = \sum_{k=1}^{K} (y_k.log(\widehat{y}k) + (1 - y_k).log(1 - y_k)) \tag{2.5}$$

La fonction mathématique 2.5 modélise l'erreur des prédictions réalisées durant chaque itération. Afin de minimiser la valeur de la sortie de cette fonction, nous utilisons la rétropropagation inverse (ou backpropagation en anglais). Cette technique permet de calculer le gradient grâce au chain rule illustré par la figure 2.4.

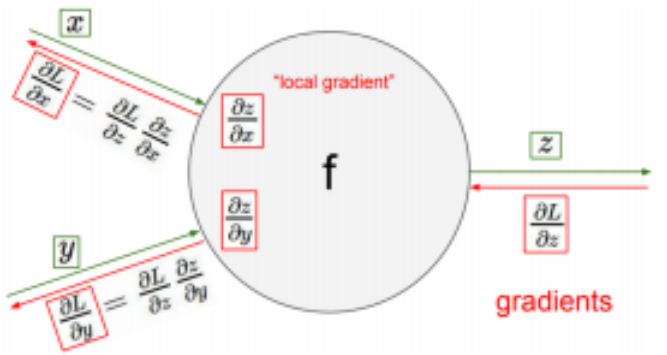


**Figure 2.4:** Calcul de gradient avec le chain rule

[10]

Après avoir calculé les gradients durant une itération, nous devons mettre à jour les paramètres du réseau. Pour se faire, la méthode la plus connue est la descente du gradient.

Apprentissage :

1. Choix des hyper-paramètres et de l'architecture du réseau.
2. Initialisation des paramètres $W_{ij}$ du réseau.

Répéter pendant n itérations :

3. Prendre aléatoirement un sous ensemble des données x.
4. Prédiction de $y$ avec le forward propagation.
5. Calcul de l'erreur avec la fonction coût $L(y, \widehat{y})$ de l'équation 2.5.
6. Mise à jour des paramètres de chaque couche $W_{ij} := W_{ij} - \alpha \frac{\partial W}{\partial w_{ij}}$.

Test :

1. Injection des valeurs d'un input qui n'ont pas été utilisé pendant l'apprentissage dans la couche d'entrée.
2. Calcul de la prédiction $\widehat{y}$ directement (couche de sortie) en passant par les couches intermédiaires.

Il existe plusieurs autres variantes d'algorithme d'optimisation comme : SGD et Adam.

— Stochastic Gradient Descent(SGD) : Cet algorithme modifie les paramètres du modèle fréquemment. En effet les poids sont mis à jour après le calcul de l'erreur pour chaque donnée

d'apprentissage. Ainsi, si l'ensemble de données contient 10000 échantillons, le SGD met à jour les paramètres du modèle 10000 fois en un seul cycle de l'ensemble de données. Les paramètres sont modifiés selon l'équation suivante :

$$W_{t+1} = W_t - \mu\nabla(W_{t+1}, X(i), Y(i)) \tag{2.6}$$

Avec :

X(i), Y(i) sont respectivement une échantillon de l'ensemble de données et son label.

$W_t$ vecteur de tous les paramètres du modèle à l'itération $t$.

$\mu$ le taux d'apprentissage, $J$ La fonction coût et $\nabla$ l'opérateur de gradient.

Cet algorithme converge rapidement puisque les paramètres sont modifiés fréquemment.

— Adam (Short for Adaptive Moment Estimation) : C'est l'un des algorithmes les plus récents et les plus efficaces pour l'optimisation par descente de gradient. Son principe est d'ajuter automatiquement le taux d'apprentissage pour chaque paramètre. Sa particularité est de calculer $mt$ et $vt$ qui sont des estimations adaptatives des moments. Adam est basé sur les équations suivantes :

$$m_{t+1,i} = \beta_1 m_{t,i} + (1 - \beta_1) g_{t,i} \tag{2.7}$$

$$v_{t+1,i} = \beta_2 v_{t,i} + (1 - \beta_2) g_{t,i}^2 \tag{2.8}$$

$$W_{t+1,i} = W_{t,i} - \mu \frac{\sqrt{1 - \beta_2}}{1 - \beta_1} \frac{m_{t,i}}{\sqrt{v_{t,i} + \epsilon}} \tag{2.9}$$

Avec :

$m_{t,i}$ premier moment du gradient (la moyenne).

$v_{t,i}$ second moment du gradient (variance non centré).

$g_{t,i}$ le gradient de la fonction coût.

$\beta_1, \beta_1 \in ]0.1[$ et $\mu, \epsilon > 0$.

Grâce à son ajustement du taux d'apprentissage avec les moments($m_t et v_t$) Adam est plus rapide et plus efficace que les autres méthodes d'optimisation.

## 2.2 Vision par ordinateur

La vision par ordinateur est le domaine d'étude dans lequel les ordinateurs sont habilités à visualiser, reconnaître et traiter ce qu'ils voient d'une manière similaire à celle des humains. L'objectif principal de la vision par ordinateur est de générer des informations pertinentes à partir de données d'images et de vidéos afin de déduire quelque chose sur le monde.

La vision par ordinateur peut être classée en tant qu'un sous-domaine de l'intelligence artificielle et de l'apprentissage automatique. Ceci est assez différent du traitement d'images, qui implique la manipulation ou l'amélioration d'informations visuelles et ne se préoccupe pas du contenu de l'image. En fait, Les applications de la vision par ordinateur incluent la classification d'images, la détection visuelle, la reconstruction de scènes 3D à partir d'images 2D, la récupération d'images, la réalité augmentée, la vision industrielle et l'automatisation du trafic [11] [12].

Aujourd'hui, l'apprentissage automatique est une composante nécessaire de nombreux algorithmes de vision par ordinateur[13]. Ces algorithmes sont généralement une combinaison de traitement d'images et des techniques d'apprentissage automatique. La principale exigence de ces algorithmes est de gérer de grandes quantités de données image ou vidéo et pouvoir effectuer des calculs en temps réel pour une large gamme d'application. Citons à titre d'exemple, la détection des objets en temps réel.

## 2.3 Réseaux de neurones convolutifs (CNN)

Les réseaux de neurones convolutifs (Convolutional Neural Network ou ConvNet ou CNN) illustrés par la figure 2.5 sont très similaires aux réseaux de neurones ordinaires, Ils ont le même principe de fonctionnement. Un CNN se compose d'une couche d'entrée et d'une couche de sortie, ainsi que de plusieurs couches cachées. Les couches cachées sont soit convolutives, soit de mise en commun (pool layer). Les architectures CNN supposent explicitement que les entrées sont des images, ce qui nous permet de coder certaines propriétés dans l'architecture. Celles-ci rendent alors la fonction de transmission directe plus efficace à mettre en œuvre et réduisent considérablement la quantité de paramètres dans le réseau.

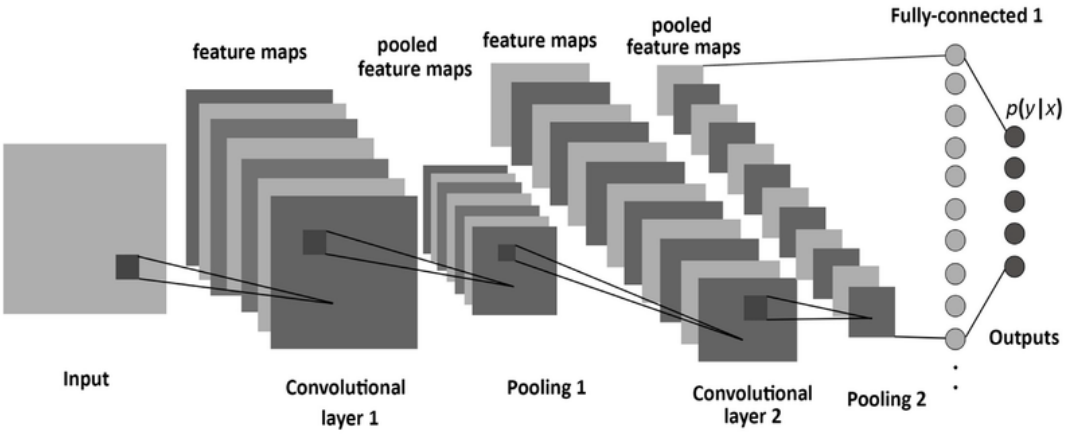


**Figure 2.5:** Architecture du CNN

[14]

— La convolution : La couche de convolution est la composante clé des réseaux de neurones convolutifs, et constitue toujours au moins leur première couche. Le but est de repérer la présence d'un ensemble de caractéristiques (features) dans les images reçues en entrée. Pour cela, on réalise un filtrage par convolution. Dans la couche de convolution, au lieu de faire un produit scalaire entre les valeurs internes et les poids de chaque neurone, on applique un produit de convolution. Ce produit de convolution sert à extraire des caractères spécifiques dans l'image. Dans un réseau de neurones convolutif, on applique une succession de filtres de convolution dans le but d'extraire petit à petit les informations caractéristiques sur l'image. Les filtres de convolution varient en fonction des effets désirés comme la détection de contour de l'image, rendre floue l'image, etc... Mais en terme d'apprentissage, les valeurs des filtres de convolution sont les paramètres à apprendre durant la phase d'apprentissage [15].

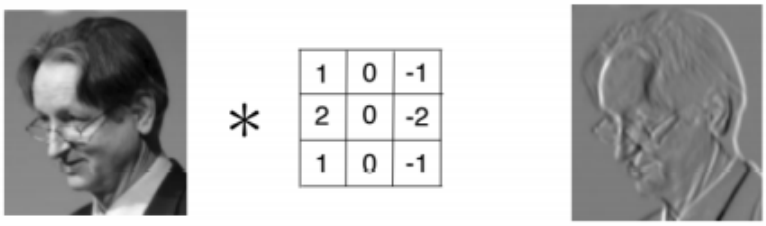


**Figure 2.6:** Produit de convolution

[4]

— Le Pooling : La couche de mise en commun est une sorte de sous échantillonnage non linéaire. Elle sert à réduire la dimension spatiale dans un réseau ConvNet. C'est une couche qui prend

en entrée chaque ”feature map” et la simplifie. Par exemple le ”maxpooling” prend une région et donne en sortie le maximum de cette région. Par exemple, si la couche cachée à partir des ”feature map” est de 4x4, en prenant une région de 2x2 pour le ”maxpooling”, il ne restera plus en sortie que 2x2 unités.

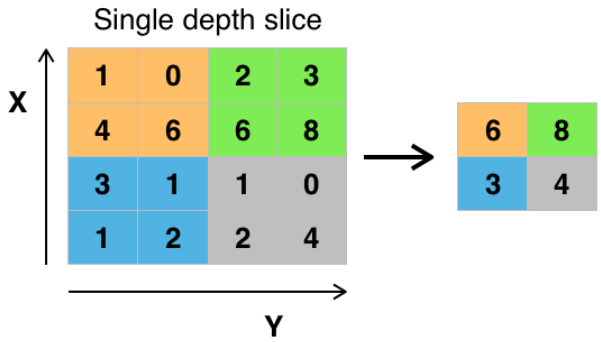


**Figure 2.7:** Illustration du principe de max pooling

[4]

Contrairement aux méthodes traditionnelles, les features ne sont pas pré-définies selon un formalisme particulier, mais apprises par le réseau lors la phase d'entraînement. Les noyaux des filtres désignent les poids de la couche de convolution. Ils sont initialisés puis mis à jour par rétro-propagation du gradient. Les réseaux de neurones convolutifs sont capables de déterminer les éléments discriminants d'une image en s'adaptant au problème posé. Ces éléments sont en bas niveau pour les premières couches et deviennent de plus en plus complexes avec la profondeur du réseau. En effet, les premiers filtres ont appris à détecter des bords (edges). Puis, plus nous avançons dans le réseau plus les filtres apprennent à détecter des caractéristiques plus complexes du visage humain comme les yeux, le nez, la bouche comme illustré par la figure 2.8.

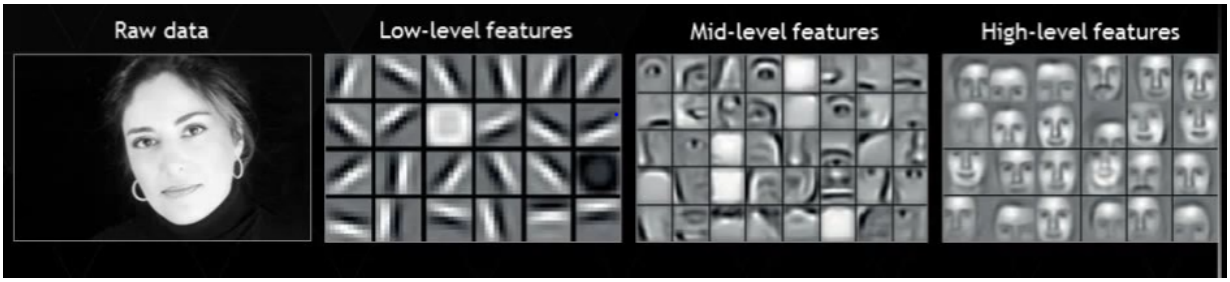


**Figure 2.8:** Caractéristiques extraits d'un visage

[16]

— Couche "Fully connected" : Reçoit un vecteur en entrée, elle applique une combinaison linéaire et une fonction d'activation sur les valeurs d'entrées pour finalement produire un vecteur en sortie. Ce vecteur est de taille $N$ où $N$ est le nombre de classes dans le problème de classification d'images. Chaque valeur du vecteur de sortie indique la probabilité d'appartenance de l'image d'entrée à une classe. La fonction d'activation peut être sigmoïde si le nombre de classe $N$ est égale à 2(classification binaire), ou softmax si $N > 2$ (classification multiple). Ces deux fonctions donnent des valeurs entre 0 et 1.

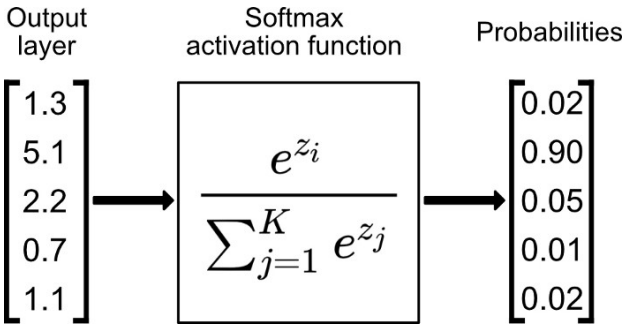


**Figure 2.9:** Résultat de la fonction Softmax

[17]

La fonction softmax, est une fonction exponentielle normalisée qui prend en entrée un vecteur de K nombres réels et qui en sorte un vecteur de K nombres réels strictement positifs et de somme 1. Les paramètres du CNN tels que, les poids des filtres de la couche de convolution et les poids de la couche "fully connected" sont définis pendant l'entrainement par un algorithme d'optimisation.

— Couche "Batch Normalization" : La couche de "BatchNormalization" est utilisée pour normaliser les entrées de chaque couche. En toute rigueur, la "BatchNormalization" normalise les données en soustrayant la moyenne du batch de données et en divisant par sa moyenne. Cette opération va permettre de rendre le réseau plus stable et aussi va permettre d'accélérer sa phase d'apprentissage. D'un point de vue mathématique, nous ne perdons pas d'information en passant de valeurs entre -10 et 10 qu'entre -1 et 1. Cela permet en outre de réduire les écarts grossiers qui pourraient avoir un trop gros impact sur la mise à jour des poids (lors de la descente du gradient).

## 2.4 Apprentissage par transfert

Entraîner un réseau de neurones convolutif est très coûteux : plus les couches s'empilent, plus le nombre de convolutions et de paramètres à optimiser est élevé. L'ordinateur doit être en mesure de stocker plusieurs gigaoctets de données et de faire efficacement les calculs. C'est pourquoi les fabricants de matériel informatique multiplient les efforts pour fournir des processeurs graphiques (GPU) performants, capables d'entraîner rapidement un réseau de neurones profond en parallélisant les calculs. L'apprentissage par transfert (Transfer Learning en anglais) permet de faire de l'apprentissage profond en gagnant beaucoup de temps. Le principe est d'utiliser les connaissances acquises par un réseau de neurones lors de la résolution d'un problème semblable afin d'en résoudre un autre.

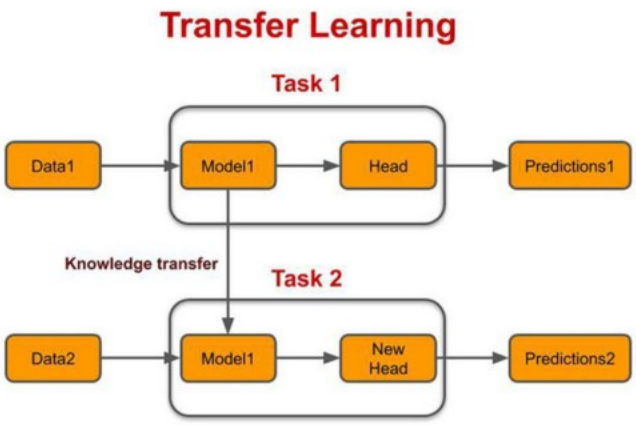


**Figure 2.10:** Principe de l'apprentissage par transfert

[17]

En pratique, très peu de personnes forment un réseau convolutionnel entier à partir de zéro (avec une initialisation aléatoire), car il est relativement rare d'avoir un ensemble de données de taille suffisante. Alors, il est courant de prétraiter un Convnet sur un très grand ensemble de données (par ex. Imagenet, qui contient 1,2 million d'images avec 1000 catégories), puis d'utiliser le Convnet soit comme une initialisation ou un extracteur de fonctionnalités fixe pour la tâche d'intérêt. Selon notre architecture CNN, cette approche peut prendre deux formes :

— Convnet comme extracteur de caractéristique fixe.

— Ajuster les paramètres du Convnet.

### 2.4.1 Convnet comme extracteur de caractéristique fixe

Prenant un Convnet préétabli sur Imagenet, nous retirons la dernière couche entièrement connectée (les sorties de cette couche sont les 1000 scores de classe pour une tâche différente comme Imagenet), puis nous traitons le reste du Convnet comme un extracteur de fonctionnalités fixe pour le nouvel ensemble de données.

### 2.4.2 Ajuster les paramètres du Convnet (Fine tuning)

Ajuster les paramètres du Convnet (Fine tuning) : la deuxième stratégie consiste non seulement à remplacer et à recycler le classificateur au-dessus du Convnet sur le nouvel ensemble de données, mais aussi à affiner les poids du réseau de prétraitement en continuant la rétropropagation. Il est possible d'ajuster toutes les couches du Convnet, ou il est possible de maintenir certaines des couches précédentes fixées (en raison de problèmes d'overfitting) et de n'ajuster qu'une partie plus élevée du réseau. Ceci est motivé par l'observation que les caractéristiques antérieures d'un Convnet contiennent des caractéristiques plus génériques qui devraient être utiles à de nombreuses tâches, mais les couches ultérieures du Convnet deviennent progressivement plus spécifiques aux détails des classes contenues dans l'ensemble original des données.

### 2.4.3 Les types de l'apprentissage par transfert

Alors quand et comment ajuster les paramètres du Convnet ? Comment décider quel type d'apprentissage de transfert à effectuer sur un nouvel ensemble de données ?

C'est en fonction de plusieurs facteurs, mais les deux plus importants sont la taille du nouvel ensemble de données (petit ou grand), et sa similitude avec l'ensemble de données original. En gardant à l'esprit que les fonctionnalités de ConvNet sont plus génériques dans les premières couches et plus spécifiques à l'ensemble de données original dans les couches ultérieures, nous présentons les 4 scénarios principaux :

— Dans le cas où le nouvel ensemble de données est grand et très différent de l'ensemble de données original il est recommandé de faire l'apprentissage sur tout le modèle (faire apprendre

toutes les couches du réseau) : l'utilisation de l'architecture du modèle préformé et on faire apprendre le modèle à partir de zéro sur le nouvel l'ensemble de données, il nécessite beaucoup de puissance de calcul.

— Dans le cas où le nouvel ensemble de données est grand et semblable à l'ensemble de données original, la solution est de faire le training sur certaines couches et laisser les autres bloqués puisque les couches inférieures font référence à des caractéristiques générales (indépendantes du problème), tandis que les couches supérieures font référence à des caractéristiques spécifiques (dépendantes du problème).

— Dans le cas où le nouvel ensemble de données est petit, mais très différent de l'ensemble de données original, la solution est de faire le training sur certaines couches et laisser les autres bloqués .

— Dans le cas où le nouvel ensemble de données est petit et semblable à l'ensemble de données original, il n'est pas judicieux de paramétrer et régler le convNet en raison de problèmes d'overfitting. Étant donné que les données sont semblables aux données originales, nous nous attendons à ce que les caractéristiques de niveau supérieur du ConvNet soient également pertinentes pour cet ensemble de données. Par conséquent, la meilleure idée pourrait être de former un classificateur linéaire sur le réseau CNN.

## 2.5 Technologies utilisées pour la détection des occupants et des objets

La détection d'objets a été témoin d'un changement révolutionnaire rapide dans le domaine de la vision par ordinateur. Son implication dans la combinaison de la classification d'objets et de la localisation d'objets en fait l'un des sujets les plus difficiles dans le domaine de la vision par ordinateur. Les modèles de détection d'objets basés sur l'apprentissage profond comportent généralement deux parties. Un encodeur prend une image en entrée et la fait passer à travers une série de blocs et de couches qui apprennent à extraire des caractéristiques statistiques utilisées pour localiser et étiqueter des objets. Les sorties de l'encodeur sont ensuite transmises à un décodeur, qui prédit les cadres de délimitation et les étiquettes pour chaque objet. La figure 2.11 illustre un exemple de détection des occupants et des objets dans une voiture.

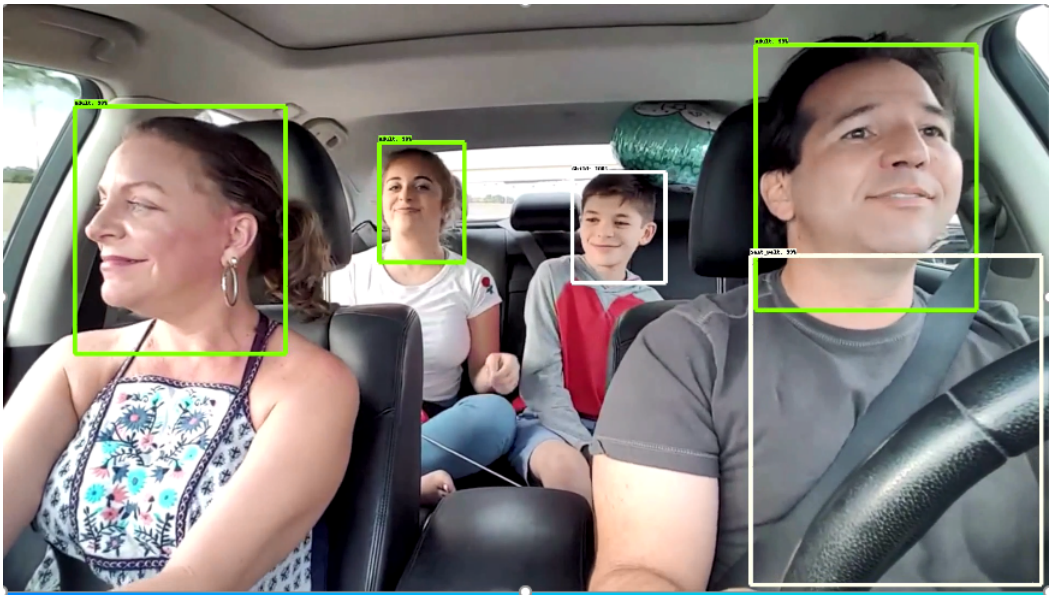

**Figure 2.11:** Exemple de détection des occupants et des objets dans une voiture

Les techniques de détection sur des images ou des vidéos peuvent se catégoriser en deux approches :

- ◇ Approche en deux passes
- ◇ Approche en une seule passe

Pour les modèles qui suivent l'approche en deux passes, l'image d'entrée est envoyée d'abord à un modèle séparé. Le rôle de ce modèle consiste à identifier les zones ou régions de l'image contenant les informations requises. Ensuite chaque région est envoyée à un second modèle où s'effectue l'identification de l'objet présent. Dans cette catégorie de modèle, on retrouve les variantes de CNN comme FasterRCNN. Maintenant, pour l'approche en une passe, il suffit une et une seule propagation pour la détection. En effet, l'image est observée par le réseau neuronal une seule fois, et ce dernier retourne directement les objets présents et leurs positions sous forme de boites englobantes. C'est exactement le principe de fonctionnement de YOLO et SSD que nous avons exploité (versions récentes) pour la modélisation.

### 2.5.1 Regional Convolutional Neural Network (R-CNN)

Les modèles R-CNN [18] sont une famille de réseaux de neurones convolutifs conçus pour la détection d'objets, développés par Ross Girshick, et al.

A partir d'une image donnée en entrée, le modèle va extraire de l'image les régions les plus susceptibles de contenir un objet. Nous parlons de zones d'intérêts. Pour chacune de ces zones d'intérêts, un ensemble de boîtes englobantes (bounding box) va être généré. Ces boîtes sont classifiées et sélectionnées en fonction de leur probabilité à contenir l'objet. Deux milles propositions de régions sont ainsi extraites. Ces régions sont ensuite reçues en entrée par le CNN. Le modèle peut alors détecter dans une image un (ou plusieurs) objet(s), pouvant appartenir à des classes différentes.

#### 2.5.1.1 Faster Region-based Convolution Neural Network (Faster R-CNN)

Faster-RCNN [19] est une des méthodes CNN les plus utilisées à ce jour. Il se compose en gros de 3 parties : le backbone,le réseau de proposition de région (RPN) et le détecteur/Fast-RCNN-head, représenter dans la figure suivante.

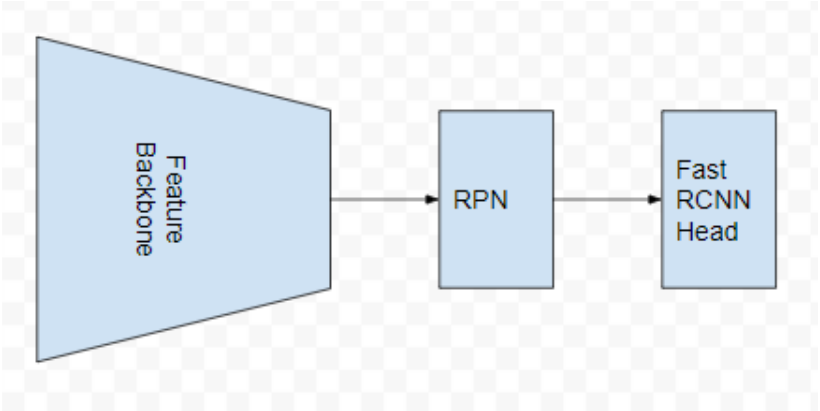


**Figure 2.12:** Composants du Faster RCNN

— L'épine dorsale (backbone) est généralement un réseau de classification d'images pré-entraîné (par exemple ResNet).

— RPN est un petit réseau pouvant être entraîné pour générer des régions d'intérêt (ROI). La fonction de RPN est principalement de générer des rectangles qui ont une bonne probabilité de contenir un objet d'intérêt et sont agnostiques de classe. Les entrées de RPN sont généralement

constituées d'un grand nombre de rectangles prédéfinis appelés ancres. Le réseau est entraîné à sélectionner les rectangles les plus probables contenant des objets.

— La tête Fast-RCNN est également un réseau pouvant être entraîné qui génère la probabilité de classe finale et les cadres de délimitation. Pour ce faire, il normalise d'abord le ROI en un rectangle de taille fixe (généralement 7x7) en utilisant le pool de ROI. Au niveau de la tête RPN et Fast-RCNN, la suppression non maximale (NMS) est utilisée pour supprimer les rectangles étroitement alignés. Le besoin de NMS se pose en raison du grand nombre d'ancres utilisées et beaucoup d'entre elles peuvent se chevaucher. D'une manière générale, les ancres sont une autre façon de faire une fenêtre coulissante, bien que dans l'espace des fonctionnalités plutôt que dans l'espace de l'image. La figure 2.13 montre l'architecture globale de Faster-RCNN.

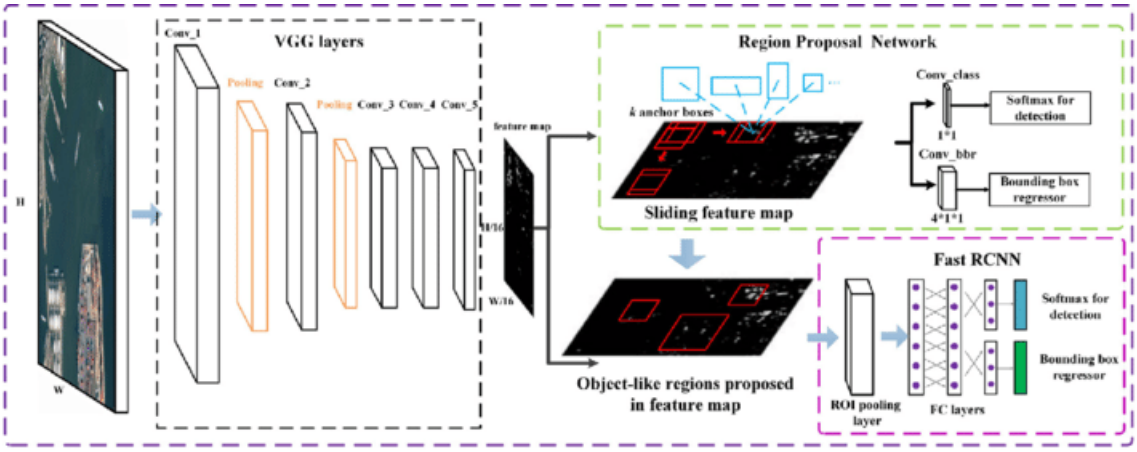


**Figure 2.13:** Architecture globale de Faster-RCNN.

[19]

### 2.5.2 Single Shot MultiBox Detector (SSD)

Single Shot MultiBox Detector [20] est un détecteur d'objets lancé en 2017 en tant qu'une architecture CNN efficace conçue pour les applications de vision mobiles et embarquées. Cette architecture utilise éprouvée en profondeur convolutions séparables pour construire des réseaux de neurones profonds légers. Le SSD a deux composants : un modèle de backbone et une tête SSD.

— Le modèle de base est généralement un réseau de classification d'images pré-entraîné en tant qu'extracteur de caractéristiques. Il s'agit généralement d'un réseau comme VGG ou ResNet formé sur ImageNet dont la couche de classification finale entièrement connectée a été supprimée. Nous nous retrouvons donc avec un réseau de neurones profond qui est capable

d'extraire le sens sémantique de l'image d'entrée tout en préservant la structure spatiale de l'image bien qu'à une résolution inférieure.

— La tête SSD est une ou plusieurs couches convolutives ajoutées à cette épine dorsale et les sorties sont interprétées comme les boîtes englobantes et les classes d'objets dans l'emplacement spatial des activations des couches finales.

Dans la figure 2.14, les deux premières couches sont l'épine dorsale, les autres couches représentent la tête SSD.

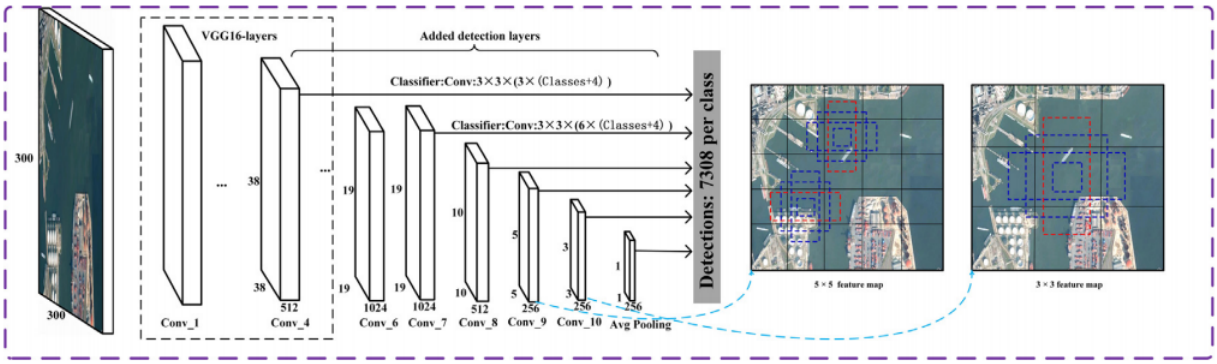


**Figure 2.14:** Architecture SSD VGG16

En utilisant SSD, nous n'avons besoin de prendre qu'une seule prise de vue pour détecter plusieurs objets dans l'image, tandis que les approches basées sur le réseau de propositions régionales (RPN) telles que les séries R-CNN qui nécessitent deux prises de vue, une pour générer des propositions de région, une pour détecter l'objet de chaque proposition. Ainsi, le SSD est beaucoup plus rapide que les approches basées sur le RPN à deux coups.

### 2.5.3 You Only Look Once(YOLO)

YOLO [21] est un système de détection d'objets en temps réel qui applique un seul réseau de neurones à l'image d'entrée complète, d'où le nom vous ne regardez qu'une seule fois. YOLO divise l'image d'entrée en une grille de taille S × S où chaque cellule de la grille est responsable de la prédiction du nombre B de cadres de délimitation avec les scores de confiance associés, et des probabilités de classe conditionnelles C représentant la probabilité pour chaque classe étant donné un certain objet prédit. Le score de confiance d'une boîte englobante est calculé en multipliant la probabilité de classe par l'IOU correspondante entre une boîte prédite et une boîte de vérité terrain.

Tout comme SSD, YOLO prédit un grand nombre de cadres de délimitation, où une fraction significative de ceux-ci fait référence aux mêmes objets. Bien que cela convienne pendant l'entraînement, au moment du test, ces prédictions en double pointant vers les mêmes objets doivent être élaguées à une par objet prédit. Cet élagage est effectué à l'aide de la suppression non maximale (NMS), qui supprime les boîtes avec de faibles probabilités de classe et fusionne les boîtes englobantes très chevauchantes des mêmes objets en boîtes uniques, comme le montre la figure 2.15.

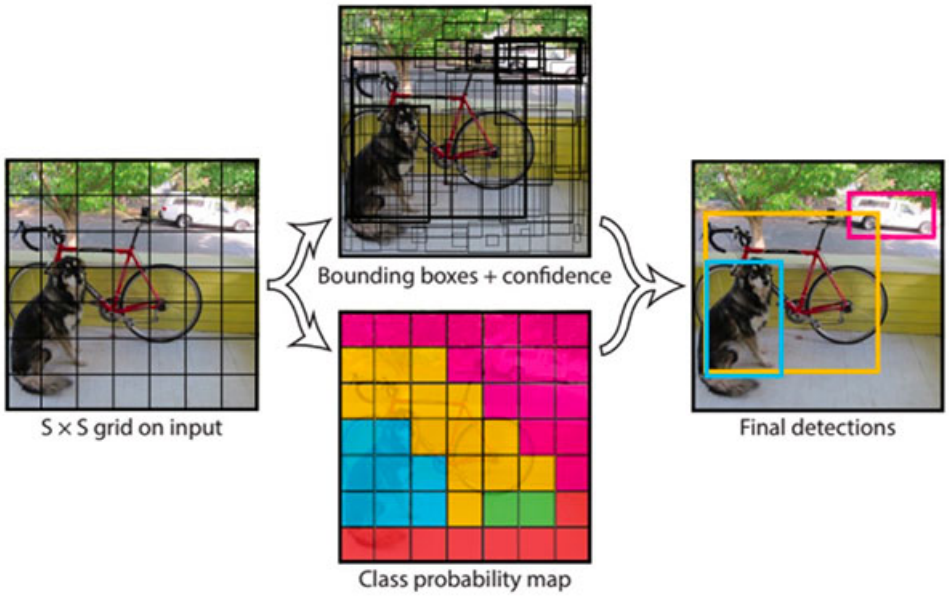


**Figure 2.15:** Processus de détection d'objets pour YOLO

[21]

#### 2.5.3.1 YOLOv4

YOLOv4 [22], est l'un des récents version de YOLO développé par trois développeurs Alexey Bochkovskiy, Chien-Yao Wang et Hong-Yuan Mark Liao en avril 2020.

YOLOv4 est une amélioration importante de YOLOv3, la mise en place d'une nouvelle architecture dans le Backbone et les modifications dans le Neck ont amélioré le mAP de 10% et le nombre de FPS de 12%. De plus, il est devenu plus facile de former ce réseau de neurones sur un seul GPU. Comme illustré par la figure 2.16, l'architecture de YOLOv4 se compose par trois principaux parties qui sont le Backbone, Neck et Head.

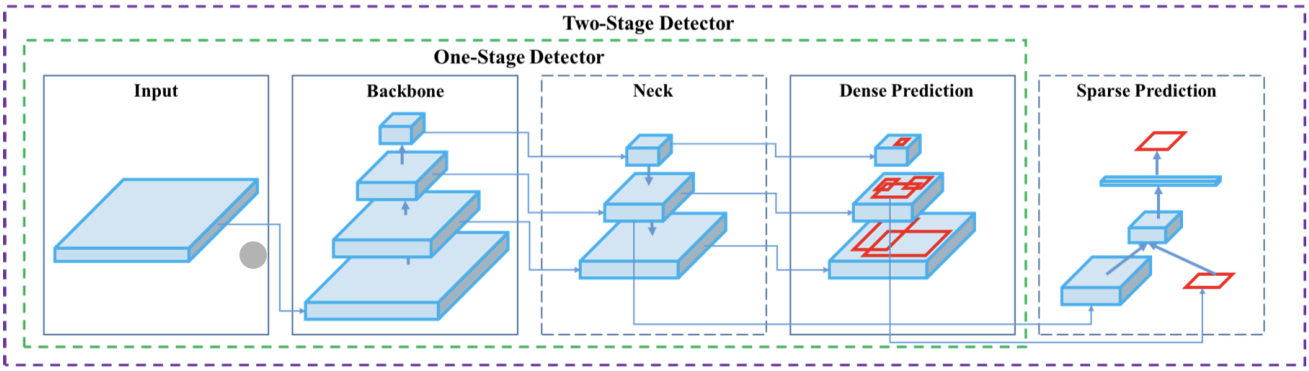


**Figure 2.16:** Architecture YOLOv4

[19]

— Backbone : c'est le corps du réseau. Il convertit l'image en entrée en un ensemble d'informations caractérisant son contenu et qu'on appelle features. Après une analyse rigoureuse de différents paramètres sur des benchmarks standards, les auteurs ont finalisé CSPDarknet5 comme épine dorsale du réseau de l'architecture YOLOv4. CSPDarknet53 peut améliorer la capacité d'apprentissage de CNN. Le bloc de mise en commun de la pyramide spatiale est ajouté sur CSPDarknet53 pour augmenter le champ réceptif et séparer les caractéristiques contextuelles les plus importantes. Au lieu des réseaux pyramidaux de fonctionnalités (FPN) pour la détection d'objets utilisés dans YOLOv3, le PANet est utilisé comme méthode d'agrégation de paramètres pour différents niveaux de détecteur.

— Neck : le cou est exploité dans les épines dorsales pour l'extraction de caractéristiques sémantiques riches qui sont ensuite utilisées pour des prédictions précises.

— Head : responsable à la décision finale du réseau.

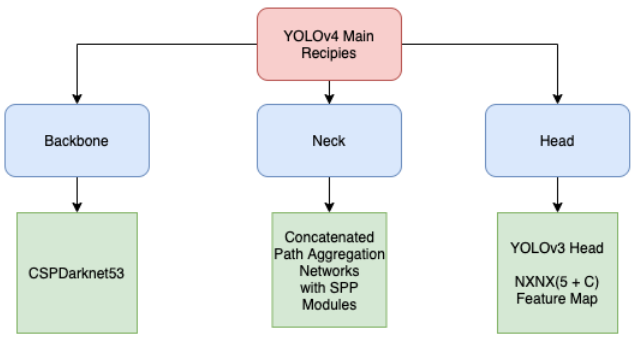


**Figure 2.17:** Architecture de base finale de YOLOv4

### 2.5.4 Métrique d'évaluation

Dans cette section, les métriques d'évaluation utilisées dans la détection d'objets sont couvertes. Ces mesures nous permettent de mesurer la performance des modelés deep learning et de comparer objectivement les systèmes de détection utilisées. L'évaluation d'un système de détection d'objets est une tâche non triviale car il y a deux fonctions distinctes à mesurer : déterminer si un objet est présent dans l'image et détection de l'emplacement de l'objet. Les deux métriques les plus couramment utilisées pour les tâches de localisation et de classification des cadres de délimitation lorsque nous travaillons avec la détection d'objets sont respectivement la zone d'intersection sur l'union (IoU) , la moyenne précision moyenne (mAP) et le FPS.

#### 2.5.4.1 Intersection sur l'union (IOU)

La figure 2.18 illustre que l'IOU mesure le chevauchement entre deux frontières. Il est utilisé pour calculer le degré de chevauchement entre la limite prévue et la vérité de terrain (la limite de l'objet réel). Dans certains ensembles de données, nous prédéfinissons un seuil d'unité d'œuvre (par exemple 0,5) pour déterminer si la prédiction est un vrai positif ou un faux positif :

— Si IOU $\geq 0.5$ : classer la détection d'objet comme un vrai positif (détection correcte).

— Si IOU $< 0.5$ : classer la détection d'objet comme un faux positif (détection erronée).

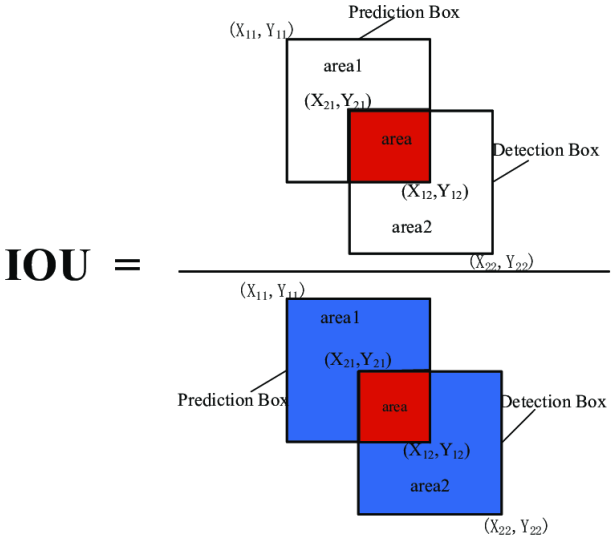


**Figure 2.18:** L'intersection over Union

[23]

#### 2.5.4.2 La moyenne des précisions moyennes

Le critère mAP ou encore mean average precision est un score en pourcentage. Pour le calculer, il faut mesurer la précision et le rappel pour chaque image du jeu de données.

— Précision : est une mesure de la précision de la prédiction, c'est-à-dire, le pourcentage de prédictions positives correctes. Elle est défini comme le nombre de vrais positifs (TP), sur la somme entre le nombre de faux positifs (FP), et le nombre de vrais positifs.

$$Precision = \frac{TP}{TP + FP} \tag{2.10}$$

— Rappel : détermine la qualité de la prédiction, ou la fraction de détections d'objets réels par rapport au nombre total d'objets dans l'ensemble de données. Le rappel est défini comme le nombre de vrais positifs sur la somme entre le nombre de faux négatifs (FN), et le nombre de vrais positifs.

$$Rappel = \frac{TP}{TP + FN} \tag{2.11}$$

Après avoir calculé la précision et le rappel, nous traçons pour chaque classe la courbe précision/rappel. L'aire sous cette courbe définit ce qu'on appelle précision moyenne (AP). Le mAP est la moyenne de l'AP de toutes les classes.

#### 2.5.4.3 Taux d'images par seconde (FPS)

L'FPS définit la vitesse à laquelle votre modèle de détection d'objets traite une vidéo et génère la sortie souhaitée. C'est une façon d'exprimer la rapidité du modèle deep learning.

## 2.6 Technologies utilisées pour la classification de l'état de distraction du conducteur

La distraction au volant est l'une des principales causes d'accidents et de la circulation. Par conséquent, les systèmes de conduite de véhicules intelligents sont devenus de plus en plus importants. Récemment, l'intérêt pour les systèmes d'assistance à la conduite qui détectent les

actions des conducteurs et les aident à conduire en toute sécurité s'est accru. Dans ces études, bien que certains types de données distincts, tels que les conditions physiques du conducteur, les caractéristiques audio et visuelles et les informations sur le véhicule, soient utilisés, la principale source de données est constituée d'images du conducteur avec une caméra à l'intérieur de la voiture. Dans ce contexte, des architectures basées sur un réseau de neurones à convolution (CNN) sont proposées pour classer et détecter la distraction du conducteur.

Comme mentionné dans la partie précédente, les CNNs ont montré des progrès impressionnants dans diverses tâches telles que la classification d'images, la détection d'objets, la reconnaissance d'actions et bien d'autres. Diverses architectures comme GoogleNet, AlexNet, VGGNet, ResNet et MobileNet ont établi des références en vision par ordinateur et plus précisément dans la classification des images.

Dans cette partie de notre projet, nous utilisons VGGNet, ResNet et MobileNet et modifions leurs architectures pour atteindre une détection optimale de conducteur distrait.

### 2.6.1 Visual Geometric Group (VGG)

La figure 2.19 est l'architecture du VGG16. Il s'agit d'un modèle de réseau de neurones convolutifs proposé par K. Simonyan et A. Zisserman[24]. Le modèle atteint une précision de 92,7% en classification des images dans le top 5 des tests dans ImageNet.

L'entrée dans la couche « conv 1 » a une taille d'image RGB de 224 x 224. L'image passe à travers une pile de couches convolutives, où les filtres ont été utilisés avec un très petit champ récepteur : 3 × 3 (qui est la plus petite taille pour capturer la notion de gauche, droite, haut, bas et centre). Dans l'une des configurations, il utilise également des filtres de convolution 1 × 1, ce qui peut être vu comme une transformation linéaire des canaux d'entrée (suivie de la non-linéarité). La foulée de convolution est fixée à 1 pixle : le remplissage spatial de conv. L'entrée de la couche est telle que la résolution spatiale est conservée après la convolution. Le pooling spatial est réalisé par cinq couches de max pooling, qui suivent les couches de convolution (toutes les couches de conv ne sont pas suivies d'un max-pooling). La mise en pool maximum est effectuée sur une fenêtre de 2 × 2 pixels, avec foulée 2. Toutes les couches cachées sont équipées de la non-linéarité de rectification (ReLU).

Trois couches entièrement connectées (FC) suivent une pile de couches convolutifs : les deux premières comportent 4096 canaux chacune, la troisième effectue la classification à 1000 sorties. La couche finale est la couche softmax. La configuration des couches entièrement connectées est la même dans tous les réseaux. On doit remplacer les 1000 canaux par 2 canaux afin d'adapter le réseau à notre problématique [25].

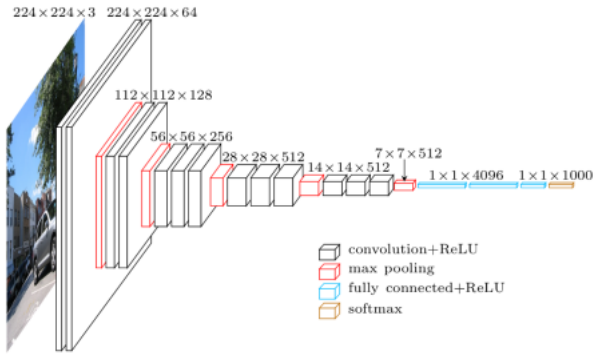


**Figure 2.19:** Architecture VGG16

[26]

L'architecture VGG a le nombre total de 16 couches convolutives et entièrement connectées. Il en a 16 dans ce cas pour VGG 16, puis 19 pour VGG 19. Comme la figure suivante montre, c'est une architecture très similaire, mais avec quelques couches de convolutives supplémentaires.

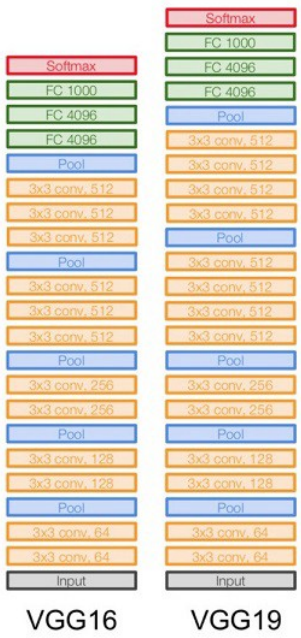


**Figure 2.20:** Les couches de vgg16 et vgg19

[27]

### 2.6.2 Residual Networks 50 (ResNet 50)

ResNet[27], est un réseau neuronal classique utilisé comme épine dorsale pour de nombreuses tâches de vision par ordinateur. Ce modèle a remporté le défi ImageNet en 2015. La percée fondamentale de ResNet a été de nous permettre de former avec succès des réseaux neuronaux extrêmement profonds de plus de 150 couches.

Comme illusré par la figure 2.21 :

— ResNet utilise le saut de connexion pour ajouter la sortie d'une couche antérieure à une couche ultérieure. Cela lui permet d'atténuer le problème de la disparition du gradient.

— F(x) est le résiduel entre la fonction H(x) désirée et la fonction identitaire : F(x) = H(x) – x.

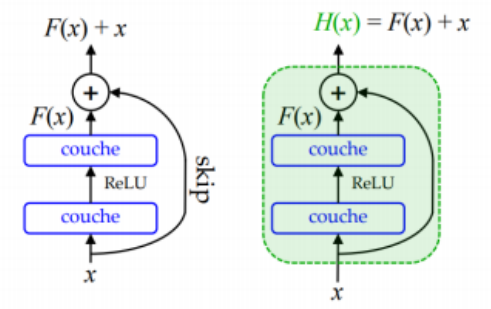


**Figure 2.21:** Le modèle Resnet50.

[27]

Le modèle ResNet-50 se compose de 5 étapes comportant chacune un bloc de convolution et d'identité. Chaque bloc de convolution comporte 3 couches de convolution et chaque bloc d'identité comporte également 3 couches de convolution. Le modèle ResNet-50 comporte plus de 23 millions de paramètres pouvant être formés. La figure suivante montre les stades du modéle ResNet50

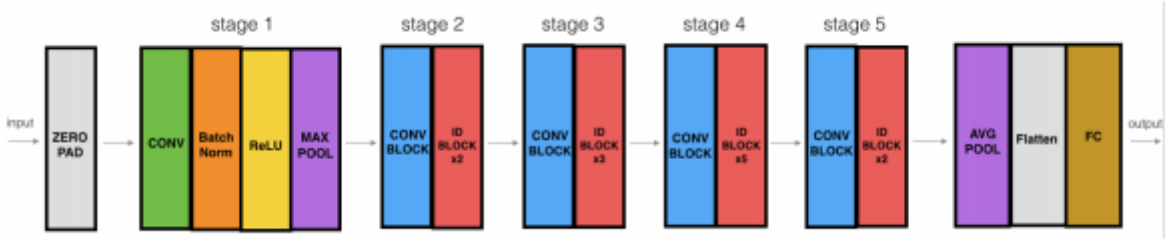


**Figure 2.22:** Les stages du modèle Resnet50.

[27]

L'architecture ResNet50 contient les éléments suivants [28] :

- Une convolution avec une taille de noyau de 7 × 7 et 64 noyaux différents avec un pas de taille 2 nous donnant 1 couche.
- Ensuite, nous voyons max pooling avec également un pas de 2.
- Dans la prochaine convolution, il y a un 1 × 1,64 noyau suivant un 3 × 3 ,64 noyau et enfin un 1 × 1, 256 noyau. Ces trois couches sont répétées au total 3 fois, ce qui nous donne 9 couches à cette étape.
- Ensuite, nous voyons un noyau de 1 × 1,128 après cela un noyau de 3 × 3,128 et enfin un noyau de 1 × 1,512. Cette étape a été répétée 4 fois donc donnant 12 couches dans cette étape.
- Après cela, il y a un noyau de 1 × 1,256 et deux autres noyaux avec 3 × 3,256 et 1 × 1,1024. Ceci est répété 6 fois donnant un total de 18 couches.
- Puis, encore un noyau 1 × 1,512 avec deux autres de 3 × 3,512 et 1 × 1,2048. Ceci a été répété 3 fois donnant un total de 9 couches.
- Après, nous faisons un pool moyen et le terminer avec une couche entièrement connectée contenant 1000 nœuds et à la fin une fonction softmax donc donne 1 couche.

| Layer | Kernel Size | Stride | Padding | Output Size |
|---|---|---|---|---|
| Input | | | | [224 × 224 × 3] |
| Conv1 | 7 × 7 × 3 | 2 | 3 | [112 × 112 × 64] |
| Max pool | 3 × 3 | 2 | - | [56 × 56] |
| Conv2 | [1×1conv,64],[3 × 3conv,64],1 × 1conv,256] | 2 | - | [56 × 56] |
| | [1×1conv,64],[3 × 3conv,64],1 × 1conv,256] | 1 | - | |
| | [1×1conv,64],[3 × 3conv,64],1 × 1conv,256] | 1 | - | |
| Conv3 | [1×1conv,128],[3 × 3conv,128],[1 × 1conv,512] | 2 | - | [28 × 28] |
| | [1×1conv,128],[3 × 3conv,128],[1 × 1conv,512] | 1 | - | |
| | [1×1conv,128],[3 × 3conv,128],[1 × 1conv,512] | 1 | - | |
| | [1×1conv,128],[3 × 3conv,128],[1 × 1conv,512] | 1 | - | |
| Conv4 | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 2 | - | [14 × 14] |
| | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 1 | - | |
| | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 1 | - | |
| | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 1 | - | |
| | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 1 | - | |
| | [1×1conv,256],[3 × 3conv,256],[1 × 1conv,1024] | 1 | - | |
| Conv5 | [1×1conv,512],[3 × 3conv,512],[1 × 1conv,2048] | 2 | - | [7 × 7] |
| | [1×1conv,512],[3 × 3conv,512],[1 × 1conv,2048] | 1 | - | |
| | [1×1conv,512],[3 × 3conv,512],[1 × 1conv,2048] | 1 | - | |
| Average pool | 7 × 7 | 7 | - | [1 × 1] |
| fc1000 | | | | 1000 |
| softmax | | | | |

**Figure 2.23:** Architecture du ResNet50.

[28]

### 2.6.3 MobilNet

C'est un modèle développé par Google pour les applications de vision par mobiles. Il a été prouvé que ce modèle réduit les coûts de calcul d'au moins 9 fois. MobileNet utilise des convolutions séparables. Dans cette architecture, une " Depthwise separable convolution " suivie par une " Pointwise convolution " a été utilisée [29], comme c'est indiqué dans la figure ci-dessous.

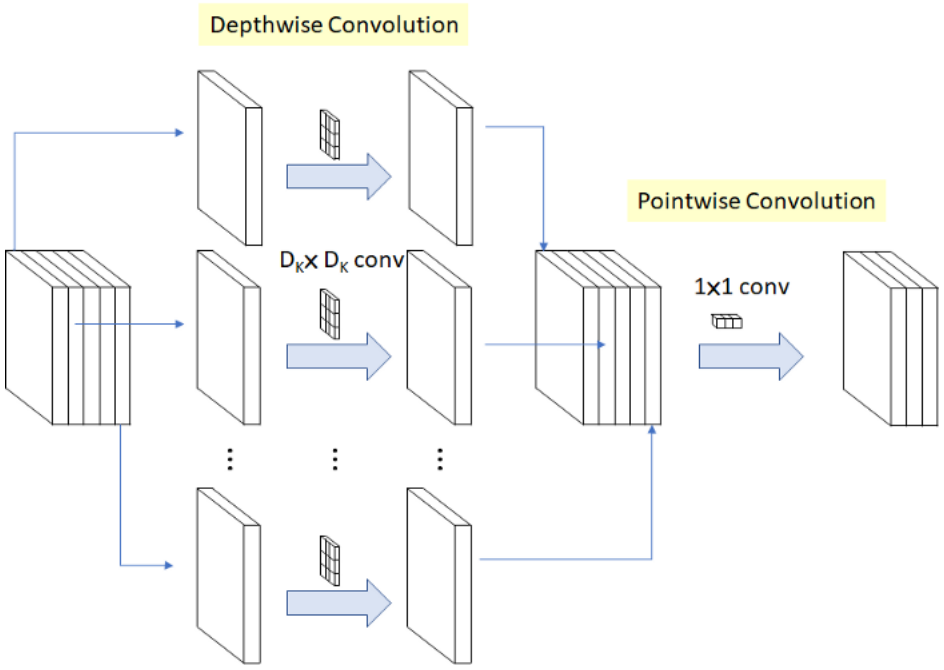


**Figure 2.24:** Les convolutions séparables utilisées dans l'architecture MobilNet. [29]

— La "pointwise convolution" : est une convolution de taille 1×1, elle permet de traiter l'image pixel par pixel et d'extraire une seule valeur à partir des différents canaux de chaque pixel.

— La "Depthwise Convolution" : consiste à appliquer un filtre sur chaque canal de l'image d'origine, contrairement à la convolution classique qui applique le même filtre sur tous les canaux. Mais pour les deux convolutions le résultat final est obtenu en combinant les sorties des différents canaux.

Ces convolutions séparables permettent d'économiser les calculs et d'alléger la mémoire sans impacter la précision du modèle. Le MobilNet a donné une précision de classification égale à 89% pour le jeu de données "ImageNet"[30].

### 2.6.4 Métrique d'évaluation

L'évaluation des algorithmes utilisés, est une étape indispensable pour la réalisation d'un projet classification. Dans ce projet, nous avons utilisé les mesures de validation suivantes :

- La Précision de la classification "classification Accuracy" : est le rapport entre le nombre des échantillons correctement classés et le nombre de toutes les prédictions effectuées.

$$Précision = \frac{\text{nombre des prédictions correctes}}{\text{nombre total des pr}édictions} \tag{2.12}$$

  La précision de classification ne prend en compte que l'exactitude de la prédiction, c'est-à-dire si l'étiquette prédite est la même que la vraie étiquette. Mais ceci est insuffisant parce que la probabilité avec laquelle nous classons l'état du conducteur est très importante pour évaluer les performances de notre modèle.

- Perte logarithmique "Logarithmic Loss" (LogLoss) : cette mesure prend en compte la probabilité de classification. En utilisant la LogLoss, le modèle doit attribuer une probabilité à chaque classe pour tous les échantillons. Supposons que nous avons N échantillons appartenant à M classes :

$$LogLoss = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{M}(Y_{ij}.log(P_{ij})) \tag{2.13}$$

  Avec :

  $Y_{ij}$ indique si l'échantillon $i$ppartient à la classe $j$ ou non.

  $P_{ij}$ la probabilité d'appartenance de l'échantillon $i$ à la classe $j$.

  L'objectif de nos modèles d'apprentissage automatique est de minimiser cette valeur. Un modèle parfait aurait une perte logarithmique de 0.

- Matrice de confusion "Confusion Matrix" : comme son nom l'indique, cette mesure nous donne une matrice en sortie et elle décrit la performance totale du modèle. Les colonnes de la matrice représentent les observations réelles, tandis que les lignes correspondent aux prédictions du modèle. Si nous disposons d'un ensemble de données contenant $N$ échantillons, tels que pour chaque échantillon la classe d'appartenance $Y$ est connue ainsi que la prédiction $\widehat{Y}$ fournis par un modèle, il est alors facile de construire la matrice de confusion de ce modèle. Si $Y$ un nombre quelconque de modalités, $Y \in$ {1,2,3..q}, la matrice de confusion M est de la forme suivante :

| | **Y=1** | **Y=2** | **...** | **Y=q** |
|---|---|---|---|---|
| $\widehat{\mathbf{Y}}$**=1** | $m_{1,1}$ | $m_{1,2}$ | ⋯ | $m_{1,q}$ |
| $\widehat{\mathbf{Y}}$**=2** | $m_{2,1}$ | $m_{2,2}$ | ⋯ | $m_{2,q}$ |
| **...** | ⋯ | ⋯ | ⋯ | ⋯ |
| $\widehat{\mathbf{Y}}$**=q** | $m_{q,1}$ | $m_{q,2}$ | ⋯ | $m_{q,q}$ |

**Figure 2.25:** Forme du matrice de confusion

Si le modèle de classification ne fait aucune erreur, c'est-à-dire qu'il prédit toujours la classe réelle, alors la matrice de confusion est diagonale. Lorsque le modèle de classification se trompe, les effectifs des couples ($Y = m$, $\widehat{y} = n$) tels $n \neq m$ que indiquent comment les erreurs faites se répartissent.

## 2.7 Technologies utilisées pour la reconnaissance facile du conducteur

La reconnaissance faciale est une catégorie de la biométrie qui cartographie mathématiquement les traits du visage d'un individu et stocke les données sous forme d'empreinte faciale. Le logiciel utilise des algorithmes d'apprentissage en profondeur pour comparer une capture en direct ou une image numérique stockée afin de vérifier l'identité d'un du conducteur.

La reconnaissance faciale est un processus qui consiste à détecter, à extraire les caractéristiques et à identifier ou vérifier l'identité du conducteur sur la base d'une photographie de son visage.

### 2.7.1 Détection de visage

La détection des visages est la méthode qui permet de découvrir tous les visages possibles à différents endroits avec différentes tailles dans une image donnée ou vidéo. Il existe des algorithmes de détection de visages tel que :

- Algorithme de Viola-Jones(V-J) : Développé par Paul Viola et Michael Jones en 2001, l'approche Viola-Jones Object Detection peut détecter rapidement et précisément les objets

dans les images et fonctionne particulièrement bien avec le visage humain. Malgré son âge, le framework est toujours un acteur de premier plan dans la détection de visage aux côtés de plusieurs de ses homologues CNN. Le cadre de détection d'objets Viola-Jones combine les concepts de fonctionnalités de type Haar (Haar-like Features), d'images intégrales (Integral Images), de l'algorithme AdaBoost et du classificateur en cascade (Cascade Classifier) pour créer un système de détection d'objets rapide et préci[31].

- L'histogramme des gradients orientés (HOG) [32] : est un descripteur de caractéristiques utilisé pour détecter des objets en vision par ordinateur et en traitement d'image. La technique de descripteur HOG compte les occurrences de l'orientation du gradient dans des parties localisées d'une fenêtre de détection d'image ou d'une région d'intérêt (ROI).

  L'implémentation de l'algorithme de descripteur HOG est la suivante :

  — Divisez l'image en petites régions connectées appelées cellules et calculez, pour chaque cellule, un histogramme des directions de gradient ou des orientations des bords pour les pixels de la cellule.

  — Discrétisez chaque cellule en forme rectangulaire en fonction de l'orientation du gradient.

  — Le pixel de chaque cellule contribue un gradient pondéré à sa corbeille angulaire correspondante.

  — Les groupes de cellules adjacentes sont considérés comme des régions spatiales appelées blocs. Le regroupement de cellules dans un bloc constitue la base du regroupement et de la normalisation des histogrammes.

  — Le groupe normalisé d'histogrammes représente l'histogramme de bloc. L'ensemble de ces histogrammes de blocs représente le descripteur.

  La figure 2.26 illustre le schéma de mise en œuvre de l'algorithme :

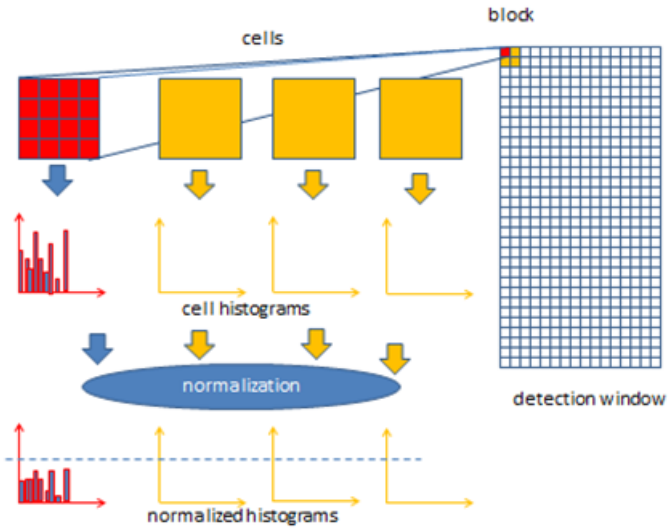


**Figure 2.26:** Schéma de l'algorithme de HOG.

[33]

- Mixture-Model-based Object Detector (MMOD) : Il s'agit d'un classificateur basé sur un réseau de neurones convolutifs CNN de la bibliothèque Dlib. Comme la figure 2.27 montre, le réseau de neurones MMOD a essentiellement huit couches dont elles sont constituées de cinq couches de convolution et trois couches de Fully-Connected.

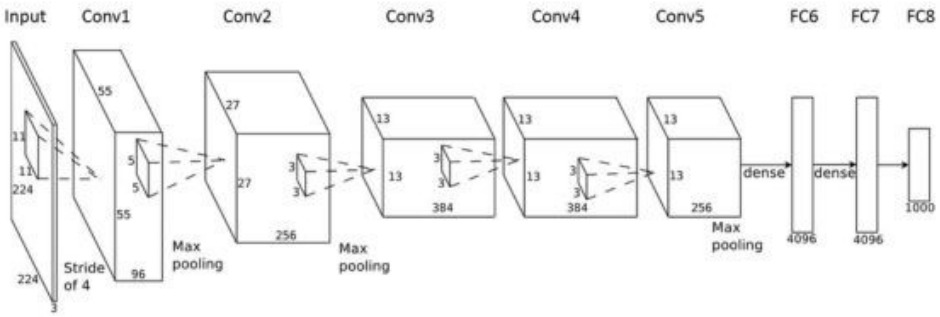


**Figure 2.27:** Illustration de l'architecture de MMOD

[34]

### 2.7.2 Extraction des caractéristiques faciales

L'extraction de caractéristiques est l'étape d'initialisation de base et la plus importante pour la reconnaissance faciale. Il extrait les composants biologiques un visage. Ces composants biologiques sont les caractéristiques d'un visage qui diffèrent d'une personne à l'autre. Il existe différentes

méthodes qui extraient diverses combinaisons de caractéristiques, communément appelées points nodaux. Aucune personne ne peut avoir tous les points nodaux similaires les uns aux autres, à l'exception des jumeaux identiques.

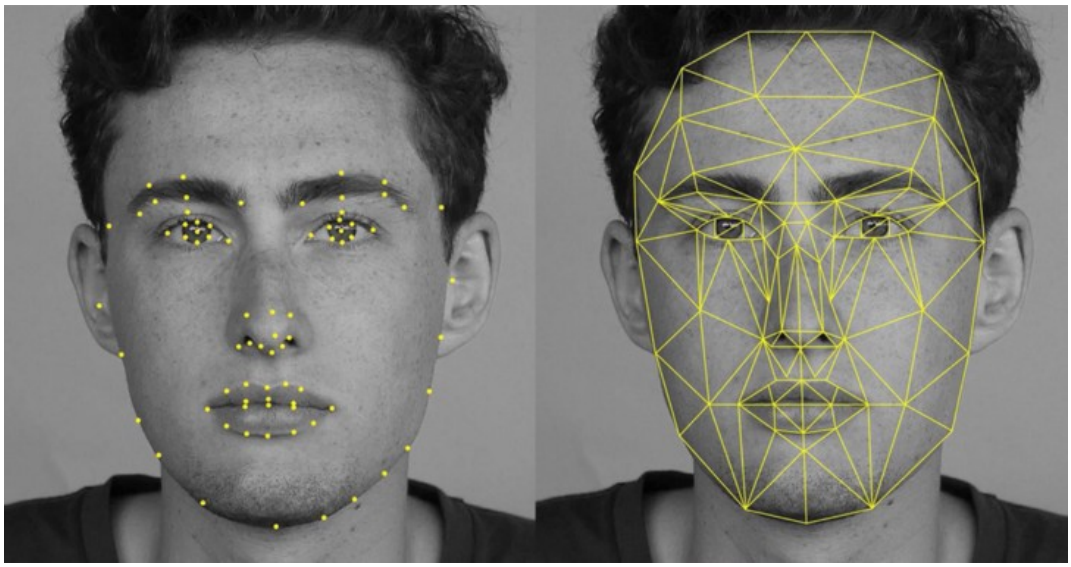

**Figure 2.28:** 128 masques de visage

[35]

L'extraction des traits du visage a deux approches :

- **Approche peu profonde**
  - Principal Component Analysis (PCA) : est utilisé pour réduire la dimensionnalité des données. Dans PCA, les caractéristiques originales de votre jeu de données seront converties en une combinaison linéaire de variables non corrélées. Ces combinaisons sont appelées composants principaux. PCA augmente les performances de l'algorithme et améliore la visualisation. Mais pour lire les composants PCA, il faut standardiser les données sinon il ne sera pas possible de trouver les composants principaux optimaux. De plus, l'PCA entraîne une perte d'informations si le nombre de composantes principales n'est pas sélectionné judicieusement.
  - Latent Dirichlet Allocation (LDA) : est une technique de réduction de dimensionnalité utilisée pour classer différentes classes en fonction des caractéristiques des données supervisées. L'inconvénient majeur de la LDA est le soi-disant problème de petite taille d'échantillon et problème de non-linéarité. Le problème se produit lorsque l'échantillon (données d'entraînement) est assez petit par rapport à la dimension des données.

- **Approche profonde**
  - VGG : utilise diverses architectures telles que VGG19, VGG16 de Keras. La différence fondamentale entre ces modèles est le nombre de couches incluses dans son architecture qui varie d'un modèle à l'autre. Ces modèles ont une assez bonne précision.
  - FaceRecognition API : est plus simple à utiliser. Il a une architecture beaucoup plus facile à implémenter avec certaines bibliothèques intégrées requises pour la reconnaissance des fonctionnalités. L'API utilise les informations de l'utilisateur pour extraire les points de reconnaissance.

  Cette approche fonctionne bien en temps réel et possède une bonne précision, c'est pour cela nous allons l'utiliser dans ce projet.

### 2.7.3 Classification des visages

Pour identifier le visage détecté il nous faut un algorithme de classifications performants et précis tel que :

- K-nearest neighbors (K-NN) : La classification K-plus proche voisin ou K-nearest neighbor (K-NN) est l'un des plus simples de tous les algorithmes d'apprentissage automatique supervisé, qui a été utilisé comme classificateur de base dans de nombreux problèmes de classification des objets. Il est basé sur la mesure des distances entre les données de test et chacune des données d'entraînement (training set) pour décider la sortie de classification finale. Il existe plusieurs types de distances pouvant être utilisées dans l'algorithme K-NN. On peut citer : la distance euclidienne, Chi-square, Minkowski, cosinus, etc. Mais, la distance la plus utilisée est la distance euclidienne qui a pour formule mathématique suivante.

$$D(X1, X2) = \|X1 - X2\|^2 \tag{2.14}$$

  Donc, comme illustré par la figure 2.29, K-NN sélectionne alors les K points de données les plus proches, où K peut être n'importe quel entier. Enfin, il affecte le point de données à la classe à laquelle appartiennent la majorité des K points de données.

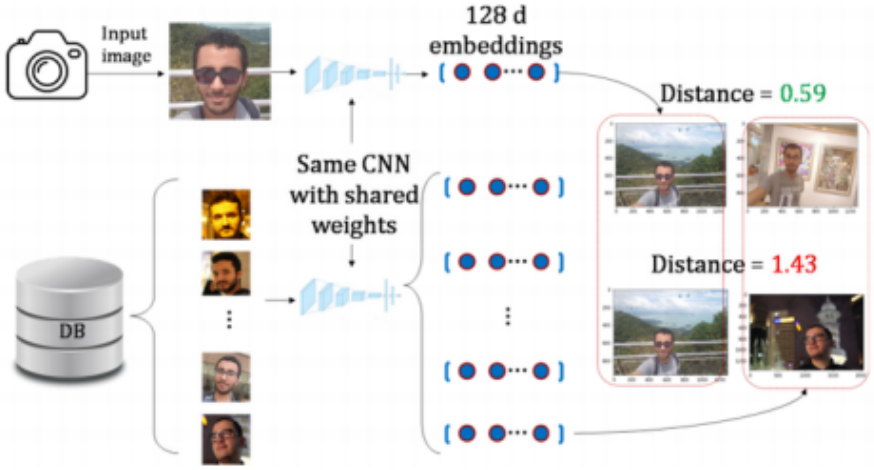


**Figure 2.29:** Classification par la distance euclidienne

[36]

— Support Vector Machine (SVM) : Un algorithme d'apprentissage automatique supervisé qui peut être utilisé à la fois pour des défis de classification ou de régression. Cependant, il est principalement utilisé dans les problèmes de classification. Dans l'algorithme SVM, chaque élément de données est tracé comme un point dans un espace à n dimensions (où n est le nombre de caractéristiques) avec la valeur de chaque caractéristique étant la valeur d'une coordonnée particulière. Ensuite,la classification est effectuée en trouvant l'hyper-plan qui différencie très bien les deux classes comme illustré dans la figure suivante.

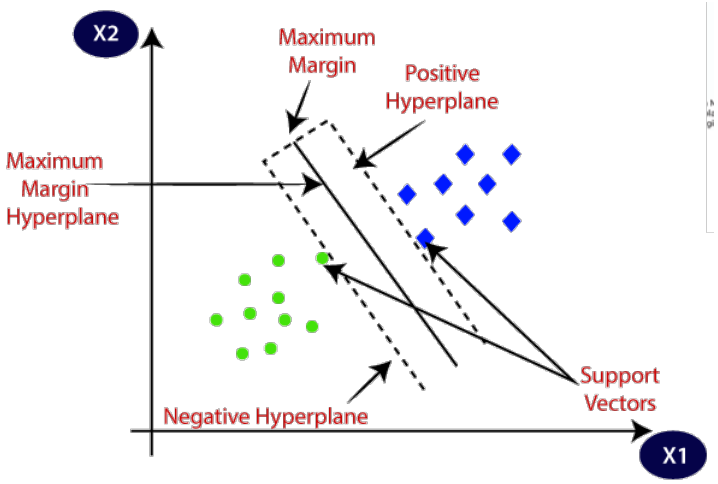


**Figure 2.30:** Séparation entre classe par un hyper plan

[37]

Pour l'évaluation de cette partie, nous allons utiliser les métriques d'évaluation définie et expliquée dans les section 2.5.4 et 2.6.1.

## Conclusion

Dans ce chapitre, quelques notions de base nécessaires à la réalisation du projet ont été définies. Nous avons présenté les concepts de l'apprentissage profond ainsi que les algorithmes et les mesures qui seront utilisées pour répondre à la problématique de ce projet. Dans le chapitre suivant, nous allons présenter la réalisation de la solution.

*Chapitre 3*

# Implémentation des solutions OMS

## Plan


**Introduction** . . . . . **47**

**1 Présentation de l'environnement du travail** . . . . . **47**

**2 Détection des occupants et des objets** . . . . . **49**

**3 La classification de l'état de distraction du conducteur** . . . . . **63**

**4 La reconnaissance faciale du conducteur** . . . . . **79**

**5 L'intégration des différentes parties du système de surveillance de l'intérieur de la voiture** . . . . . **84**

**6 Déploiement de la solution développée** . . . . . **86**

**Conclusion** . . . . . **89**

# Introduction

Ce dernier chapitre vise à décrire d'abord l'environnement de travail matériel et logiciel utilisés. Ensuite les étapes de réalisation du projet seront détaillées et évaluées en utilisant les algorithmes présentés dans le chapitre précédent. Finalement l'implémentation de la solution finale qui intègre les trois parties ensemble sur une carte embarquée sera expliquée.

## 3.1 Présentation de l'environnement du travail

Tout au long de la réalisation de ce projet, nous avons utilisé des matériels et des logiciels bien particuliers qu'on présente dans ce qui suit.

### 3.1.1 Environnements matériels de travail

Les matériaux utilisés pour développer et valider les fonctionnalités de la solution appartiennent à SAGEMCOM. L'organisme d'accueil nous a fourni trois équipements matériels nécessaires à la réalisation de ce projet :

- Un ordinateur dont les caractéristiques sont les suivantes :

| **Modèle** | DELL |
|---|---|
| **Processeur** | Intel ® Core™ i5-8500 CPU @ 3.00GHZ |
| **Mémoire RAM** | 32G |
| **Disque dur** | 236G |
| **Système d'exploitation** | Windows 10 |

**Tableau 3.1:** Caractéristiques de l'orinateur

- Un Kit de développement "Nvidia Jetson Nano" (va être bien expliqué dans la dernière section de ce chapitre) .
- Une Web Caméra avec une résolution 640 × 480.

### 3.1.2 Environnement logiciel de travail

Pour développer la solution "Système de surveillance de l'intérieur de voiture", un large éventail de logiciels et de technologies sont utilisées.

- Langage de programmation
  - **Python** [38] : c'est un langage de programmation de haut niveau conçu pour être facile à lire et simple à implémenter. Il est open source et est souvent utilisé pour créer des applications Web et du contenu Web dynamique.
- Outil de programmation
  - **PyCharm**[39] : c'est l'un des environnements de développement intégré (IDE) Python les plus connus, développé par une organisation tchèque appelée JetBrains. Il prend également en charge le développement web.
  - **Google Colab** [40] : permet d'écrire et d'exécuter du code Python dans le navigateur avec un partage facile, accès gratuit aux GPU et aucune configuration requise.
- Outil de prétraitement de données
  - **LabelImg**[41] : fournit une interface utilisateur qui permet de dessiner des boites englobantes et de saisir les axes x et y ainsi que la hauteur et la largeur du cadre de sélection. Il permet également de marquer plusieurs objets dans une seule image. Cet outil stocke la valeur de l'axe dans un fichier distinct qui existe au même emplacement de l'image.
  - **Roboflow**[42] : est un outil de gestion, de prétraitement et d'enrichissement des données pour la vision par ordinateur. Les développeurs l'utilisent pour automatiser la qualité des annotations, réduire le temps de formation et optimiser la reproductibilité des modèles.
- les bibliothèques utilisées :
  - **TensorFlow** [43] : c'est une bibliothèque d'apprentissage automatique open source créée par Google pour développer et exécuter des applications d'apprentissage automatique et d'apprentissage en profondeur.
  - **Keras** [44] : est une API de haut niveau de TensorFlow permettant de créer et d'entraîner des modèles de DL.

— **OpenCV**[45] : c'est une bibliothèque de vision par ordinateur et d'apprentissage par machine à code source ouvert. OpenCV a été construite pour fournir une infrastructure commune pour les applications de vision par ordinateur et pour accélérer l'utilisation de la perception des machines dans les produits commerciaux.

— **NumPy**[46] : la bibliothèque NumPy permet d'effectuer des calculs numériques avec Python. Elle introduit une gestion facilitée des tableaux de nombres.

— **SciPy**[47] : c'est une bibliothèque numérique d'algorithmes et de fonctions mathématiques pour améliorer, en termes de performances, les fonctions de NumPy.

— **Dlib**[48] : est une bibliothèque C++ open source qui met en œuvre une variété d'algorithmes d'apprentissage automatique, y compris la classification, la régression, le regroupement, la transformation des données, la prévision structurée et aussi il est très utilisé pour la détection de visage.

— **CUDA et cuDNN**[49] : bibliothèquess de bas niveau, ils permettent l'entraînement de l'algorithme sur un GPU, ce qui le rend beaucoup plus rapide et efficace que l'entraînement sur un CPU.

## 3.2 Détection des occupants et des objets

### 3.2.1 Collecte des données

L'un des éléments clés de la création d'un système d'apprentissage automatique est la collecte de bases de données de haute qualité. D'ailleurs, pour entraîner un modèle basé sur des données conventionnelles, l'acquisition des données est une étape primordiale. Pour cela nous avons essayé de colleter une base de donnée bien riche et de bonne qualité.

Nous avons 10 objets d'intérêts à détecter qui sont : Adulte, Chien, Bébé, Enfants, ceinture de sécurité, ceinture de sécurité pour bébé, phone, portefeuille, sac à main et sac à dos.

Pour collecter tous ces données, nous avons utilisées deux moyens de collections d'images :

— Google's Open Images Dataset[50] qui est un moyen de collecte de données efficace. En effet, il permet d'obtenir non seulement les images mais aussi leurs annotations. Mais malheureusement

Google's Open Images Dataset ne contient que les classes « Dog », «Cell phone» et «Back pack». C'est la raison pour laquelle nous avons eu recours à Google Image Search pour l'obtention du jeu de données nécessaires.

— Google Image Search est l'un des moyens les plus simples à utiliser. Cependant, télécharger manuellement les images qui apparaissent dans les résultats de recherche est une tâche fastidieuse. Pour résoudre ce problème, nous avons eu recours à un algorithme de web scrapping. Cet algorithme permet le téléchargement en masse les images qui répond au mots clés données dans une page Web. Ensuite, il suffit de parcourir ces images et de supprimer celle non pertinentes.

Par le biais des moyens cité ci-dessus, 6 285 images ont été collectées ( 600 images par classe).

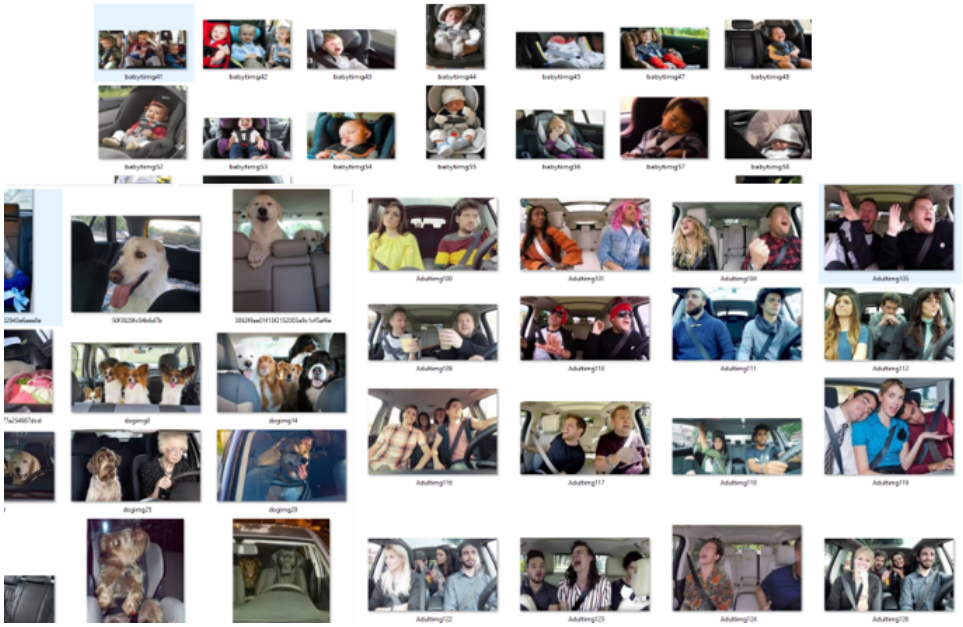

**Figure 3.1:** Une portion la base de données collectée

### 3.2.2 Annotation des données

Une fois les données sont collectées, l'étape suivante consiste à les annoter. D'ailleurs l'annotation des images est une technique essentielle servant à créer des données d'entraînement. En effet, afin que les machines puissent percevoir les objets dans leur milieu, les images annotées sont nécessaires pour entraîner les algorithmes d'apprentissage. Les étiquettes sont choisies pour identifier au modèle de vision par ordinateur des informations sur différents objets représentés dans une image.

Dans le contexte de la détection d'objets, le marquage consiste à dessiner des cadres de délimitation autour des objets à détecter dans l'image et à les associer aux classes d'objets correspondantes pour les montrer clairement à la machine. Par conséquent, nous étiquetons les objets d'intérêt dans chaque image afin d'obtenir un ensemble de données personnalisé.

Pour la détection d'objet, le modèle inclut certains aspects à reconnaître à partir de l'image, tels que l'axe X, l'axe Y, la hauteur et la largeur de l'objet dans l'image.

Pour l'annotation des images de notre base de données, nous utilisons le logiciel labelImg. Il nous permet de dessiner manuellement les boîtes englobantes pour chaque objet d'intérêt.

Les annotations générées par label images ont été enregistrées sous format YOLO [object-id, centerx, centery, width, height] comme ilustré dans la figure 3.2 :

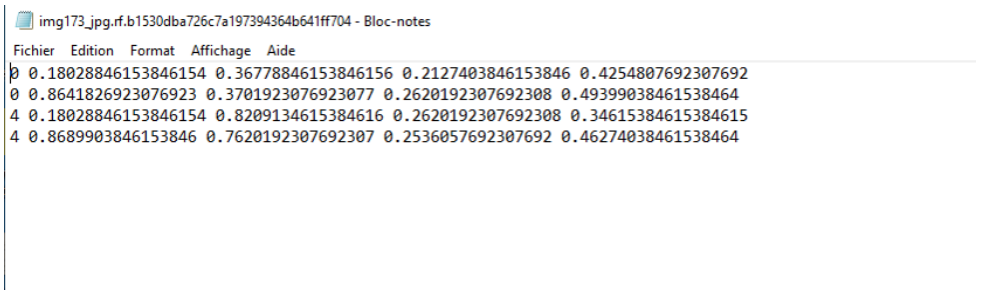


**Figure 3.2:** Annotation format YOLO

avec :

- Object-id est l'identifiant de la classe à laquelle appartient l'objet.
- Centerx et centery sont les coordonnées du centre de la boite englobante.
- Width et height sont, respectivement, la largeur et la hauteur de la boite englobante.

La figure suivante montre un exemple d'image de notre base de données étiqueté par labelImg :

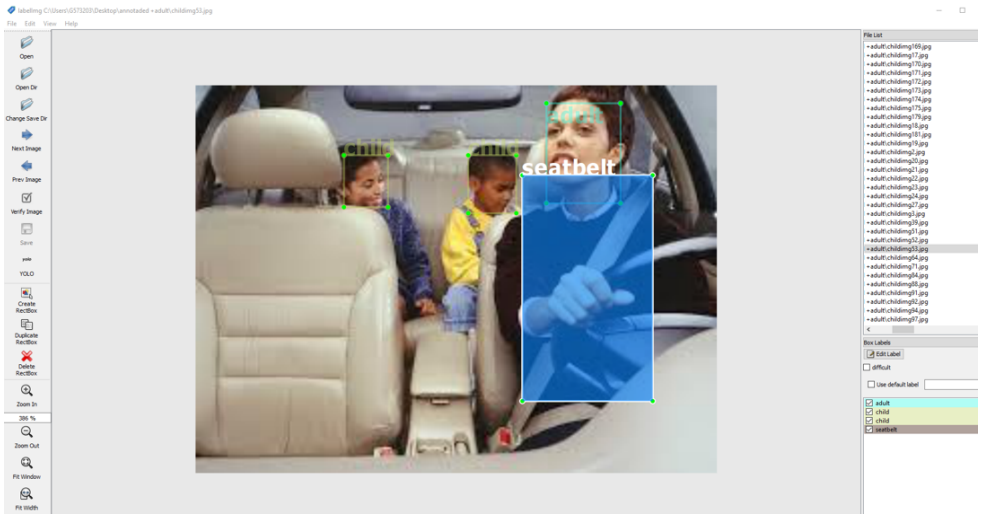


**Figure 3.3:** Étiqueter les données à l'aide de LabelImg

Puisque nous avons besoin avoir les annotations sous format PascalVOC afin d'entrainer le fasterRcnn et SSD. Nous avons utilisé un script python pour convertir les annotations au format désiré.

```
<?xml version="1.0" ?><annotation>
 <folder/>
 <filename>frame10.jpg</filename>
 <size>
  <width>1920</width>
  <height>1080</height>
  <depth>3</depth>
 </size>
 <object>
  <name>Adult</name>
  <difficult>0</difficult>
  <bndbox>
   <xmin>152</xmin>
   <ymin>227</ymin>
   <xmax>460</xmax>
   <ymax>678</ymax>
  </bndbox>
 </object>
 <object>
  <name>Child</name>
  <difficult>0</difficult>
  <bndbox>
   <xmin>971</xmin>
   <ymin>347</ymin>
```

**Figure 3.4:** Annotation format PascalVOC

### 3.2.3 Augmentation des données

Les performances des réseaux neuronaux de DL augmentent généralement avec la quantité de données existantes. D'ailleurs, l'augmentation des données est une technique qui permet de créer de manière artificielle de nouvelles données à partir des données déjà existantes. En fait, elle permet d'améliorer la reproductibilité des modèles, en facilitant le débogage et l'amélioration des

performances.

Pour assurer cette étape, nous avons utilisé Roboflow. Il est un outil de gestion, de prétraitement et d'enrichissement des données pour la vision par ordinateur. La particularité de Roboflow est que l'augmentation de l'image s'accompagne par l'augmentation des boîtes de délimitation qu'elle contient donc nous n'avons pas besoin d'annoter de nouveau les images augmentées. De plus il offre plusieurs types d'augmentation les images (rotation, contraste, saturation, zoom, couleur, flip, bruit...) ce qui nous offre non seulement un nombre important de données mais aussi un jeu de données bien varié.

Les figures ci-dessous illustrent un exemple pour chaque transformation effectuée par l'augmentation des données :

**Figure 3.5:** Deux images générées de la même photo

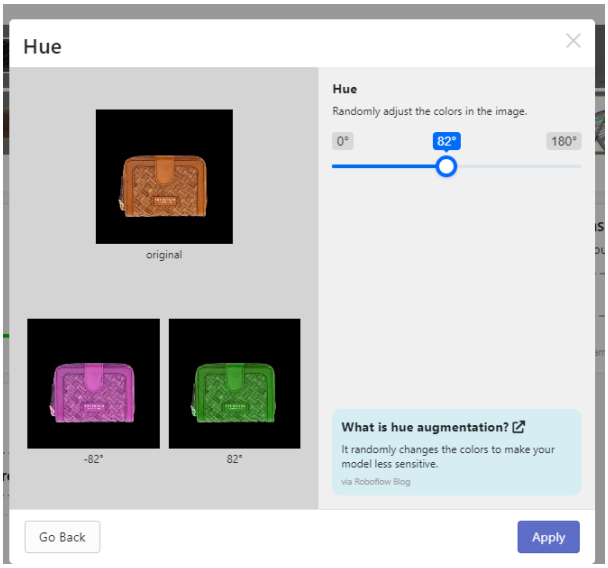


**Figure 3.6:** Data augmentation par roboflow

Cependant, l'inconvénient de Roboflow c'est qu'il inverse l'ordre des classes lors de l'augmentation de données. Donc nous avons fait recours à un script python pour corriger l'ordre des classes.

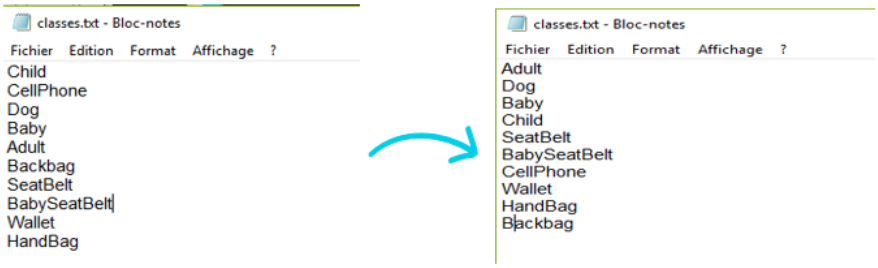


**Figure 3.7:** Correction des classes des données

Après l'application de l'augmentation des données sur la base de données collectée dans la section précédente. Nous avons obtenu un jeu de données qui contient **25082** images contenant **48211** Bounding box répartis comme la figure 3.8 :

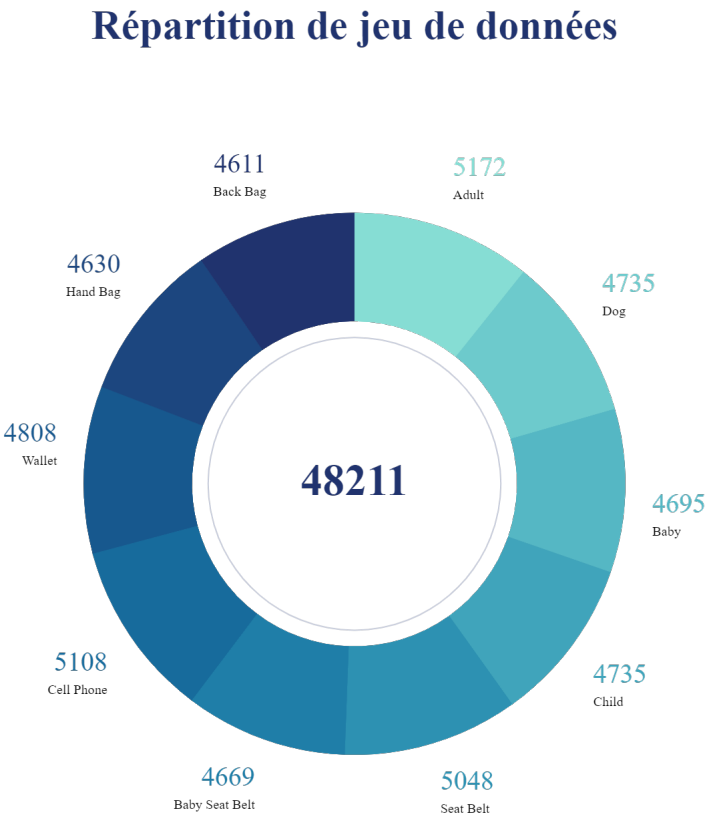


**Figure 3.8:** Répartition de jeu des données

### 3.2.4 Modélisation des détecteurs des occupants et des objets

Une fois notre jeu de donnée est prêt, nous allons maintenant attaquer la partie modélisation afin de savoir quel détecteur d'objet entre Yolov4, Faster RCNN et SSD est le plus performant pour notre cas d'étude et répond plus efficacement aux besoins.

#### 3.2.4.1 Modélisation avec YOLOv4

Pour entrainer ce modèle, nous utilisons Darknet, un Framework open source écrit en C et contient le code source de YOLO. D'ailleurs, DarkNet nécessite un fichier texte répertoriant toutes les images qui seront utilisées pour la formation. Généralement, de 60 à 90% de l'ensemble de données est utilisé pour l'entrainement et le reste pour les tests /validation. Nous avons, donc, créé un fichier train.txt qui contient les chemins de 20 076 images qui représentent le 80% de la base et un fichier test.txt qui contient les chemins de 5 006 images. Ceci est illustré par la figure 3.9.

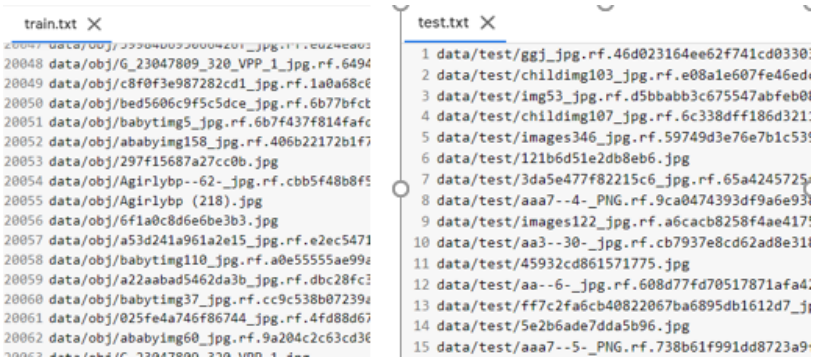


**Figure 3.9:** Fichiers train.txt et test.txt

Puis, nous créons deux fichiers. Le premier c'est « obj.data » qui contient le nombre de classes, les chemins des fichiers train.txt et test.txt et le chemin où le fichier ou le model sera stocké.et le deuxième c'est « obj.names » qui contient les catégories d'objets classées dans le même ordre que dans le fichier classes.txt. Chaque ligne du fichier créé fait référence à une catégorie de classe.

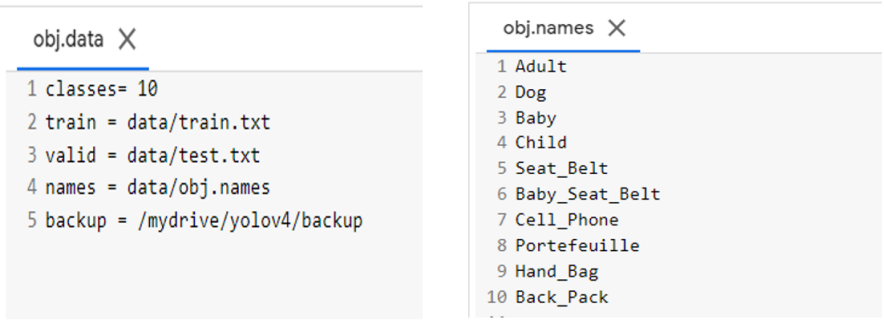


**Figure 3.10:** Fichiers obj.data et obj.names

Ensuite nous commençons la modélisation de notre Yolov4 par l'application de l'apprentissage par transfert. Tous d'abord, nous téléchargeons les poids des couches convolutives du réseau YOLOv4 darknet53.conv.137. En fait, L'utilisation de ces poids permet au détecteur des objets d'être plus précis et de ne pas avoir à s'entraîner aussi longtemps. Il aidera le modèle à converger rapidement. Après, nous configurons les paramètres de YOLOv4 tout en modifiant le contenu du fichier de configuration yolov4.cfg, cité dans darknet,. Nous choisissons les paramètres suivants :

— batch_size = 64.

— subdivision = 16.

— width = 416 , height = 416.

— max_batches = (#classes * 2000) soit alors max_batches = 20 000.

— steps = max_batches * 0.8, max_batches * 0.9 soit alors steps = 16 000, 18 000.

— filters = (classes + 5) * 3 soit alors filters = 45.

Enfin, nous entrainons le modèle sur google colab puisqu'il fournit des ressources GPU performantes et gratuites.

#### 3.2.4.2 Modélisation avec Faster RCNN et SSD

Les deux détecteurs des données Faster RCNN et SSD ont les mêmes étapes de pré traitement et de modélisation.

Pour la modélisation de Faster RCNN et SSD nous devons générer des TFRecords pour TensorFlow afin de pouvoir lire nos images et leurs étiquettes. TFRecord est un format de fichier

simple. Il permet de sauvegarder une séquence d'enregistrements binaires. Ce type de fichier utilise moins d'espace disque. Ce qui signifie que chaque opération de lecture et d'écriture est effectuée rapidement. En plus, ce format couvre tous les types de données : tableaux, textes, séries temporelles.

Donc pour pouvoir générer le fichier TFRecords , nous avons converti les fichiers XML des données en deux fichiers CSV composé chacun de 8 colonne comme montrer dans la figure 3.11. L'une de ces fichiers CSV contient les données de l'entrainement et l'autre contient les données de test.

| filename | width | height | class | xmin | ymin | xmax | ymax |
|---|---|---|---|---|---|---|---|
| cbn49p.jpg | 450 | 319 | Portefeuille | 58 | 16 | 346 | 272 |
| 56f286c4ab089d5b5 | 416 | 416 | Baby | 153 | 57 | 264 | 148 |
| 56f286c4ab089d5b5 | 416 | 416 | Baby_Seat_Belt | 137 | 134 | 263 | 257 |
| 0053a5d4ef4e66d3( | 416 | 416 | Adult | 1 | 0 | 195 | 37 |
| 0053a5d4ef4e66d3( | 416 | 416 | Baby | 207 | 40 | 354 | 82 |
| 0053a5d4ef4e66d3( | 416 | 416 | Baby_Seat_Belt | 84 | 285 | 283 | 375 |
| 0053a5d4ef4e66d3( | 416 | 416 | Baby | 114 | 244 | 260 | 285 |
| 0053a5d4ef4e66d3( | 416 | 416 | Baby_Seat_Belt | 164 | 75 | 292 | 118 |
| ababyimg40_jpg.rf.7 | 416 | 416 | Baby | 221 | 46 | 340 | 170 |
| 487772_png.rf.6a9b | 416 | 416 | Portefeuille | 40 | 64 | 381 | 367 |
| apital-leather-walle | 416 | 416 | Portefeuille | 0 | 116 | 406 | 307 |
| abytimg37_jpg.rf.0 | 416 | 416 | Baby | 56 | 141 | 129 | 238 |
| babytimg37_jpg.rf.0 | 416 | 416 | Baby_Seat_Belt | 65 | 240 | 198 | 351 |
| 16b6c22fbf91197c_ | 416 | 416 | Hand_Bag | 4 | 77 | 406 | 359 |

**Figure 3.11:** Résultat de conversion des données en fichiers CSV.

En utilisant le script de la détection d'objet TensorFlow les fichiers CSV précédemment générés, nous avons pu générer les deux fichiers TFRecords voulu.

L'étape suivante c'est la création des fichiers de mappage d'étiquettes (avec extension. pbtxt) qui mappe notamment chacune des étiquettes utilisées à une valeur entière.

Après, Nous téléchargeons les modèles préformés Faster R-CNN "faster_rcnn_resnet50_coco_2018_01_2" et SSD "ssd_resnet50_v1", pré-entrainés sur la base de données COCO 10. Cette étape est faite à l'aide d'une API de Tensorflow de détection d'objets.

Et avant de commencer l'entrainement, nous configurons les paramètres de chacun de deux modèles :

— Pour le FasterRCNN nous choisissions : batch_size=16, num_steps=200000, num_examples=8000 et num_class=10.

— Pour le SSD nous choisissions : batch_size=64, num_steps=200000, num_examples=8000 et num_class=10.

Une fois les fichiers et dossiers impliqués dans l'entrainement sont tous modifiés et bien configurés, nous entraînons finalement les modèles sur nos données sous format TFRecord en appliquant un apprentissage par transfert en utilisant le modèle de faster R-CNN et SSD téléchargés.

### 3.2.5 Résultat et évaluation des modèles

Suite à l'entrainement des trois modèles de détection des objets, l'étape suivante consiste à évaluer leurs performances. Le tableau ci-dessous décrit les performances des modèles en termes de mAP, Précision et rappel.

| **Modèles** | YOLOv4 | Faster RCNN | SSD |
|---|---|---|---|
| **mAP** | 82,9% | 94,2% | 89,7% |
| **Précision** | 85,3% | 91% | 82,9% |
| **Rappel** | 68% | 90,2% | 78% |

**Tableau 3.2:** Performances des modèles.

Pour mieux évaluer nos modèles et afin de savoir le modèle le plus performant entre ces trois, nous avons décidé de calculer les KPIs de trois vidéos tests. Et pour ce faire, nous avons annoté avec LabelImg les frames de chaque vidéo dans le but d'avoir les boîtes englobantes de ground truth. Ensuite, nous comparons les vidéos idéales avec les vidéos testées par chacun des algorithmes. Les résultats comparatifs sont affichés dans les tableaux ci-dessus.

- Vidéo n°1 :

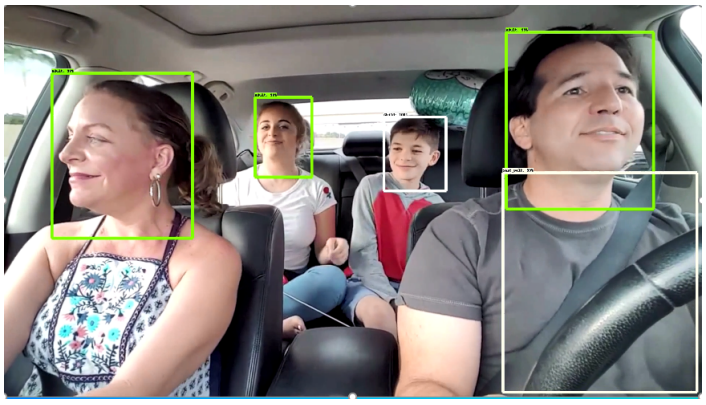

**Figure 3.12:** Frame de vidéo n°1.

Résultats pour un seuil égal à 0,6 :

| **Algorithmes** | Précision | Recall | mAP | FPS |
|---|---|---|---|---|
| **Faster RCNN** | 90% | 85% | 83% | 19 |
| **YOLOv4** | 70% | 70% | 60% | 38 |
| **SSD** | 74% | 80% | 79% | 21 |

**Tableau 3.3:** Résultats vidéo n°1

- Vidéo n°2 :

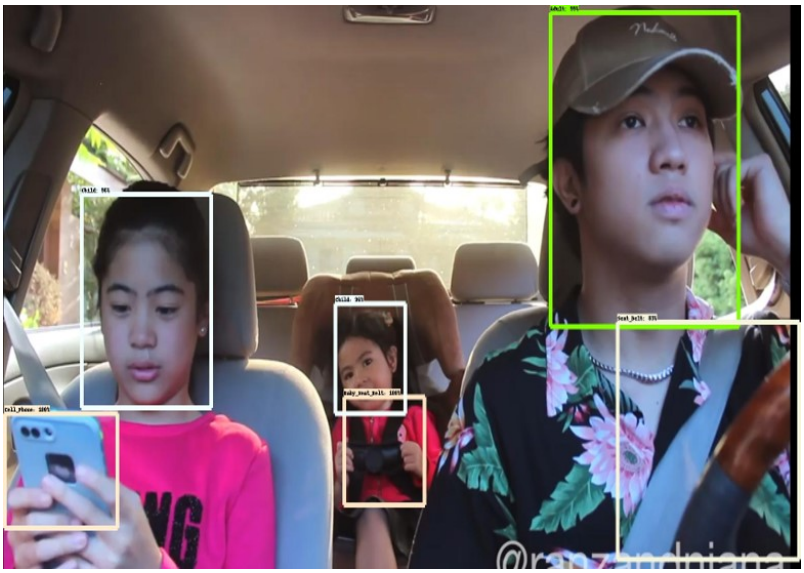

**Figure 3.13:** Frame de vidéo n°2.

Résultats pour un seuil égal à 0,6 :

| **Algorithmes** | Précision | Recall | mAP | FPS |
|---|---|---|---|---|
| **Faster RCNN** | 91% | 96% | 92% | 19 |
| **YOLOv4** | 84% | 92% | 79% | 38 |
| **SSD** | 78% | 89% | 85% | 21 |

**Tableau 3.4:** Résultats vidéo n°2

- Vidéo n°3 :

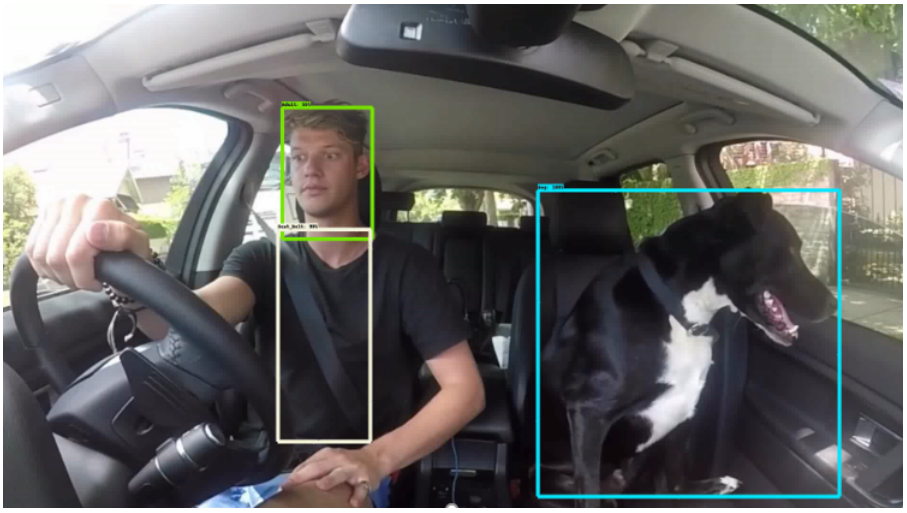

**Figure 3.14:** Frame de vidéo n°3.

Résultats pour un seuil égal à 0,6 :

| **Algorithmes** | Précision | Recall | mAP | FPS |
|---|---|---|---|---|
| **Faster RCNN** | 85% | 90% | 87% | 19 |
| **YOLOv4** | 79% | 78% | 67% | 38 |
| **SSD** | 77% | 81% | 75% | 21 |

**Tableau 3.5:** Résultats vidéo n°3

### 3.2.6 Choix du modèle

D'après les résultats d'évaluation des modèles présentés tableaux ci-dessus, nous pouvons déduire que l'algorithme le plus précis et le plus performant dans notre cas est le Faster Rcnn. Malgré qu'il est l'algorithme le plus lent mais il a un temps de traitement acceptable et suffisant pour une détection en temps réel. Par conséquent, nous allons choisir Faster Rcnn comme étant le détecteur de objets d'un système de surveillance de l'intérieur de voiture.

Les figures ci-dessous illustrent des résultats du modéle choisi sur des images.

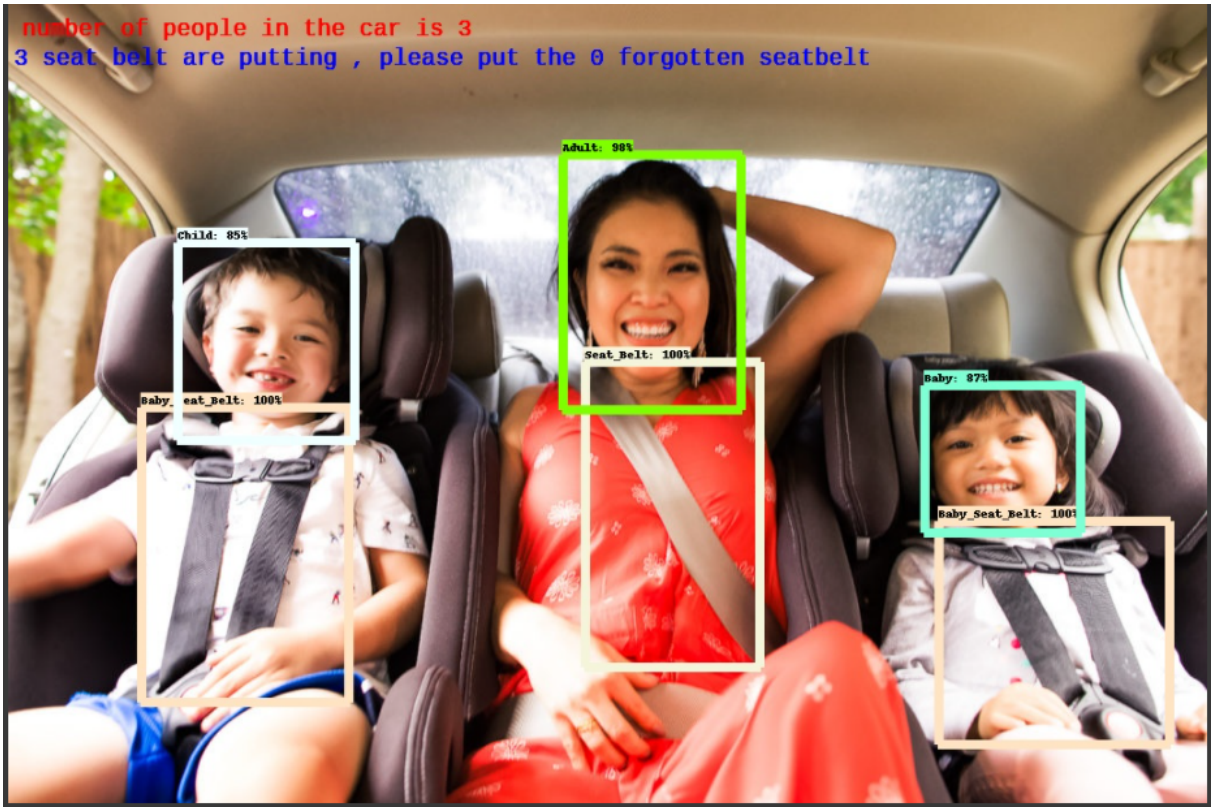


**Figure 3.15:** Résultat n°1

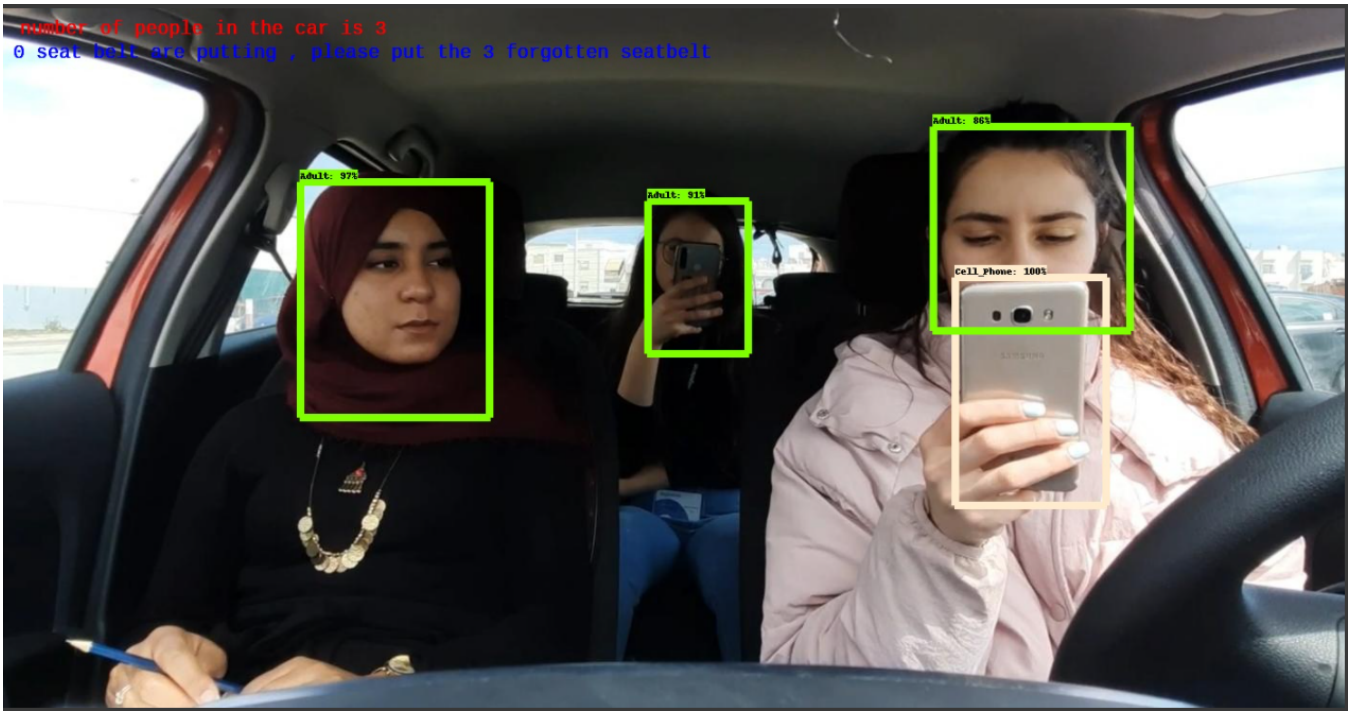


**Figure 3.16:** Résultat n°2

**Figure 3.17:** Résultat n°3

**Figure 3.18:** Résultat n°4

## 3.3 La classification de l'état de distraction du conducteur

L'objectif de cette partie est de classifier l'état d'un conducteur, en se basant sur les images provenant d'une caméra frontale qui couvre une partie de la silhouette conductrice, et ce en utilisant les techniques classification multi-classes de l'apprentissage profond.

Dans ce projet nous sommes focalisés à la classification de 7 états de conducteur qui appartient à ces 3 types de distraction : distractions manuelles, distractions visuelles et distractions cognitives. Le premier état à classifier c'est l'état de conduite sure et les 6 autres états sont représentés dans le tableau suivant avec leurs types de distraction :

| **Classes** | **Type de distraction** |
|---|---|
| **C1** : Boire (Drinking) | manuelle |
| **C2** : Parler avec un passager (Talking to passenger) | visuelle +cognitive |
| **C3** :Parler au téléphone (Talking on the phone) | cognitive +manuelle |
| **C4** :Chercher des objets (Reaching behind ) | visuelle + manuelle |
| **C5** : Ecrire un message(Texting ) | visuelle + manuelle |
| **C6** : Fumer la cigarette (Smoking ) | manuelle |

**Tableau 3.6:** Types et classes de distraction

### 3.3.1 Collecte des données

La collecte des données représente l'étape la plus importante pour un sujet de classification multi classe. Le modèle résultant ne peut être aussi efficace que le jeu de données, duquel il est construit, demeure riche, propre et pertinent.

Après une recherche profonde des bases des données gratuites pour ce sujet, nous n'avons pas trouvé une base de données qui peut nous permets d'attendre l'objectif du projet. Par conséquent nous avons décidé de créer notre propre base de données.

Pour avoir une base de données riche et diversifiée, nous avons filmé des vidéos par une caméra frontale intégrée à l'intérieur d'une voiture, en simulant les 7 États voulus avec plusieurs personnes pendant leur conduite sous des conditions différentes (luminosité, port des lunettes ou pas...) et dans

7 voitures différentes.17 personnes ont participé à la création de cette base de données dont 7 filles et 10 garçons. La figure suivante montre un échantillon des vidéos de notre base de données.

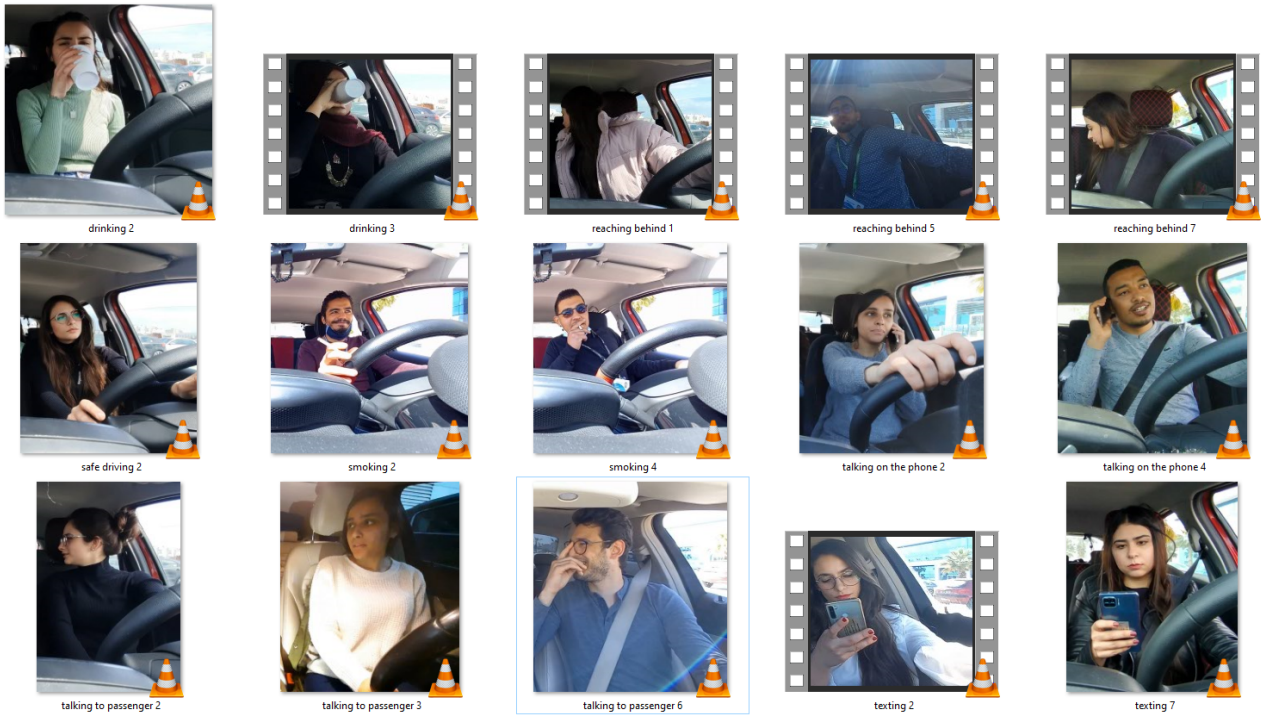


**Figure 3.19:** Échantillon des vidéos collectés

### 3.3.2 Préparation des données

Pour préparer les données collectées, nous avons commencé par la convention des vidéos collectées en trames à l'aide d'un script développé en python. Il enregistre dans un dossier les trames des vidéos d'une même classe avec un saut de 20 trames dans le but d'avoir un ensemble des données varié et éviter le problème de sur apprentissage. En fait, nous avons remarqué que chaque 20 trames successives d'une même vidéo représente pratiquement la même photo.

L'étape suivante était le nettoyage des images collectées pour chaque classe. Tout d'abord, nous avons supprimé les images adhérentes qui ne contiennent pas les caractéristiques de classe et nous avons supprimé aussi celles qui ont l'aire de causer des chevauchements entre les classes.

Après ces étapes de préparations des données nous avons obtenu une base de données bien équilibré contenant **42 830** images.

**Figure 3.20:** Une portion la base de données collectée et nettoyée

Une fois les données sont bien nettoyées, nous sommes passés à l'étape de division du jeu de données entre l'apprentissage et la validation.

Pour éviter le problème de fuite de données qui est dû à l'entrainement et la validation des modelés sur le même ensemble de données, il faut que les personnes qui vont figurer dans les images d'apprentissage ne doivent pas être présent dans les images de tests. Donc nous avons divisé les données entre l'entrainement et la validation en fonction des identifiants des conducteurs. Nous avons choisi les images de 4 conducteurs pour la validation et le reste pour l'entrainement.

Avant d'entamer la phase d'entrainement, les étapes suivantes ont été réalisées :

— Redimensionnement des images à une taille égale à 224×224, pour répondre aux exigences des modèles pré-entrainés utilisés.

— Création d'une liste contenant les images d'entrainement transformées à des tableaux avec la fonction 'array' de la bibliothèque Numpy.

— Création d'une liste contenant les labels : '0', '1', ... '6'. Cette liste a été ensuite codée avec

la fonction 'to_categorical' de la bibliothèque Keras. La fonction 'to_categorical' transforme le vecteur des labels à une matrice binaire.

### 3.3.3 Entrainement et évaluation

Après la préparation des données dans un format tel qu'un réseau de neurones convolutif peut les traiter, l'étape suivante est le choix du modèle.

#### 3.3.3.1 CNN "from scratch"

Lors de la première étape, une architecture d'un CNN a été configurée avec Keras, Comme le montre la figure 3.21.

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_4 (Conv2D)            (None, 224, 224, 32)      320
_________________________________________________________________
batch_normalization_4 (Batch (None, 224, 224, 32)      128
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 112, 112, 32)      0
_________________________________________________________________
conv2d_5 (Conv2D)            (None, 112, 112, 64)      18496
_________________________________________________________________
batch_normalization_5 (Batch (None, 112, 112, 64)      256
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 112, 112, 64)      36928
_________________________________________________________________
batch_normalization_6 (Batch (None, 112, 112, 64)      256
_________________________________________________________________
max_pooling2d_3 (MaxPooling2 (None, 56, 56, 64)        0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 56, 56, 128)       73856
_________________________________________________________________
batch_normalization_7 (Batch (None, 56, 56, 128)       512
_________________________________________________________________
flatten_1 (Flatten)          (None, 401408)            0
_________________________________________________________________
dense_2 (Dense)              (None, 128)               51380352
_________________________________________________________________
dense_3 (Dense)              (None, 7)                 903
=================================================================
Total params: 51,512,007
Trainable params: 51,511,431
Non-trainable params: 576
```

**Figure 3.21:** Architecture d'un CNN

Ce réseau se compose de deux blocs , le premier contient une couche de convolution avec une fonction d'activation ReLU, suivie par la "batch normalization" . Le deuxième est construit par une couche de convolution avec une fonction d'activation ReLU, suivie par la "batch normalization" et « Max Pooling». Chacun de ces deux blocs se répète 2 fois alternativement, puis nous trouvant une couche Flatten qui transforme l'ensemble de la matrice de la carte des caractéristiques regroupées

en un seule vecteur colonne. 2 couches complétement connectées ont été ajoutées. La dernière assure la tâche de classification avec sa fonction d'activation Softmax.

Le CNN a été entrainé sur les données d'entrainement prétraitées et validé avec les données de test définis dans la section précédente. L'entrainement a été fait sur 5 itérations en utilisant SGD comme algorithme d'optimisation (pas d'apprentissage égale à 0.001), et le critère LogLoss et précision comme une mesure de performance.

Cette première tentative de modélisation n'a pas donné des bons résultats et c'est attendu, puisque l'architecture de CNN utilisée est très simple et n'est pas suffisante pour la classification des images. Le tableau suivant montre les valeurs de précision et de LogLoss obtenues.

| | **LogLoss** | **Précision** |
|---|---|---|
| **Entrainement** | 1.4 | 38.4% |
| **Validation** | 2.08 | 25.4% |

**Tableau 3.7:** Résultats CNN

Pour améliorer les résultats, nous avons opté pour appliquer de l'apprentissage par transfert avec des architectures de CNN pré-entrainées.

#### 3.3.3.2 Approche fondée sur un seul modèle d'apprentissage par transfert

Comme c'est indiqué dans la section 2.6 : ResNet50, MobilNet, VGG16 et VGG 19 ont donné les meilleures performances sur la base de données "ImageNet". Alors nous avons appliqué de l'apprentissage par transfert avec ces quatre architectures en passant par les étapes suivantes :

1. Choix de l'algorithme d'optimisation :

   Selon l'étude faite dans la section 2.1, SGD et ADAM sont les deux algorithmes d'optimisation les plus rapides et les plus utilisés. Pour choisir l'un des deux, nous avons entrainé le modèle VGG16 avec les deux algorithmes, en ajoutant juste une dernière couche de classification "Dense Layer" ayant comme nombre de classes 7. Après l'entrainement des deux algorithmes pendant 30 itérations, nous avons déduit que ADAM a rendu l'apprentissage du modèle non stable alors que le SGD a permis un apprentissage progressif et stable. Donc nous avons utilisé

le SGD pour tous les modèles adoptés.

2. Ajout des couches supplémentaires :

   Pour chaque modèle pré-entrainé sur la base de données "ImageNet", nous avons utilisé sa partie de convolution qui permet d'extraire les descripteurs, et afin de l'adapter à notre cas d'utilisation, les couches suivantes ont été ajoutées :

   - "Average Pooling" : permet de réduire la taille en gardant la moyenne de chaque partie de l'image.
   - "Dropout" : permet de minimiser le sur-apprentissage "overfitting", en supprimant aléatoirement une partie des paramètres du réseau de neurones à chaque itération de l'apprentissage.
   - "Batch normalization" : normalise les entrées vers la couche suivante, ce qui permet un entraînement plus rapide et plus efficace.
   - "Dense" : c'est la dernière couche qui fait la classification avec une fonction d'activation softmax.

3. Entrainement des modèles :

   Cette étape consiste à entraîner chacun des modèles : VGG16 ,VGG19 , ResNet50 et MobilNet avec les données divisées entre entrainement et validation selon les identifiants des conducteurs. Un algorithme d'optimisation SGD a été utilisé avec un taux d'apprentissage égal à 0.001. Le nombre des itérations a été fixé à 80, et la taille du lot des données "Batch size" a été fixé à 32. Afin d'éviter le sur-apprentissage une fonction "Early stopping" a été utilisée, cette fonction surveille la valeur de LogLoss pour les données de validation et arrête l'apprentissage si la valeur de cette mesure ne s'améliore pas pendant 10 itérations consécutives.

4. Evaluation des modèles :

   Dès les premières itérations de l'entrainement, nous avons constaté que le modèle à fait un sur apprentissage (overfiting) sur les données de l'entrainement et il n'arrive pas à classifier les données de test. Les courbes de la figure suivante illustrent l'évolution des précisions et LogLoss d'entrainement et de validation en fonction des itérations, pour l'algorithme ResNet50.

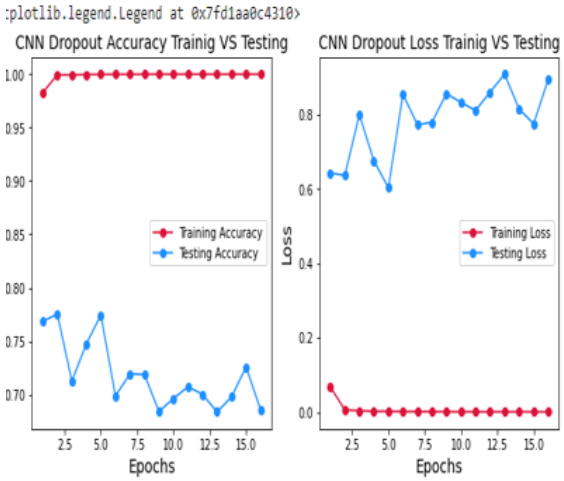


**Figure 3.22:** Overfiting

Pour examiner la cause de ce problème, nous avons effectué un feedback par rapport à l'étape de visualisation des données et nous avons constaté que malgré l'échantillonnage et le nettoyage que nous avons effectué sur les images collectées, nous trouvons des images très semblables et qui contiennent les mêmes caractéristiques. Donc c'est le problème majeur qui a causé l'Overfiting des modèles.

Pour faire face au problème d'overfitting, nous sommes revenus à l'étape de prétraitement et nettoyage des données. Afin d'éliminer le maximum possible les images semblables, nous avons effectué un sur échantillonnage manuelle de données. Et pour être sûr que nous avons des images bien distincts. Nous avons utilisé un algorithme qui détecte la similitude entre les images et il supprime celles qui ont un pourcentage de ressemblance supérieur à 30%.

Après ce deuxième prétraitement de données nous avons obtenu un nouveau jeu de données qui se compose de 8 950 images. Mais cette fois ci la base de données n'est plus équilibrée. En fait, la classe « Smoking » contient moins de données que le reste des classes puisqu'il est stimulé par 8 acteurs seulement qui sont tous des garçons. Pour surmonter ce problème, nous avons ajouté des images de filles qui fument pour enrichir la base de données. Ces images ont été de Google Image Search et quelques vidéos trouvées dans Youtube.

**Figure 3.23:** Portion de nouvelles images de classe « Smoking ».

Pour enrichir encore plus le jeu de données, nous avons décidé d'appliquer l'augmentation des données sur notre base.

L'augmentation des données a été réalisée en utilisant la méthode "ImageDataGenerator" de la bibliothèque Keras. Cette méthode nous permet de générer automatiquement 10 nouvelles images à partir de chaque image de jeu de donné en appliquant les types de l'augmentation illustré dans la figure 3.24 .

```
datagen = ImageDataGenerator(
    zoom_range = 0.1,
    rotation_range=0.15,
    width_shift_range=0.1,
    shear_range=0.4,
    height_shift_range=0.1,
    brightness_range=[0.2,1.2]
        )
```

**Figure 3.24:** Types d'augmentation des données appliqués.

5. Evaluation des modèles avec la nouvelle base de données : Une fois les données sont prêtes nous revenons à la phase d'entrainement et évaluation. Les courbes de la figure 3.25 illustrent l'évolution de la précision et LogLoss d'entrainement et de validation en fonction des itérations, pour l'algorithme ResNet50 entrainé avec la nouvelle base des données.

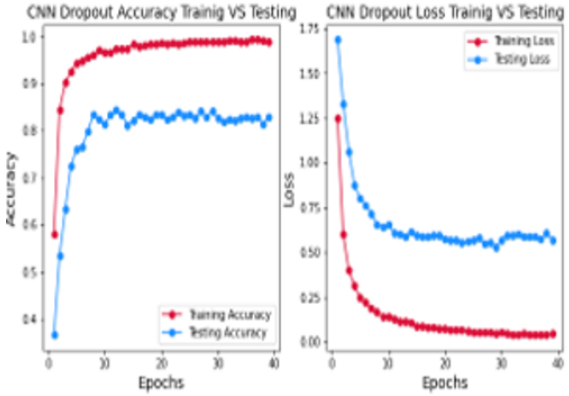


**Figure 3.25:** Allure des résultats obtenus

Les mesures de performance de chaque modèle pour les données de validation sont représentées dans le tableau suivant :

| | **Précision** | **LogLoss** |
|---|---|---|
| **MobileNet** | 0.87 | 0.42 |
| **ResNet50** | 0.92 | 0.31 |
| **VGG16** | 0.85 | 0.38 |
| **VGG19** | 0.73 | 0.53 |

**Tableau 3.8:** Résultats d'entrainement

Les matrices de confusion obtenues pour les quatre modèles sont illustrées dans la figure 3.26 :

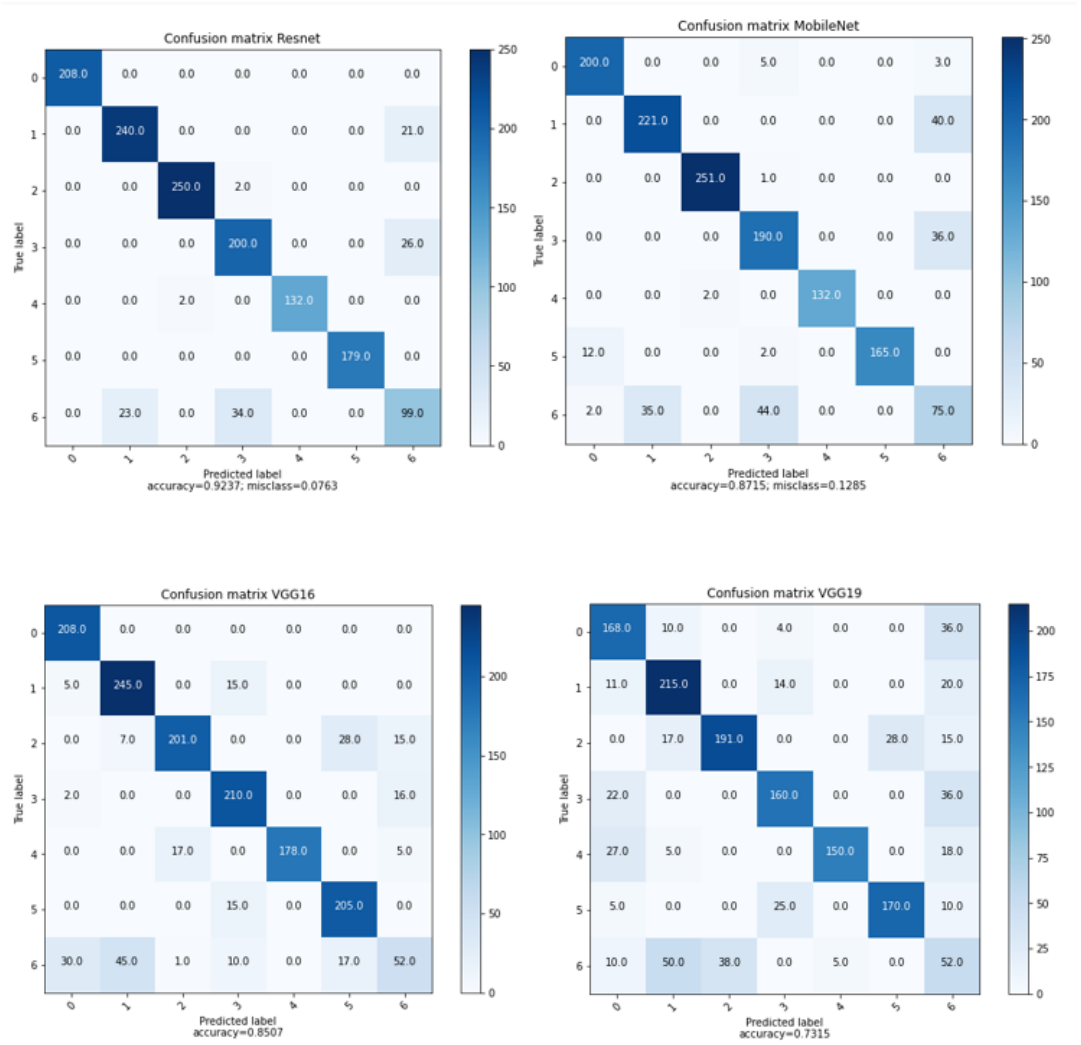


**Figure 3.26:** Les matrices de confusion des 4 modèles

Les architectures VGG16, ResNet50, MobilNet ont donné de meilleurs résultats que le CNN proposé précédemment, mais il existe une variance dans les performances par classe de chaque modèle. D'après les matrices de confusion, nous remarquons que les meilleures précisions par classe sont atteintes par des modèles différents, par exemple pour la classe "Talking to passenger", la meilleure précision est atteinte avec l'architecture MobilNet, alors que pour la classe "Texting" l'architecture ResNet50 a donné la meilleure précision de classification. Nous avons alors décidé de combiner les 3 modèles.

6. Combinaison des modèles :

Chaque modèle prend une image en entrée et calcule à sa sortie une liste de 7 probabilités d'appartenance. La classe de l'image d'entrée est celle ayant la probabilité maximale. Si nous supposons que : $P_{ij}$ la probabilité d'appartenance à la classe i, calculée par le modèle j ( j=1

pour le VGG16, j=2 pour le ResNet50, j=3 pour le MobilNet et j=4 pour le VGG19) La combinaison des modèles consiste à attribuer à chaque classe i une probabilité $Q_i$ tel que :

$$Q_i = \frac{P_{i1} + P_{i2} + P_{i3} + P_{ij4} - max(P_{i1} + P_{i2} + P_{i3} + P_{ij4}) - min(P_{i1} + P_{i2} + P_{i3} + P_{i4})}{4} \tag{3.1}$$

La soustraction du minimum et de maximum permet d'éviter les cas extrêmes. Cette combinaison donne aussi pour une image d'entrée une liste de 7 probabilités. La combinaison des modèles a amélioré les valeurs de la précision et de la LogLoss pour les données de validation tel que : précision=0.95 et LogLoss=0.12.

7. Test avec les modèles combinés : Pour le test, nous avons enregistré des vidéos en simulant les 7 classes avec des personnes qui n'apparaissent pas dans les données d'entrainement. Une alerte apparaît si une classe autre que "safe driving" est prédite pendant 4 images consécutives. Les résultats des tests de la méthode de combinaison sur les vidéos sont satisfaisants. La figure suivante montre quelques images de ces vidéos.

**Figure 3.27:** Images test 1.

La combinaison des modèles a amélioré les performances mais elle a augmenté le temps de traitement des images. En effet, en utilisant une caméra pour tester la classification avec l'un des modèles et la classification avec la combinaison, le nombre des images traitées par secondes (FPS) obtenu est :

— FPS=48 si la classification est faite avec le modèle ResNet50 seulement.

— FPS= 8 si la classification est faite avec la combinaison des modèles.

De plus, en testant sur des images autres que celles de la base de données, nous avons constaté qu'il y confusion entre la classe « Smoking » et la classe « Talking on the phone » et aussi entre la classe « Drinking » et la classe « Somking ». En fait, ces deux classes en des caractéristiques communes avec la classe « Smoking », la première c'est d'avoir un objet près de la bouche et la deuxième c'est la position de bras.

La figure 3.28 montre deux exemples de résultats de classifications erronés à cause la confusion entre classes. Pour résoudre les problèmes de confusion des classes et du temps de traitement, nous avons opté pour développer une deuxième approche plus optimale.

**Figure 3.28:** Résultats erronés

#### 3.3.3.3 Approche fondée sur une cascade de deux modèles d'apprentissage par transfert

Les objectifs de cette approche sont :

1. Minimiser le temps de traitement en utilisant un seul modèle.
2. Résoudre le problème de confusion entre classes, en séparant les classes impliqués dans ce problème.

L'architecture globale de cette approche est expliquée dans la figure suivante. Elle s'agit d'une cascade de deux classifieurs.

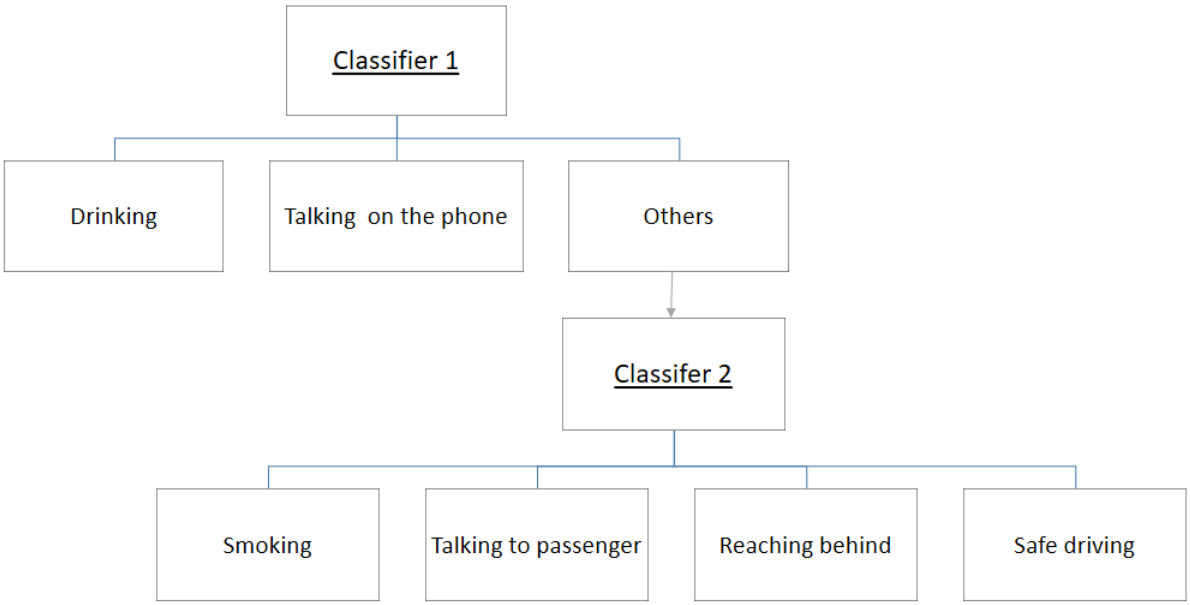


**Figure 3.29:** Architecture de l'approche fondée sur une cascade de deux modèles.

1. Prétraitement des données : Pour assurer cette nouvelle approche, nous avons besoin de revenir encore une fois à la phase de prétraitement des données. D'ailleurs nous devons diviser la base des données en deux ensembles du jeu de données :
   - Premier ensemble "Jeu de données1" : composé de 3 classes suivantes :
     - C0 : Drinking
     - C1 : Talking on the phone.
     - C3 :Others (C'est un mélange des données appartenant aux 5 classes restantes : "safe driving",Texting ,"reaching behind" , "Talking to passenger" et "Smoking")
   - Deuxième ensemble "Jeu de données 2" : composé de 4 classes suivantes :

— C0 : Smoking.

— C1 : Talking to passenger.

— C3 : Reaching behind.

— C4 : Safe driving.

2. Entrainement et validation :

Cette phase a été réalisée sur trois étapes :

1. Choix des modèles :

Pour avoir la possibilité de faire l'augmentation des données, deux architectures : "modèle 1"et "modèle2" ont été entrainées séparément.

— Modèle1 : est constitué par la partie de convolution de ResNet50 (pré-entrainé sur "ImageNet") avec les couches supplémentaires définies dans la section3.3.3.2. Les données d'apprentissage pour ce modèle sont les images de "jeu des données 1", alors le nombre de classes de sa dernière couche "Dense" doit être égal à 3.

— Modèle2 : est aussi constitué par la partie de convolution de ResNet50 avec les couches supplémentaires. Les données d'entrainement pour ce modèle sont les images de "Jeu des données 2", alors le nombre de classes de sa dernière couche "Dense" doit être égal à 5.

Nous avons choisi le "ResNet50" puisqu'il a donné les meilleurs résultats dans le tableau 3.8.

2. Entrainement :

Pour entrainer les deux modèles, un algorithme d'optimisation SGD a été utilisé avec un taux d'apprentissage égal à 0.001. Le nombre des itérations a été fixé à 100, et la taille du lot des données "Batch size" a été fixé à 32. Afin d'éviter le sur-apprentissage une fonction " Early stopping " a été utilisée.

3. Validation : Les résultats de validation de chaque modèle sont représentés par le tableau 3.9 :

| | **Précision** | **LogLoss** |
|---|---|---|
| **Modèle 1** | 0.96 | 0.19 |
| **Modèle 2** | 0.98 | 0.11 |

**Tableau 3.9:** Résultats des Modèle 1 et Modèle 2.

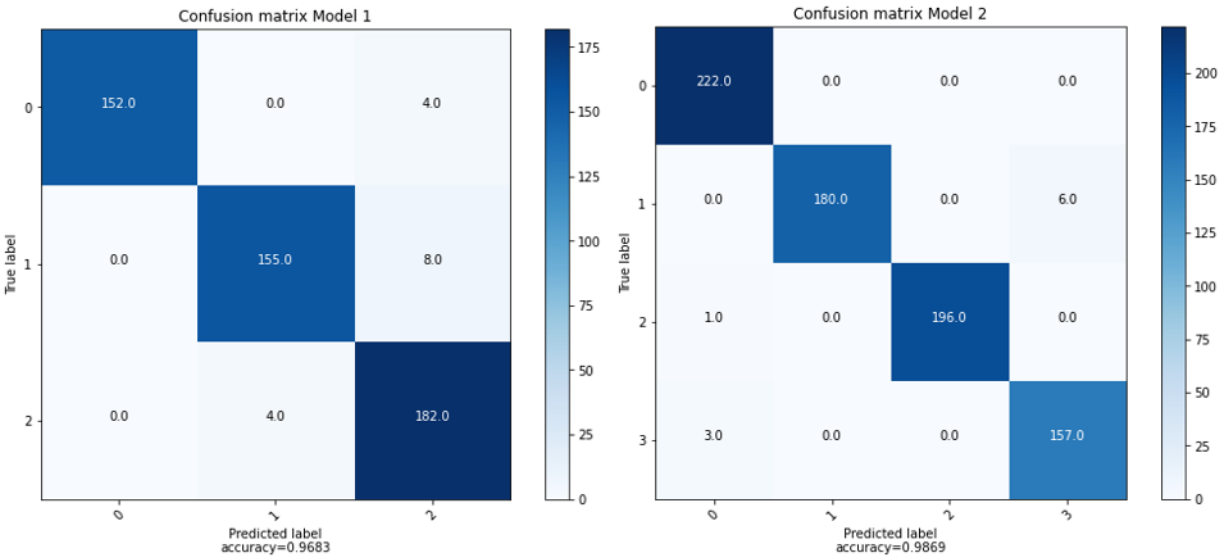


**Figure 3.30:** Matrices de confusion du modèle 1 et modèle 2.

Comme indiqué dans la figure 3.29, pour chaque image d'un conducteur, la classification avec cette architecture se fait en passant d'abord par le "modèle1" qui va classifier l'état du conducteur en "Talking on the phone" ou "Drinking" ou "other". Si la classe prédite est "other" alors l'image d'entrée passe au deuxième modèle, qui va l'assigner à l'une des classes : "safe driving", "smoking","drinking", "reachong behind" ou "talking to passenger". Cette approche a donné une précision égale à 0.97% et une LogLoss égale à 0.11 sur les mêmes données de validation de l'approche précédente, alors elle est plus performante.

3. Test :

Le Test de cette approche a été fait aussi sur les vidéos utilisées pour le test de la première approche. Si un état autre que "safe driving" est prédit pendant 4 images successives, une alerte sera affichée. Les résultats obtenus sont montrés dans les figures suivantes :

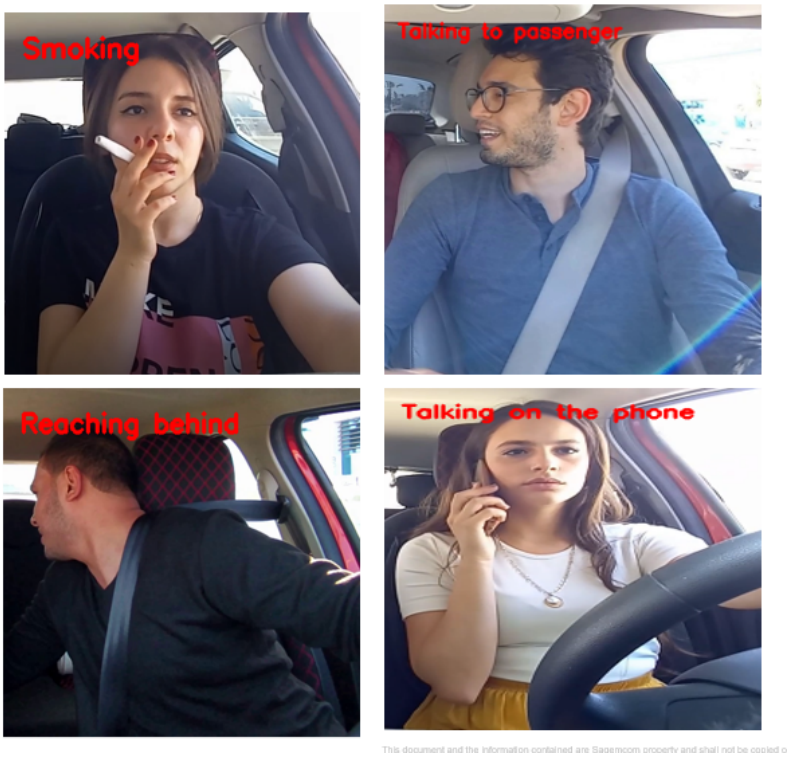


**Figure 3.31:** Résultats 1 du modéle cascade.

**Figure 3.32:** Résultats 2 du modéle cascade.

Cette approche est plus rapide que la première, en effet le FPS est égale à 42, en plus le problème de confusion des classes a été réduit.

## 3.4 La reconnaissance faciale du conducteur

Comme mentionné dans la section 2.7, le processus de la reconnaissance faciale du conducteur se compose de 3 principaux étapes successives et dépendante l'une de l'autre . la figure suivante montre les étapes du processus :

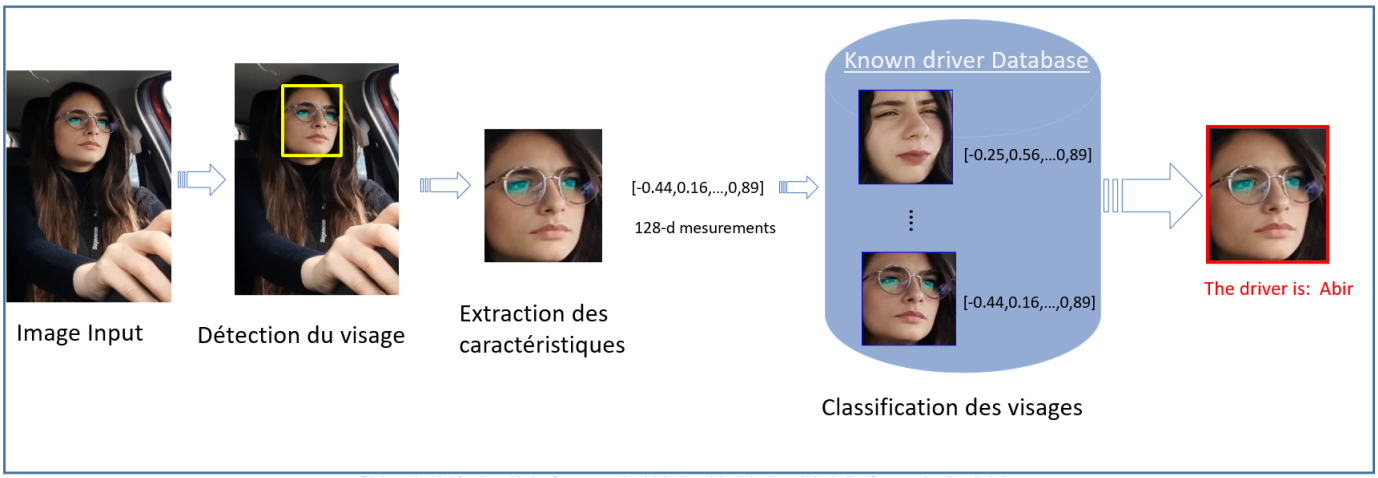


**Figure 3.33:** Architecture de modèle reconnaissance faciale.

### 3.4.1 Préparation de la base de données

Avant d'entamer la modélisation du modèle de reconnaissance faciale du conducteur, nous devons tout d'abord préparer la base de données des conducteurs reconnues. Cette base de données va nous servir à test et valider le modèle de reconnaissance faciale du conducteur. La base de données est composée par un nombre de dossiers qui est égale au nombre des conducteur connus. Chacun de ces dossiers est nommée par le nom du conducteur correspondant.

Pour construire cette base de données, nous avons utilisé les images des 6 conducteurs collectées et augmentées dans la partie « classification de l'image du conducteur ». La figure suivante montre le dossier de la base de données préparée :

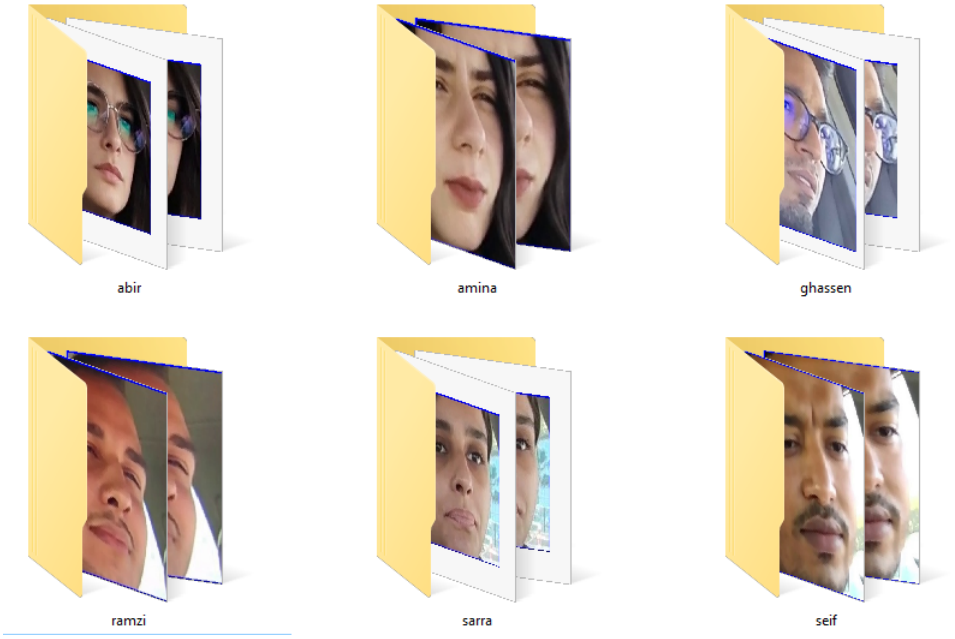


**Figure 3.34:** Base de données .

### 3.4.2 Détection de visage

La première étape dans la reconnaissance faciale est la détection des visages, c'est à dire l'ensemble de données en entrée pour cette étape sont les images déjà préparées. Une image est représentée en tant que tableau Numpy de trois dimensions :

— Hauteur : nombre des lignes.

— Largeur : nombre des colonnes.

— Profondeur : nombre des canaux. Le cas le plus utilisé est celui du RVB (Rouge, Vert, Bleu).

Chaque pixel d'un canal est codé sur 8 bits, ce qui lui attribue 256 valeurs possibles (de 0 à 255). Dans le cas de l'espace calorimétrique RGB, un ensemble (R, G, B) est associé à chaque pixel où chaque valeur est dans la plage de 0 à 255. De ce fait, la probabilité que des couleurs existent dans un pixel d'une image RGB est égale à 256 x 256 x 256 = 1677216 couleurs possibles.

- **Lecture de l'image :** Comme montré dans la figure 3.2, après avoir chargé chaque image à l'aide de la fonction "open" de la bibliothèque PIL (Python Imaging Library), on doit la convertir de BGR en RGB et puis, nous devons la convertir en tableaux de valeurs (fichier numpy) à l'aide de la fonction asarray.

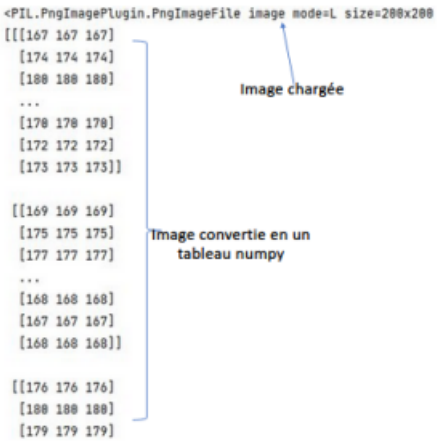


**Figure 3.35:** Conversion d'une image en un tableau.

Maintenant ces tableaux numpy sont prêts pour la détection des visages, l'étape suivante c'est la comparaison entre les détecteurs de visages les plus répandus en utilisant une base de données préparée précédemment.

Le tableau 3.10 illustre la comparaison des approches de détection faciale expliqués dans la section 2.7.1 en se référant à des métriques qui sont la précision , mAP et Temps d'exécution.

| | **Précision** | **mAP** | **Temps d'exécution** |
|---|---|---|---|
| **V-J** | 0.75 | 0.65 | 0.15 s |
| **HOG** | 0.97 | 0.87 | 0.19s |
| **MMOD** | 0.92 | 0.78 | 0.21s |

**Tableau 3.10:** Comparaison des approches de détection faciale

Les résultats de tableau comparatif montrent que l'algorithme optimal est HOG. Donc, utilisons l'algorithme HOG pour le modèle de reconnaissance faciale.

### 3.4.3 Extraction des caractéristiques

En utilisant la bibliothèque face_recognition, VGG-19 prend comme entrée l'image du visage de la personne résultant du modèle de détection de visage avec une dimension fixe 224x224x3, la convertit en une image RGB et produit un vecteur de 128 caractéristiques qui représentent les caractéristiques les plus importantes d'un visage.

Dans l'apprentissage automatique, ce vecteur est appelé intégration ou embedding en anglais. Il a pris cette nomination parce que toutes les informations importantes d'une image sont intégrées dans ce vecteur. Idéalement, les vecteurs de faces similaires sont également similaires.

Comme résultat d'extraction de features, nous avons obtenu pour chaque image un vecteur (nommé face_encoding) de dimension 128 contenant les caractéristiques du visage.

```
face_encoding [-0.09005922  0.01463923 -0.00593187 -0.06629957 -0.07103433 -0.03860251
 -0.04829317 -0.08879665  0.17537707 -0.07686696  0.10850047  0.01530737
 -0.16510212  0.04270584  0.01277056  0.08492394 -0.11371204 -0.10837359
 -0.06797922 -0.06880094 -0.00718763  0.05611361  0.06096867  0.16166313
 -0.04812546 -0.31136605 -0.10136671 -0.13876659 -0.06482106 -0.05432185
  0.08594099  0.01615704 -0.15938143 -0.00041566  0.031364   -0.01219718
 -0.10898597 -0.07925229  0.29057842  0.03437383 -0.19103196 -0.02177278
```

**Figure 3.36:** Vecteur résultant d'extraction des features.

### 3.4.4 Classification des visages

À ce stade-là, nous avons pu extraire les features des visages de notre base de données. Donc, nous nous retrouvons maintenant face à un problème de classification. Puisque les classifieurs utilisés dans cette tâche sont essentiellement des classificateurs binaires, nous avons utilisé l'approche OneVsRest (OVR) pour les entraîner à plusieurs classes. Donc, nous avons comparé des algorithmes de classification expliqués dans la section 2.7.3 en utilisant les 6 classes (conducteurs) de la base de données et nous avons choisi le plus optimal.

L'évaluation des modèles de classification expliqué dans la section 2.7.3 est faite à l'aide des matrices de confusion illustrées dans la figure 3.37 qui nous a aidé à calculer des métriques de performance qui sont la précision et le rappel.

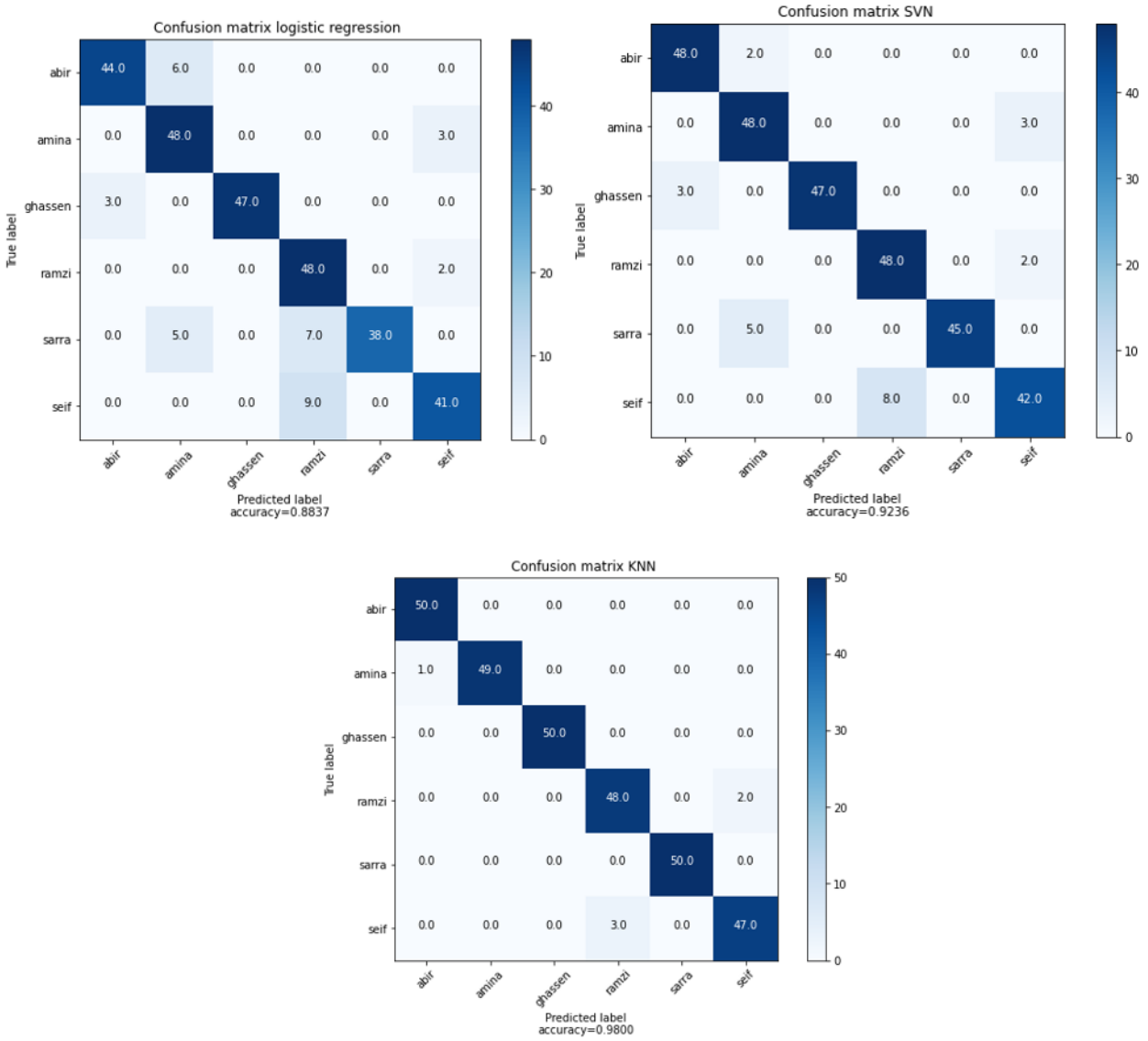


**Figure 3.37:** Matrice de confusion des 3 modèles

| Algorithmes | Précision | mAP |
|---|---|---|
| **Logistic Regression(k=6)** | 0.88 | 0,92 |
| **KNN(k=6)** | 0.98 | 0.94 |
| **SVM (rbf)** | 0.92 | 0.89 |

**Tableau 3.11:** Comparaison entre les scores trouvés

D'après les résultats d'évaluation des modèles présentés tableaux ci-dessus, nous pouvons déduire que l'algorithme le plus précis et le plus performant pour la classification des visages des conducteurs est l'algorithme de classification KNN.

En récapitulant, le modèle final de reconnaissance faciale du conducteur utilise l'algorithme HOG pour la détection de visage , VGG19 pour l'extraction de features et enfin KNN pour la classification et l'identification du conducteur . La figure suivante montre quelques résultats obtenus par le modèle de reconnaissance faciale du conducteur proposé.

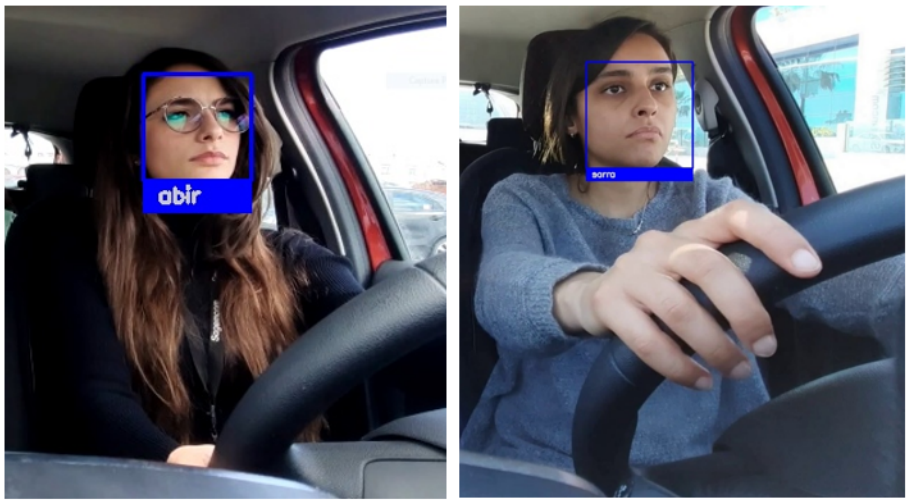


**Figure 3.38:** Résultats de la reconnaissance faciale du conducteur

## 3.5 L'intégration des différentes parties du système de surveillance de l'intérieur de la voiture

Après avoir bien préparé, modélisé et évalué chacune des trois parties de notre projet, il faut maintenant les intégrer ensemble afin d'avoir un seul modèle qui contient tous les éléments nécessaires pour une bonne surveillance de l'intérieur de voiture.

Les figures suivantes montrent quelques résultats de test obtenus en appliquant le modèle final de surveillance de l'intérieur de voiture :

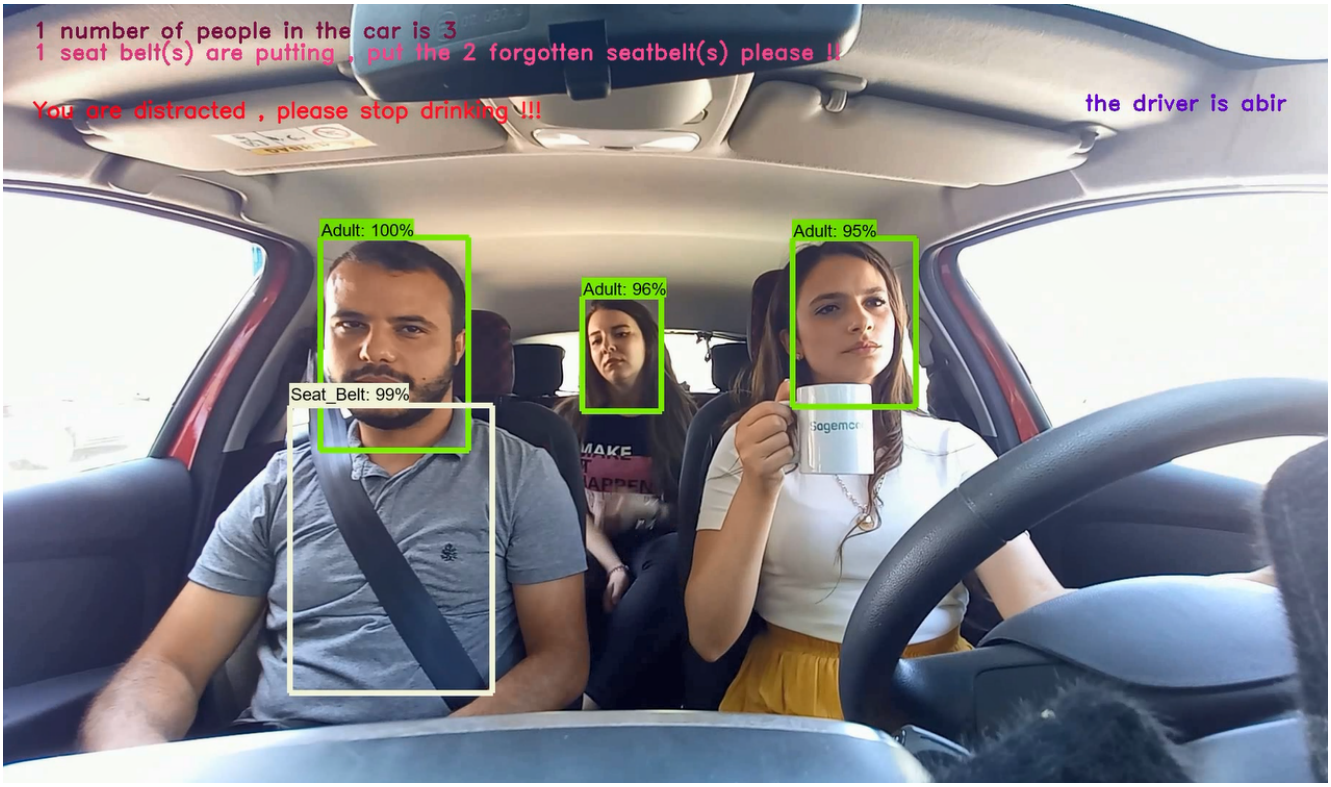


**Figure 3.39:** Résultat 1

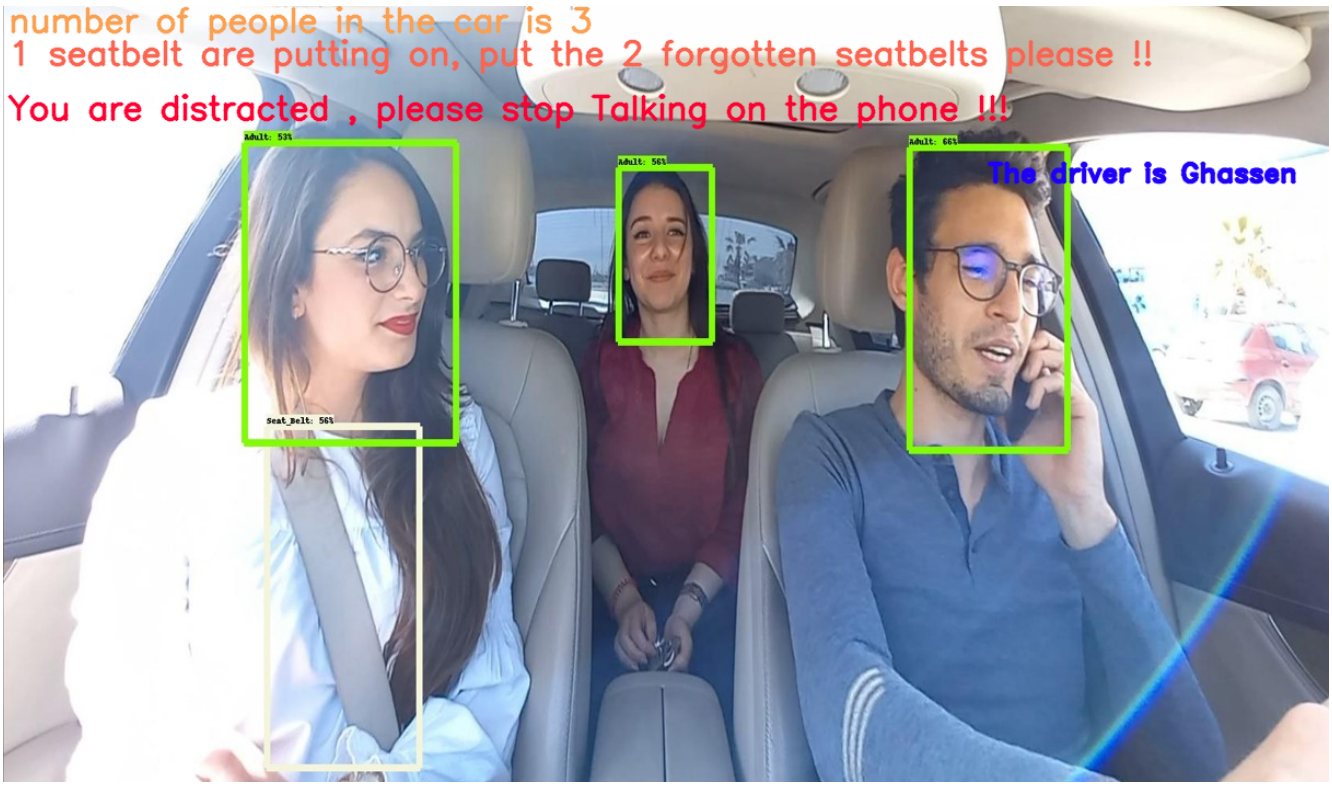


**Figure 3.40:** Résultat 2

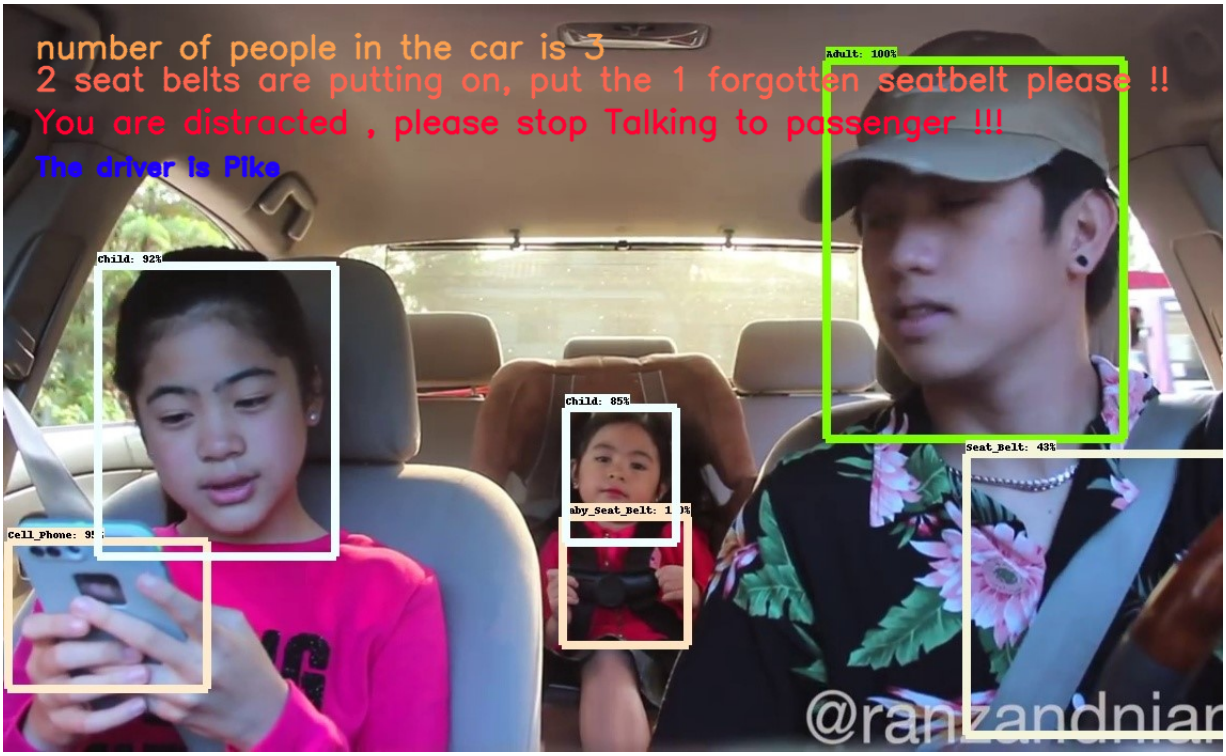


**Figure 3.41:** Résultat 3

## 3.6 Déploiement de la solution développée

Comme indiqué dans les parties précédentes, les modèles d'intelligence artificielle et les techniques de vision par ordinateur optés pour le développement de notre système de surveillance de l'intérieur de voiture nécessitent de grandes capacités de calcul et de mémoires et des processeurs graphiques puissants pour qu'ils puissent fonctionner, c'est pourquoi ce type de projet est généralement utilisée comme un traitement centralisé sur des serveurs ou dans le cloud. Mais le fait d'envoyer les données vers le serveur pour les traiter puis récupérer les résultats introduit généralement un temps de latence.

Par conséquent pour les applications qui nécessitent des traitements en temps réel, il vaut mieux analyser les données dès leur acquisition, ceci peut être réalisé avec les systèmes embarqués.

De ce fait, l'équipe innovation de SAGEMCOM a décidé d'utiliser une carte embarquée pour le déploiement et la centralisation de ces différentes solutions développées dans le domaine de la vision par ordinateur.La carte Nvidia jatson nano a été sélectionnée pour le portage des algorithmes et des techniques conçues.

**Figure 3.42:** Architecture extérieure de la carte Nvidia Jetson Nano.

[51]

Le kit NVIDIA Jetson Nano [52] est un petit ordinateur puissant avec le système d'exploitation Ubuntu qui permet d'exécuter plusieurs réseaux neuronaux en parallèle pour des applications telles que la classification d'images, la détection d'objets, la segmentation et le traitement de la parole et elle est compatible avec plusieurs plateformes d'intelligence artificielle comme Tensorflow, Caffe, PyTorch, MXNet. Ses spécifications techniques sont illustrées dans le tableau suivant :

| | |
|---|---|
| **Processeur (CPU)** | Quad-core ARM Cortex-A57 cadencé à 1.43 GHz |
| **Processeur graphiques (GPU)** | Nvidia Maxwell avec 128 coeurs CUDA |
| **Mémoire** | 4 Go LPDDR4 |
| **Stockage** | 16 Go eMMC 5.1 |
| **Enregistrement vidéo** | 4K avec FPS=30 |
| **Lecture vidéo** | 4K avec FPS=60 |
| **Connectivité** | Ethernet |
| **Affichage** | HDMI 2.0, eDP 1.4, DP 1.2 (deux simultanément) |
| **Ports USB** | 1x USB 3.0, 3x USB 2.0 |

**Tableau 3.12:** Les spécifications techniques de la carte Nvidia Jetson Nano.

Avant l'intégration de la solution dans la carte, nous devons d'abord la configurer et bien préparer son environnement de travail. Mais Bien qu'il s'agit d'un ordinateur puissant, la configuration de la carte Nvidia Jetson Nano nécessite la réalisation du plusieurs étapes :

1. Téléchargement de l'image du système d'exploitation basé sur Ubuntu : 'Jetpack 4.2' de NVIDIA
2. Ecriture de cette image sur une carte MicroSD.
3. Insertion de la carte MicroSD dans le kit Jetson Nano.
4. Démarrage de la carte avec la microSD . Cette étape nécessite des éléments tel que : Écran HDMI ,Clavier + souris USB , une alimentation microSD 5 V 2,5 A , Connexion réseau par un câble Ethernet.
5. Mise à jour du système et suppression des programmes pour économiser de l'espace.
6. Installation des packages système, des prérequis et les dépendances au niveau du système.
7. Installation d'une version récente de l'outil de compilation Cmake, puisqu'il est nécessaire pour compiler les programmes basés sur OpenCV.
8. Installation de la version 3 de Python, car le système d'exploitation de Jetson Nano contient par défaut Python2 et les algorithmes sont développés avec Python3.
9. Création d'un environnement virtuel Python, pour avoir un milieu spécifique au projet dans lequel nous installons toutes les bibliothèques nécessaires.
10. Installation de la version de CUDA compatible avec la carte Jetson Nano pour pouvoir utiliser son GPU.
11. Installation du compilateur Protobuf pour augmenter les performances de TensorFlow.
12. Installation de Tensorflow, OpenCv, Keras, Numpy, Scipy, Dlib et leur prérequis, dans l'environnement virtuel.
13. Implémentation des algorithmes du système de surveillance de l'intérieure de la voiture.
14. Test des algorithmes développés sur la carte Nvidia Jetson Nano.

### 3.6.1 Evaluation des performances de la carte

Les principales métriques de l'évaluation des performances des algorithmes OMS portés sur la catre Nvidia sont liées au comportement temporel. Les autres classes de métriques d'évaluation des

performances sont la consommation d'énergie et la capacité de la mémoire utilisée par l'application.

Dans ce projet nous nous sommes focalisés spécialement sur le comportement temporel car il s'agit des techniques de détection et classification en temps réel donc le facteur temps est le facteur le plus important. Pour atteindre les performances optimales, nous avons optimisé les paramètres et les hyper métriques des algorithmes en fonction des caractéristiques de la carte Nvidia nano jatson.

Les FPS obtenus avec cette carte sont plus faibles que ceux obtenus avec l'ordinateur tel que :

— FPS=14 pour l'algorithme de détection des objets.

— FPS=35 pour l'algorithme de classification de l'état de distraction du conducteur.

— FPS=26 pour l'algorithme de reconnaissance faciale du conducteur.

— FPS = 11 pour l'algorithme qui intègre les 3 solution ensemble

Malgré cette faible diminution des FPS, la carte Nvidia Jetson Nano permet d'exécuter les algorithmes proposés en temps réel.

## Conclusion

Dans ce chapitre nous avons abordé tous les aspects expérimentaux de ce projet, nous avons commencé par détailler les étapes de la mise en œuvre de chaque modèle séparément, ces étapes ont été mises en valeur à travers les tests effectués, et nous avons terminé le chapitre avec la partie déploiement de la solution dans un kit de développement Nvidia Jetson Nano.

# Conclusion générale

L'intelligence artificielle, la technologie la plus à la mode, s'invite de plus en plus dans notre vie. Celle-ci permet de doter d'intelligence plusieurs domaines. Les voitures autonomes représentent l'innovation la plus spectaculaire de l'intelligence artificielle.

C'est dans cette optique que ce stage de fin d'études qui s'est déroulé au sein de l'entreprise SAGEMCOM Software & Technologies s'impose. Il consiste à développer un système de surveillance de l'intérieur de voiture basé sur l'intelligence artificielle et plus précisément sur la vision par ordinateur.

Pour la réalisation de ce projet, nous avons commencé par la compréhension du contexte du projet ainsi que la méthode de travail à suivre. Ensuite, nous avons présenté les approches les plus utilisées dans le cadre de la thématique et nous avons opté pour l'approche CNN de l'apprentissage profond puisqu'elle est la plus sophistiquée et la plus adéquate à la réalisation des objectifs du projet. A la suite de quoi, il a fallu collecter et préparer les bases de données pour chaque partie de projet. Puis, nous avons développé et intégré les trois modèles, le premier fait la détection des objets présents à l'intérieur de voiture, le deuxième permet de classifier l'état de distraction du conducteur et le troisième assure la reconnaissance faciale du conducteur, toute en décrivant la mise en œuvre et les résultats de chaque modèle proposé. Finalement, nous avons déployé le système de surveillance de l'intérieur de voiture dans la carte embarquée Nvidia jatson nano.

Il est important de noter que les objectifs de ce projet ont été atteints et les résultats obtenus sont très satisfaisants. Néanmoins, comme toute solution dans le domaine des technologies de l'information et la communication, ce projet présente des pistes d'amélioration. Donc en matière de perspectives, nous comptons enrichir la base de la première partie du projet de données par l'ajout des données d'autres objets qui peuvent exister dans la voiture dans la partie détection d'objet. Nous pouvons aussi améliorer la partie classification de l'état de distraction du conducteur par l'ajout de tous les comportements possibles du conducteur.

Pour conclure, Le travail dans le cadre de ce PFE était d'une importance considérable dans la mesure où il nous a servi comme portail vers le monde professionnel et la vie en entreprise. D'un point de vue technique, il nous a permis de mettre en œuvre les acquis théoriques que nous avons appris tout le long de notre cursus universitaire et de les enrichir ainsi que l'approfondissement de connaissance dans le domaine de vision par ordinateur et apprentissage approfondi.

# Bibliographie


[1] *SAGEMCOM.* adresse : https://www.sagemcom.com/fr.

[2] *NHTSA.* adresse : https://one.nhtsa.gov/portal/site/NHTSA/menuitem..

[3] *CRISP.* adresse : https://www.proglobalbusinesssolutions.com/six-steps-in-crisp-dm-the-standard-data-mining-process/.

[4] *P.Isola, J.-Y.Zhu,T.Zhou,and A.A.Efros,* Imagez-to-image translation with conditional adversarial networks,in Proceedings of the IEEE conference on computer vision and pattern recognition, pp.1125–1134, 2017.

[5] *M.Aparicio IV,D.S.Levine,and W.S.McCulloch*, Why are neural networks relevant to higher cognitive function," Neural networks for knowledge representation and inference, pp.1–26, 1994.

[6] *B.Poole, J.Sohl-Dickstein, and S. Ganguli*, Analyzing noise in autoencoders and deep networks, arXiv preprint arXiv :1406.1831, 2014.

[7] *B.C.Csáji*, Approximation with artificial neural networks,"Faculty of Sciences,Etvs Lornd University, Hungary, vol. 24, p. 48, 2001.

[8] *M.H.Hassoun et al.* Fundamentals of artificial neural networks. MIT press, 1995.

[9] *Andrej Karpathy yes you should understand backprop.* adresse : https://medium.com/@karpathy/yes-you-should-understand-backpropˆe2f06eab496b.

[10] *Andrej Karpathy yes you should understand backprop.* adresse : https://medium.com/@karpathy/yes-you-should-understand-backprop.e2f06eab496.

[11] *H.Kuseto gullari and T.Celik,* Real time vehicle detection and driver warning system using computer vision,2010.

[12] *A.Cheddad, H.Kusetogullari, and H.Grahn,* Object recognition using shape growth pattern,in Proceedings of the 10th International Symposium on Image and Signal Processing and Analysis.IEEE, 2017, pp. 47–52.

[13] *N. Sebe, I. Cohen, A. Garg, and T. S. Huang*, Machine learning in computer vision. Springer Science Business Media, 2005, vol. 29.

[14] *CNN.* adresse : https://qiita.com/Rowing0914/items/e815ca24427874030526.

[15] *V. Dumoulin and F. Visin.* adresse : Aguidetoconvolutionarithmeticfordeeplearning , arXivpreprintarXiv:1603.07285,2016..

[16] *FAIZAN SHAIKH deep learning vs. machine learning – the essential differences you need to know !* Adresse : https://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/.

[17] *SOFTMAX.* adresse : https://towardsdatascience.com/softmax-activation-function-explained-a7e1bc3ad60.

[18] *RCNN.* adresse : https://makina-corpus.com/blog/metier/2020/extraction-dobjets-pour-la-cartographie-par-deep-learning-choix-du-modele.

[19] *Faster RCNN.* adresse : https://www.researchgate.net/publication/324903264_Multi-scale_object_detection_in_remote_sensing_imagery_with_convolutional_neural_networks.

[20] *SSD.* adresse : https://www.mdpi.com/2076-3417/10/20/7301/pdf.

[21] *YOLO.* adresse : https://towardsdatascience.com/yolo-you-only-look-once-real-time-object-detection-explained-492dc9230006..

[22] *YOLOv4.* adresse : https://medium.com/visionwizard/yolov4-version-0-introduction-90514b413ccf(yolov4).

[23] *IOU.* adresse : https://www.researchgate.net/figure/Intersection-over-Union-IOU-calculation-diagram_fig2_335876570.

[24] *K.Simonyan and A.Zisserman*, Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv :1409.1556, 2014.

[25] *H.Qassim, A.Verma, and D.Feinzimer*, Compressed residual-vgg16 cnn modelfor big data places image recognition, in 2018 IEEE 8th Annual Computing and Communication Workshop and Conference(CCWC), pp.169–175,IEEE, 2018.

[26] *VGG16.* adresse : https://towardsdatascience.com/step-by-step-vgg16-implementation-in-keras-for-beginners-a833c686ae6c.

[27] *Architecture VGG16.* adresse : https://towardsdatascience.com/architecture-comparison-of-alexnet-vggnet-resnet-inception-densenet-beb8b116866d.

[28] *Arna Fariza ; Mu'arifin ; Agus Zainal Arifin .Understanding ResNet50 architecture.* adresse : https://iq.opengenus.org/resnet50-architecture/..

[29] *Review : MobileNetV1 — Depthwise Separable Convolution (Light Weight Model).* adresse : https://towardsdatascience.com/review-mobilenetv1-depthwise-separable-convolution-light-weight-model-a382df364b69.

[30] *ImageNet.* adresse : https://keras.io/api/applications.

[31] *VJ.* adresse : https://towardsdatascience.com/understanding-face-detection-with-the-viola-jones-object-detection-framework-c55cc2a9da14.

[32] *HOG.* adresse : https://fr.wikipedia.org/wiki/Histogramme_de_gradient_orient/C3/A9.

[33] *HOG architecture.* adresse : http://gensoft.pasteur.fr/docs/icc/xe_2016/en/ipp/common/ipp_manual/GUID-83B0EE35-E5EF-4C23-96A4-F0918DFA826B.htm.

[34] *MMOD.* adresse : https://maelfabien.github.io/project/face-detection_proj/#1-a-bit-of-theory.

[35] *Face recognition.* adresse : https://ichi.pro/fr/comment-creer-un-modele-de-reconnaissance-faciale-a-l-aide-de-facenet-keras-278613466321649.

[36] *KNN.* adresse : https://towardsdatascience.com/face-recognition-using-deep-learning-b9be73689a23.

[37] *SVM.* adresse : https://www.spiele-schenken.top/products.aspx?cname=simple+svm+implementation+python&cid=28.

[38] *Python.* adresse : https://www.python.org/.

[39] *PyCharm.* adresse : https://www.jetbrains.com/pycharm/.

[40] *Google Colab.* adresse : https://colab.research.google.com/notebooks/intro.ipynb.

[41] *LabelImg.* adresse : https://github.com/tzutalin/labelImg.

[42] *Roboflow.* adresse : https://roboflow.com/.

[43] *TensorFlow.* adresse : https://www.lebigdata.fr/tensorflow-definition-tout-savoir.

[44] *Keras.* adresse : https://www.ionos.com/digitalguide/online-marketing/search-engine-marketing/what-is-keras/.

[45] *OpenCV*. adresse : https://opencv.org/about/.

[46] *NumPy*. adresse : https://numpy.org/.

[47] *SciPy*. adresse : https://informatique-python.readthedocs.io/fr/latest/Cours/science.html.

[48] *Dlib*. adresse : https://fr.wikipedia.org/wiki/Histogramme_de_gradient_orientC3A9.

[49] *CUDA*. adresse : https://towardsdatascience.com/installing-tensorflow-with-cuda-cudnn-and-gpu-support-on-windows-10-60693e46e781.

[50] *Open Image*. adresse : https://opensource.google/projects/open-images-dataset.

[51] *NVIDIA jatson nano*. adresse : https://www.tangytip.com/all-about-jetson-nano-4-gb-developer-kit/.

[52] *NVIDIA jatson nano*. adresse : https://developer.nvidia.com/embedded/jetson-nano-developer-kit.